Lecture 6

CME 295: Transformers & Large Language Models





Afshine Amidi & Shervine Amidi



Stanford University

ICME



Transformers & Large Language Models

Motivation

Retrieval

Tool calling

Harness engineering

Coding agents

Discussion

Copy/pasting code in 2023…

2023

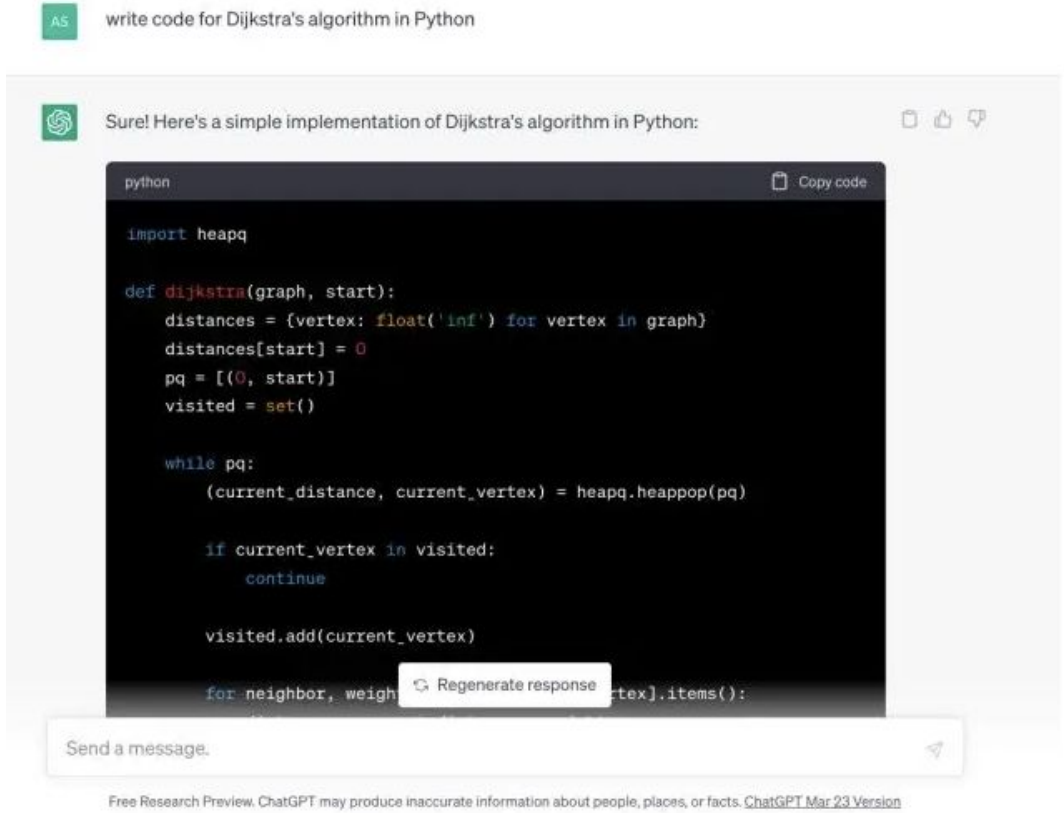


Image from Beebom.

...autocomplete (tab tab tab) within the IDE in 2024...

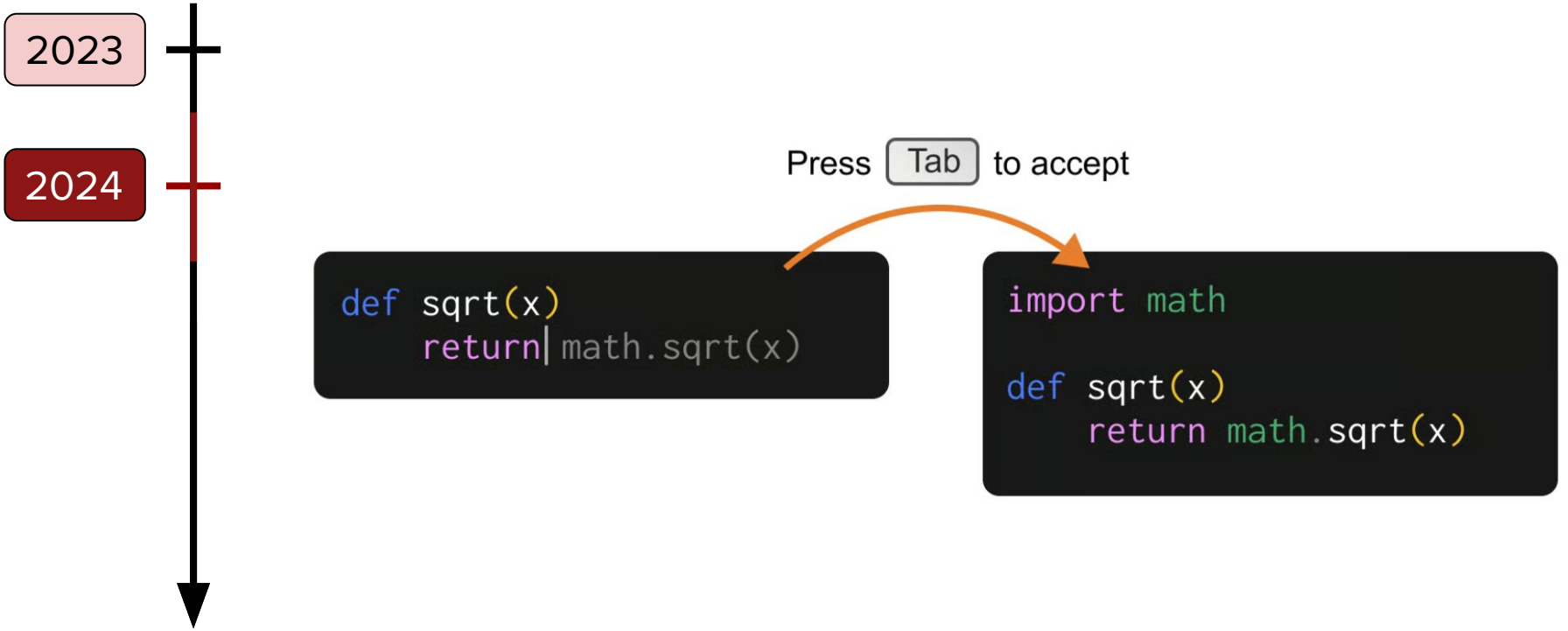


Image from Double.

...full tasks in 2025...

2023

2024

2025

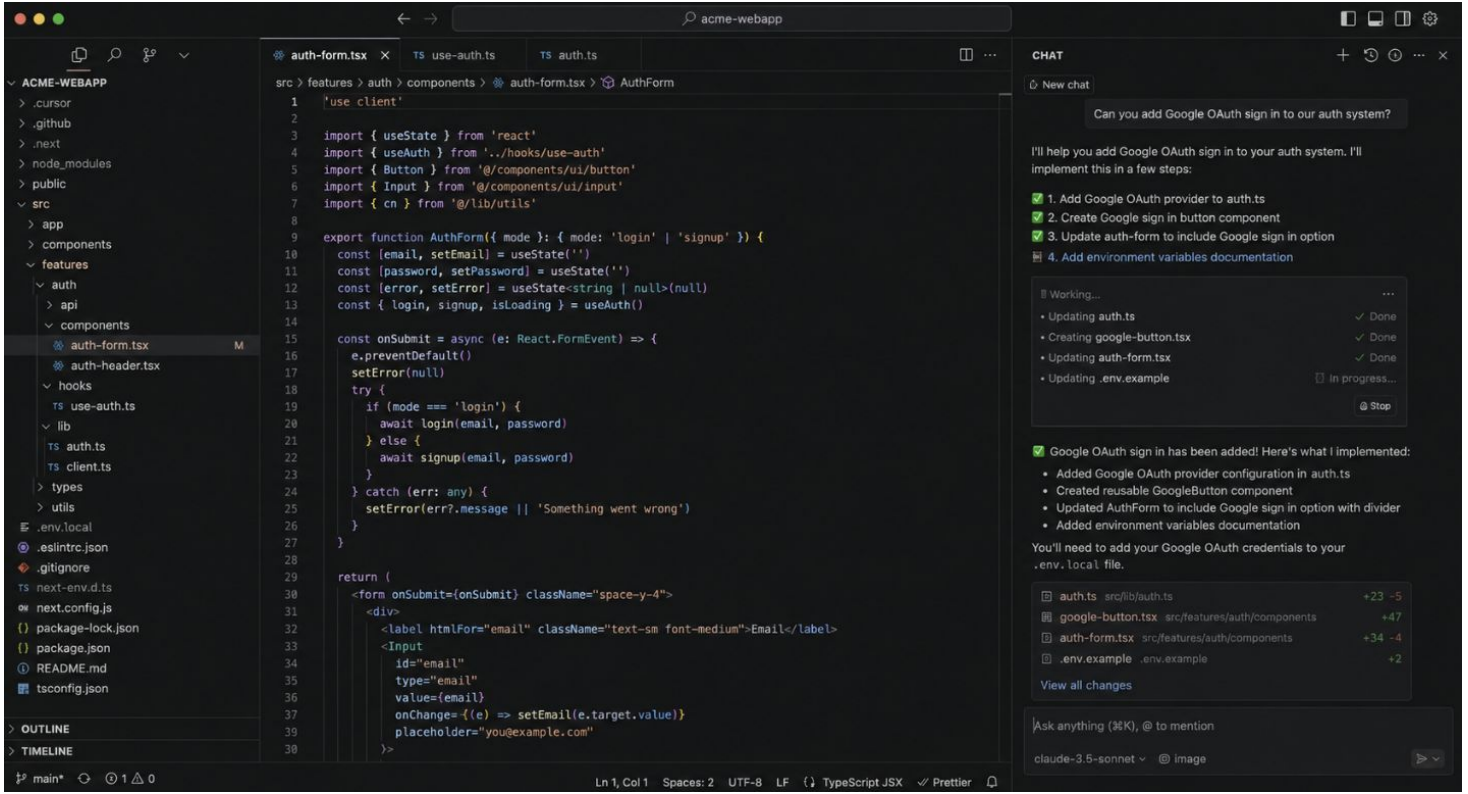
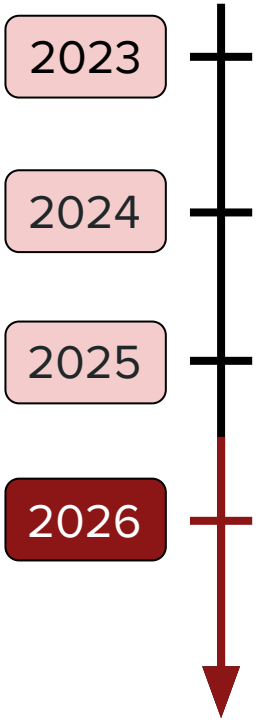


Image generated by ChatGPT, which represents Cursor and its chat interface on the right.

...and agentic behavior in 2026 (and beyond?)!



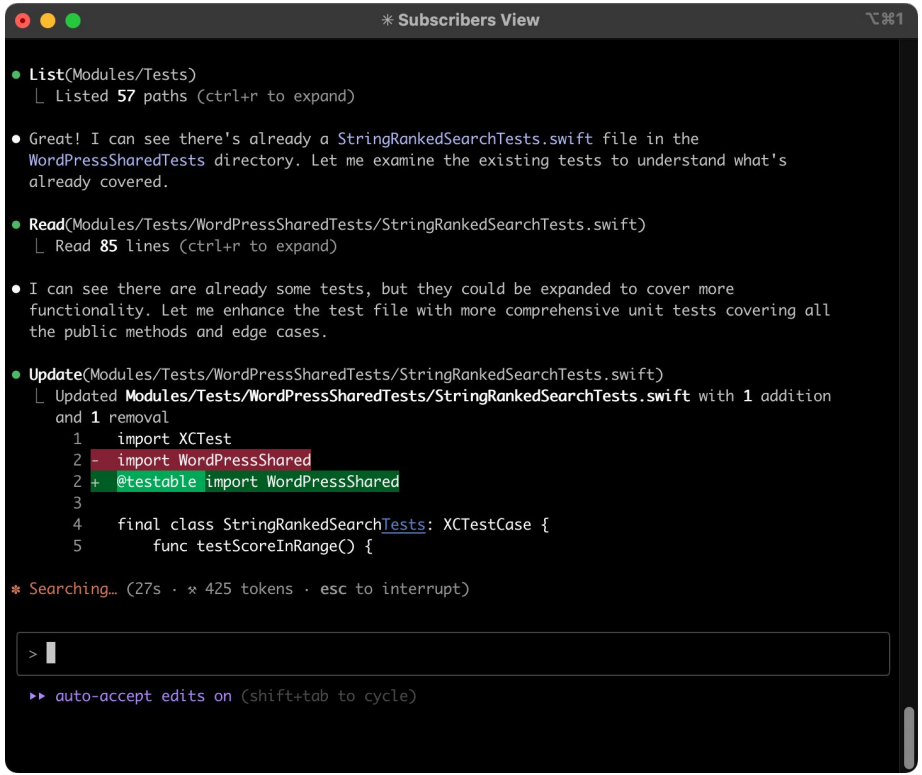
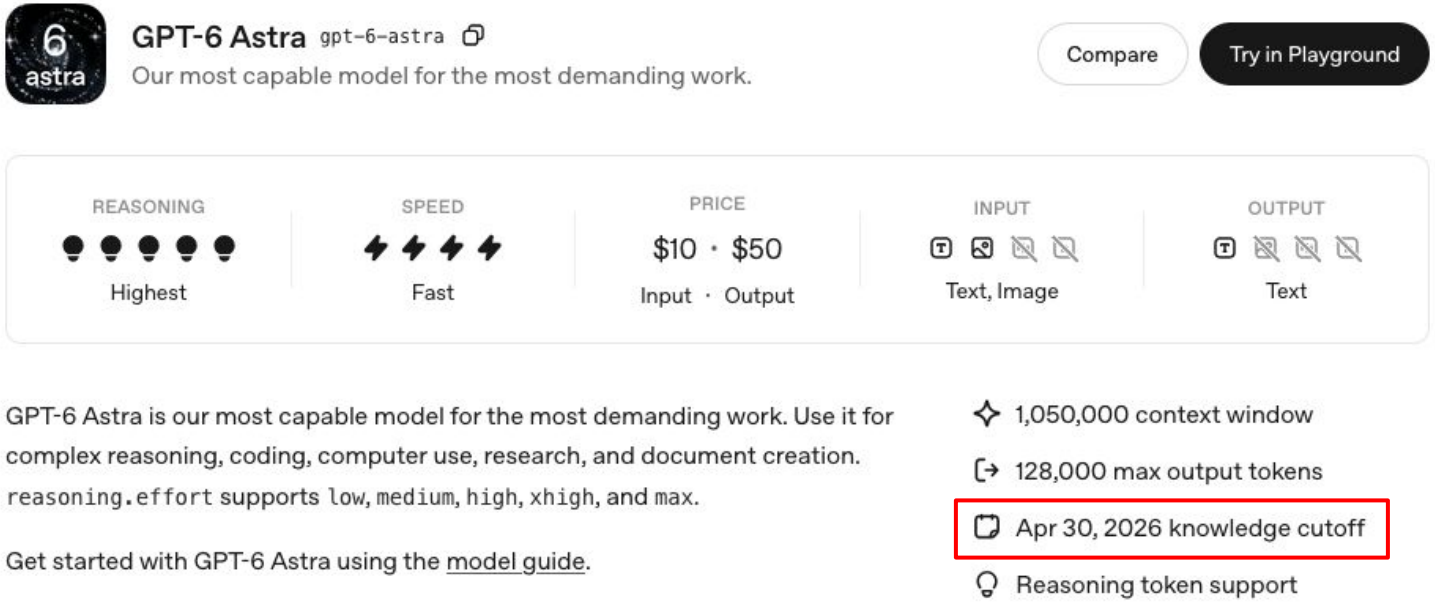


Image from kean.blog.

Limitations

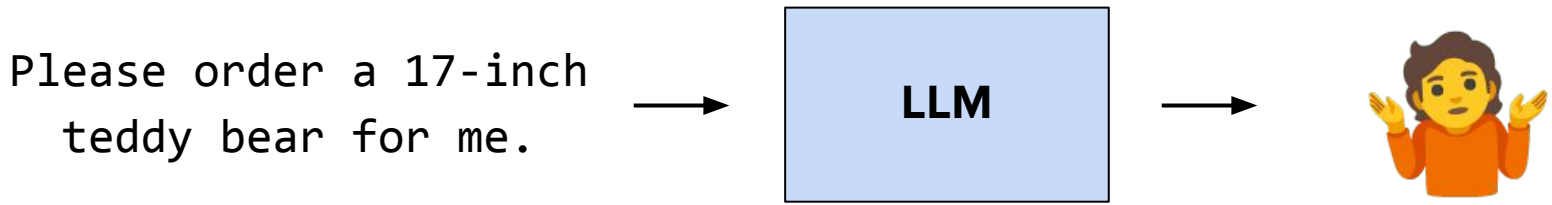
- **Knowledge** of LLM **constrained** to what is in the training data



Screenshot of OpenAI's GPT-6 model card, taken on November 1st, 2026.

Limitations

- **Knowledge** of LLM **constrained** to what is in the training data
- **Cannot** perform **actions**





Transformers & Large Language Models

Motivation

Retrieval

Tool calling

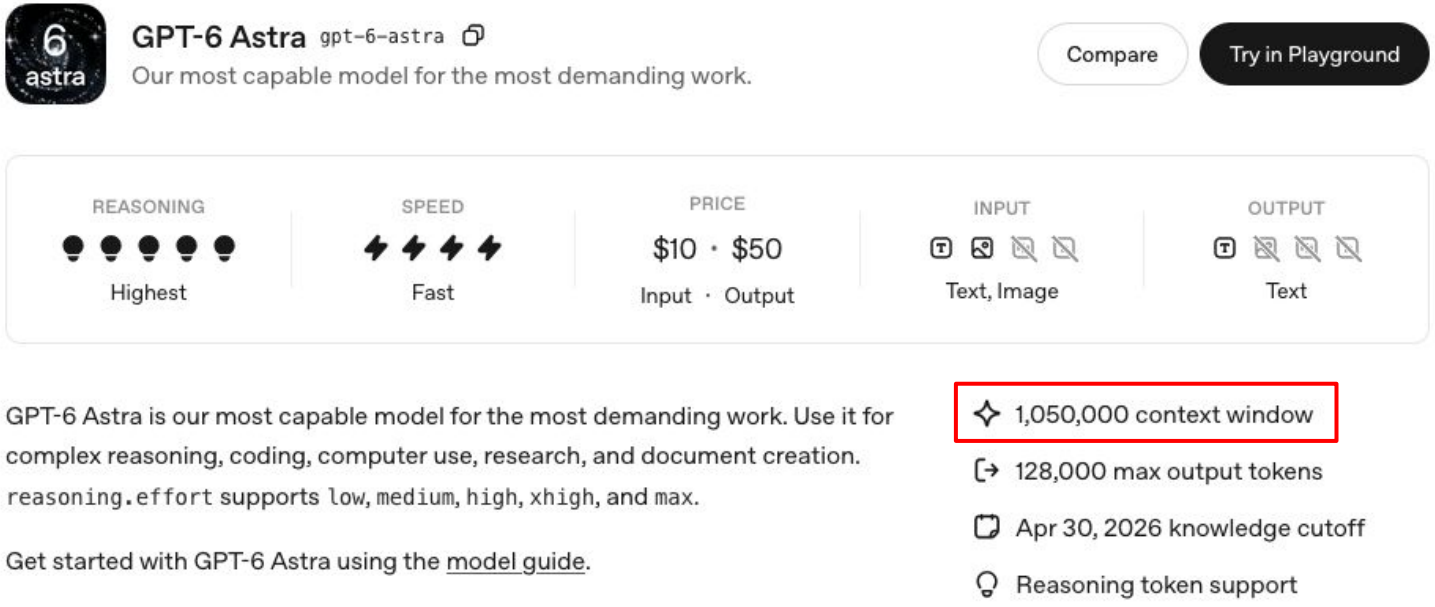
Harness engineering

Coding agents

Discussion

Motivation

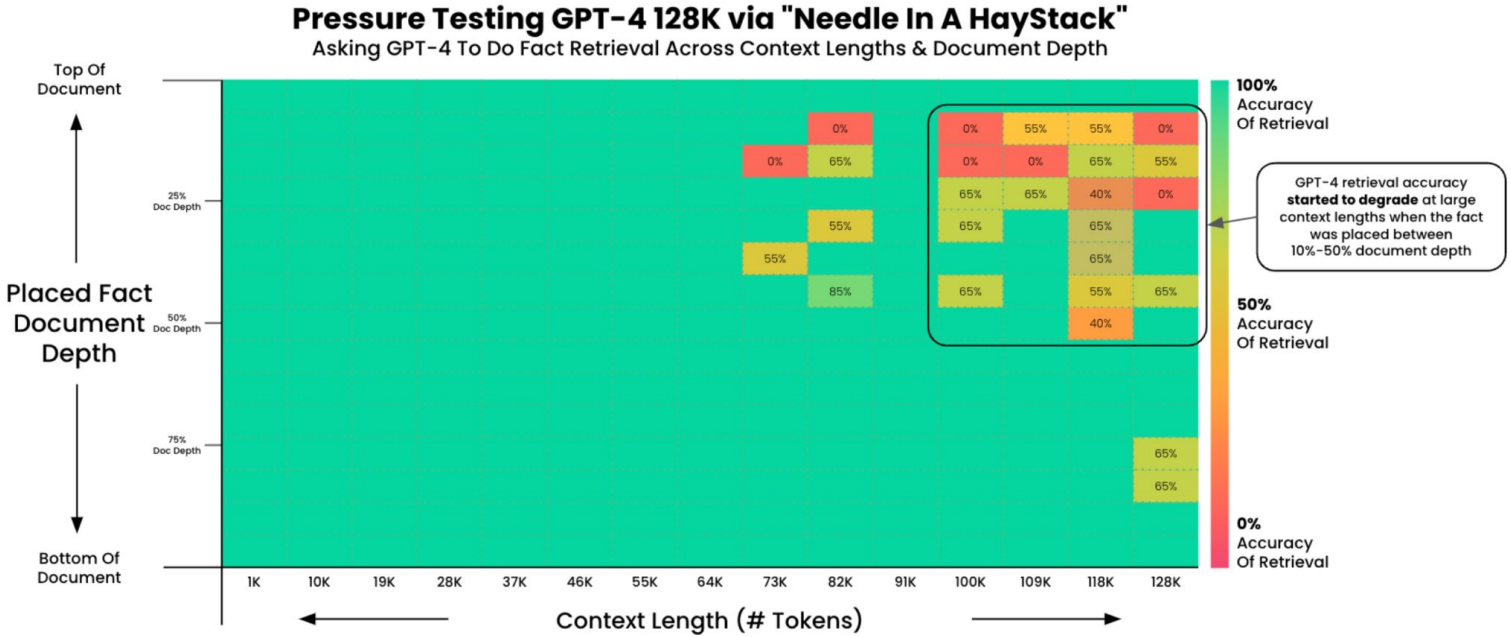
- **Limited** context size



Screenshot of OpenAI's GPT-6 model card, taken on November 1st, 2026.

Motivation

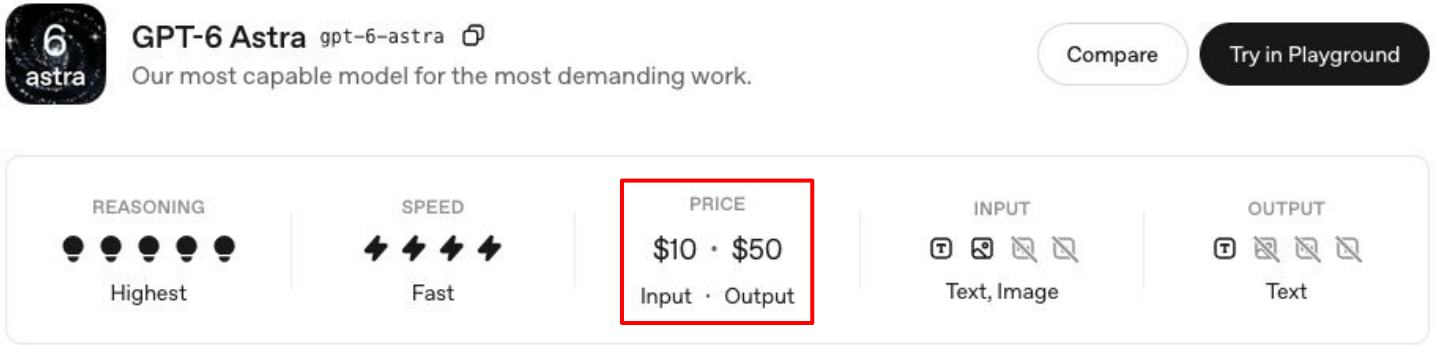
- **Limited** context size
- LLM gets **distracted** by **useless** information



Needle in a Haystack, Kamradt, 2023.

Motivation

- **Limited** context size
- LLM gets **distracted** by **useless** information
- **Pricing** is **per** input/output **token**



Screenshot of OpenAI's GPT-6 model card, taken on November 1st, 2026.

Overview

RAG = **R**etrieval-**A**ugmented **G**eneration

Idea. Augment prompt with **relevant** pieces of information.

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, Lewis et al., 2020.

Overview

RAG = **R**etrieval-**A**ugmented **G**eneration

Idea. Augment prompt with **relevant** pieces of information.

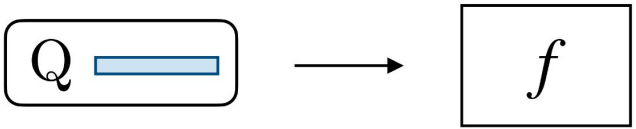


Figure from Super Study Guide: Transformers & Large Language Models, Amidi et al., 2024.

Overview

RAG = **R**etrieval-**A**ugmented **G**eneration

Idea. Augment prompt with **relevant** pieces of information.

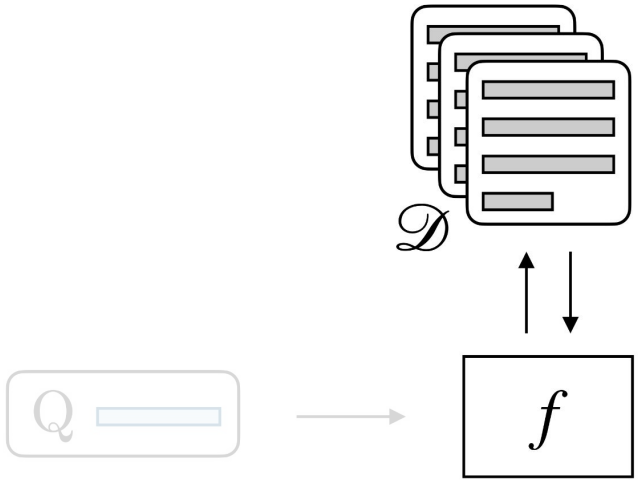


Figure from Super Study Guide: Transformers & Large Language Models, Amidi et al., 2024.

Overview

RAG = **R**etrieval-**A**ugmented **G**eneration

Idea. Augment prompt with **relevant** pieces of information.

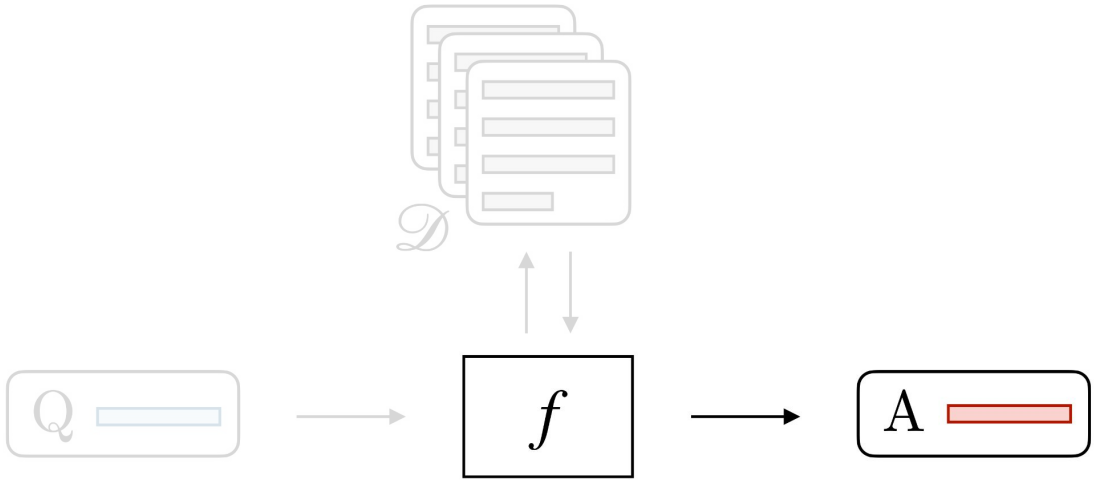
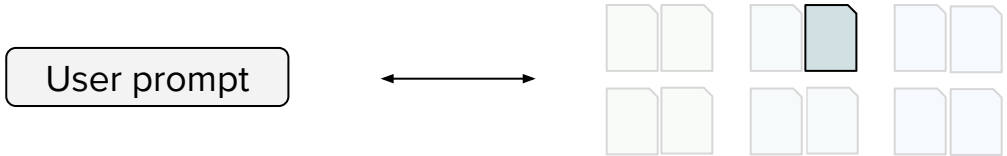


Figure from Super Study Guide: Transformers & Large Language Models, Amidi et al., 2024.

Retrieve, Augment, Generate

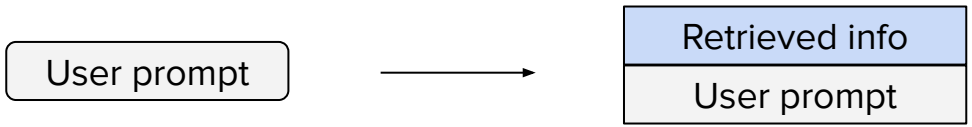
1. **Retrieve** relevant document via similarity operation across the knowledge base



Retrieve, Augment, Generate

1. **Retrieve** relevant document via similarity operation across the knowledge base

2. **Augment** prompt with retrieved information

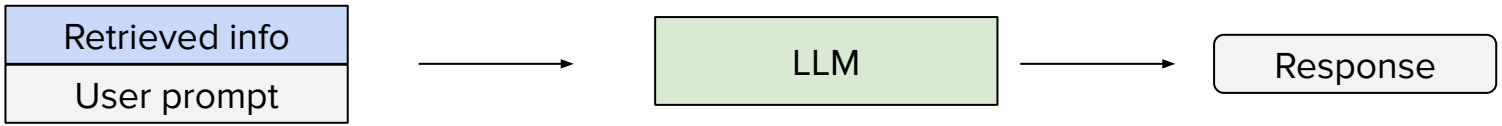


Retrieve, Augment, Generate

1. **Retrieve** relevant document via similarity operation across the knowledge base

2. **Augment** prompt with retrieved information

3. **Generate** response



Retrieve, Augment, Generate

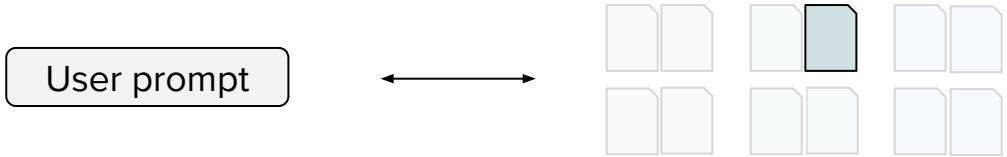
1. **R**etrieve relevant document via similarity operation across the knowledge base

2. **A**ugment prompt with retrieved information

3. **G**enerate response

Focus on retrieval stage

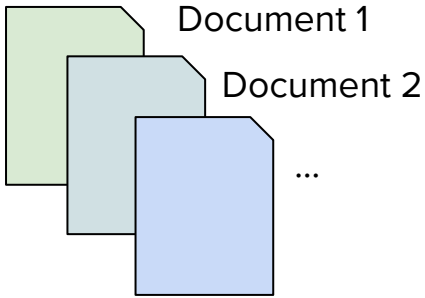
1. **R**etrieve relevant document via similarity operation across the knowledge base



2. **A**ugment prompt with retrieved information

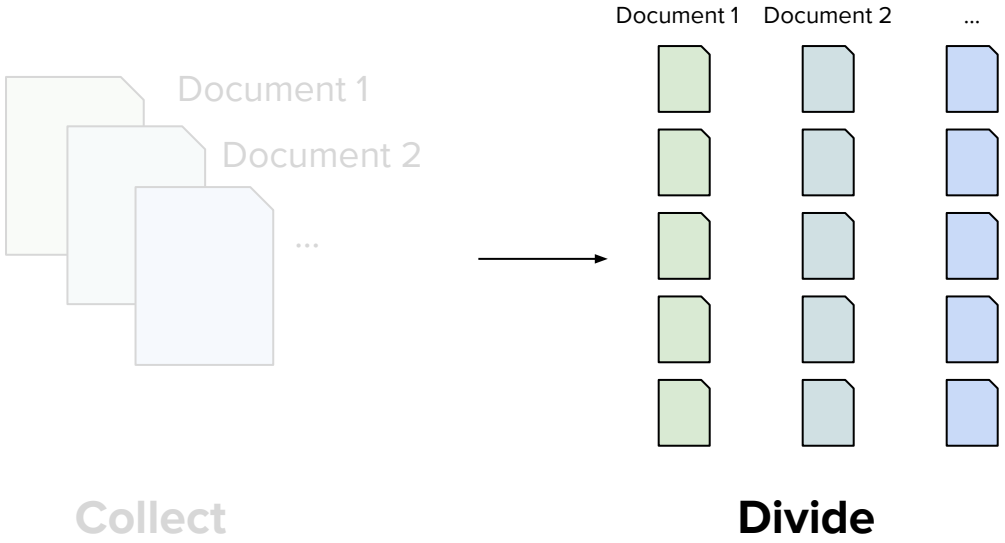
3. **G**enerate response

Prerequisite: Create knowledge base

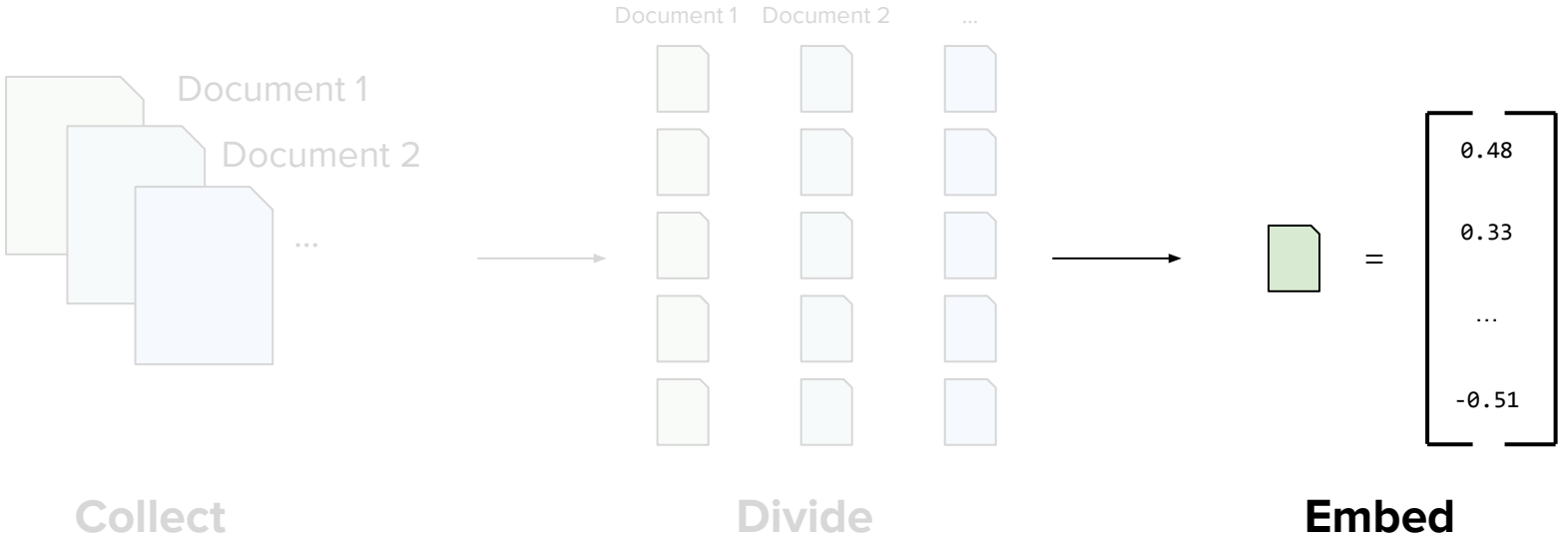


Collect

Prerequisite: Create knowledge base



Prerequisite: Create knowledge base



Prerequisite: Create knowledge base

Hyperparameters. Embedding size, chunk size, overlap between chunks

Retrieval overview



Step 1 – Candidate retrieval: Select potentially-relevant candidates

- Maximize recall
- Semantic embeddings and optionally keyword-based methods

Retrieval overview

Step 1 – Candidate retrieval: Select potentially-relevant candidates

- Maximize recall
- Semantic embeddings and optionally keyword-based methods



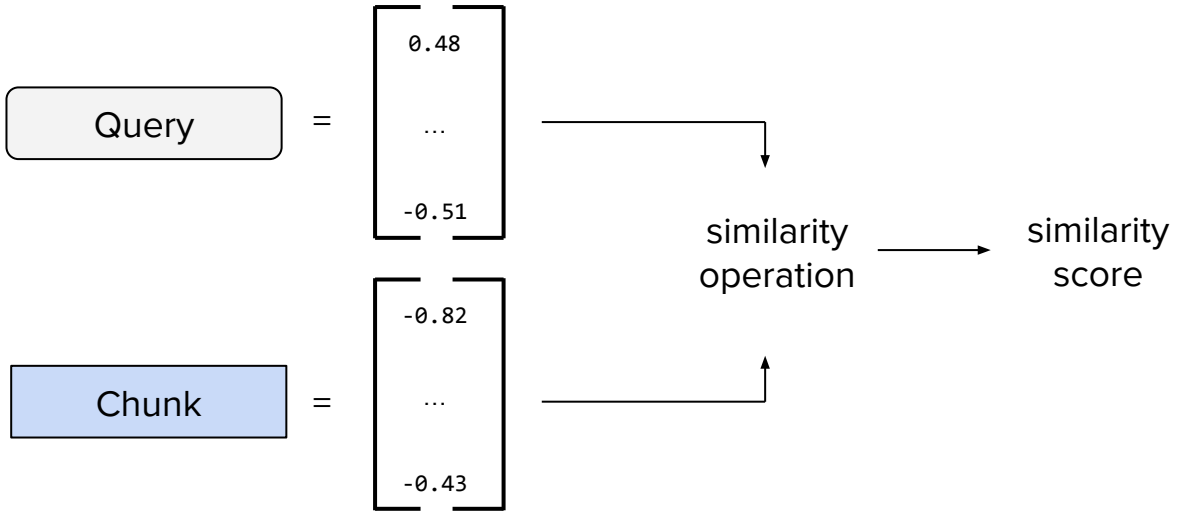
Step 2 – Ranking: Give final relevance score

- Maximize precision
- Re-ranking on smaller set of candidates

Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

Method 1. Semantic search using **embeddings-based similarity**

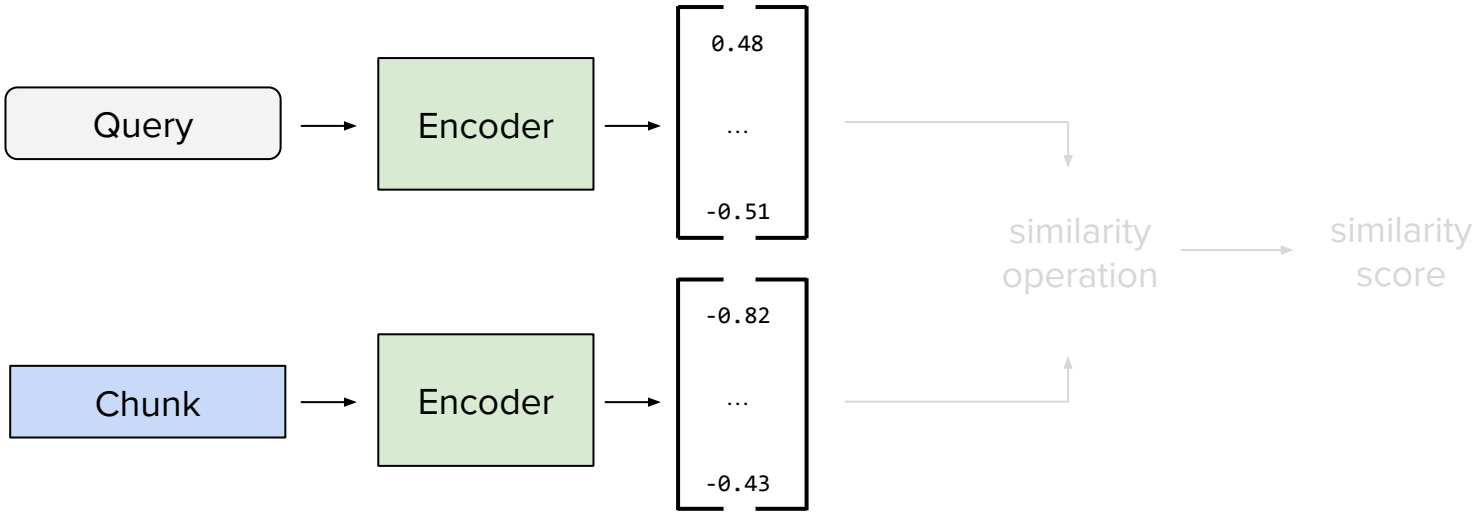


Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

Method 1. Semantic search using **embeddings-based similarity**

"bi-encoder"

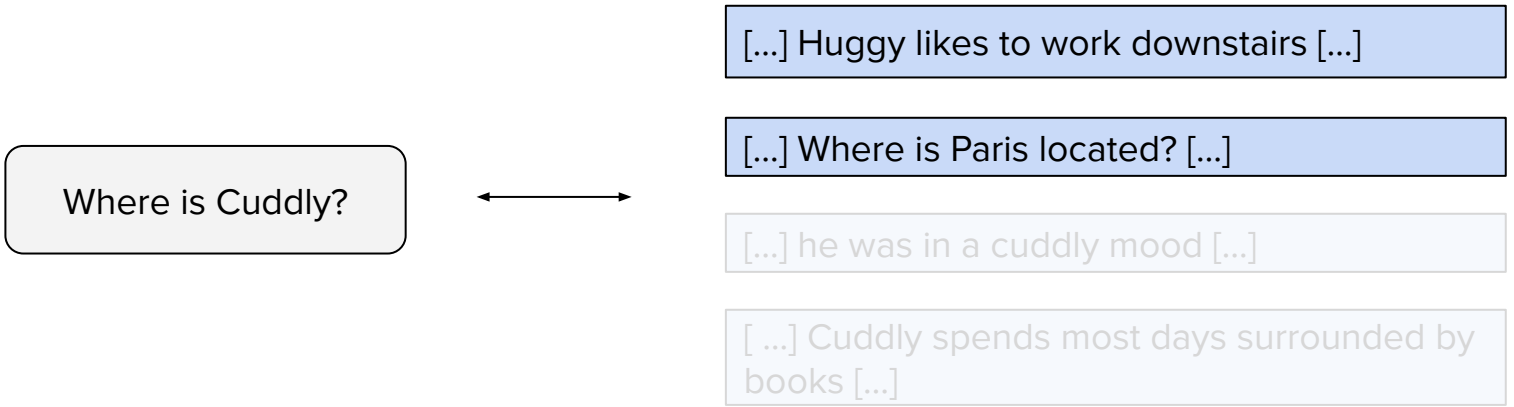


Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks, Reimers et al., 2019.

Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

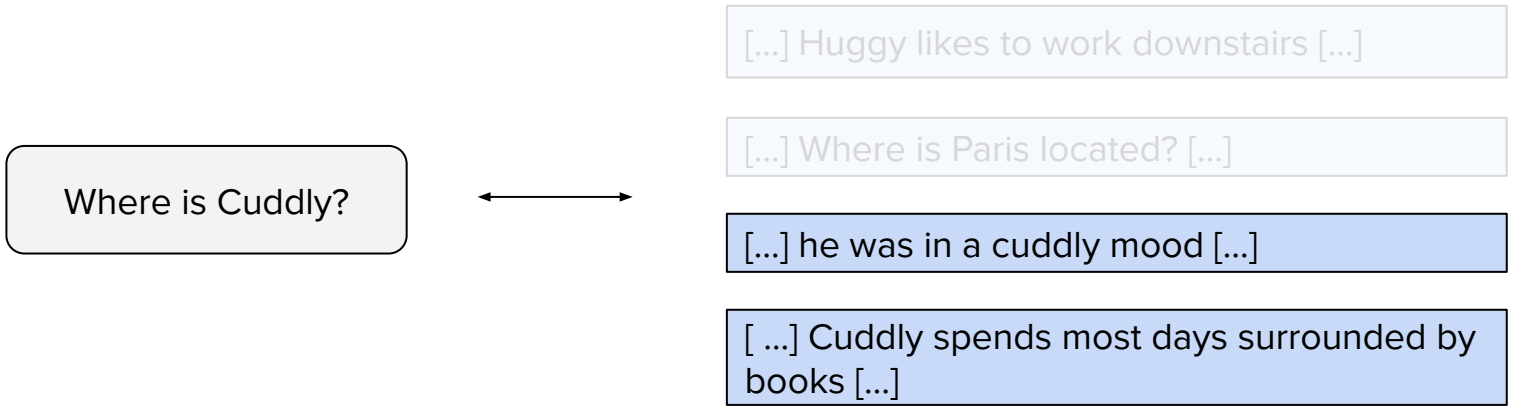
Method 1. Semantic search using **embeddings-based similarity**



Step 1: candidate retrieval

Objective. Select potentially-relevant candidates via search across knowledge base

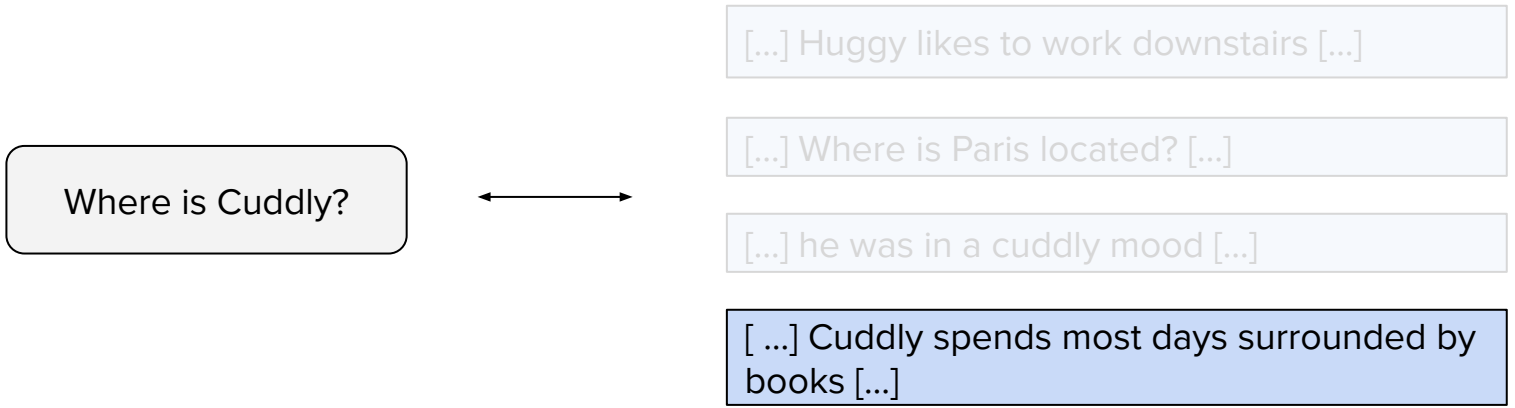
Method 2. Search based on keywords matching via **BM25**



Step 1: candidate retrieval

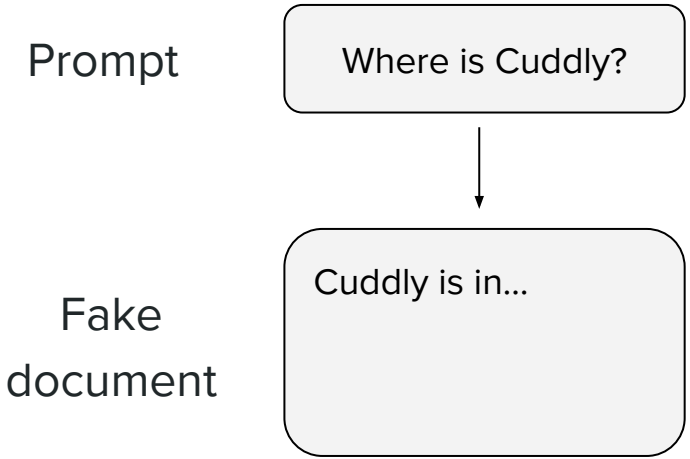
Objective. Select potentially-relevant candidates via search across knowledge base

Method 3. Search based on **hybrid combination** of semantics and BM25



Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings

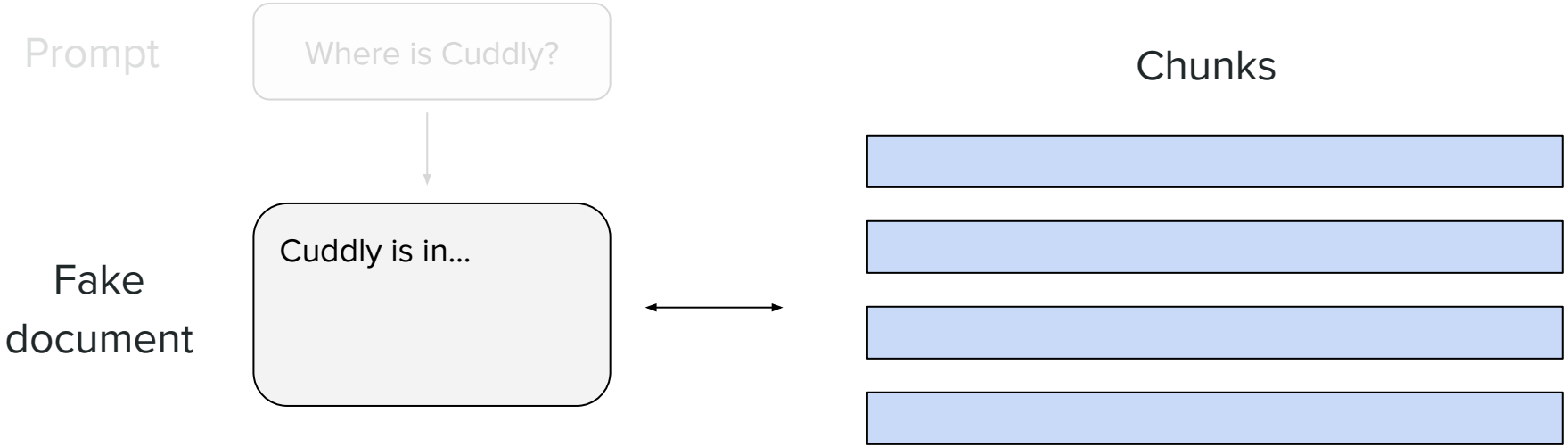


Precise Zero-Shot Dense Retrieval without Relevance Labels, Gao et al., 2022.

Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings

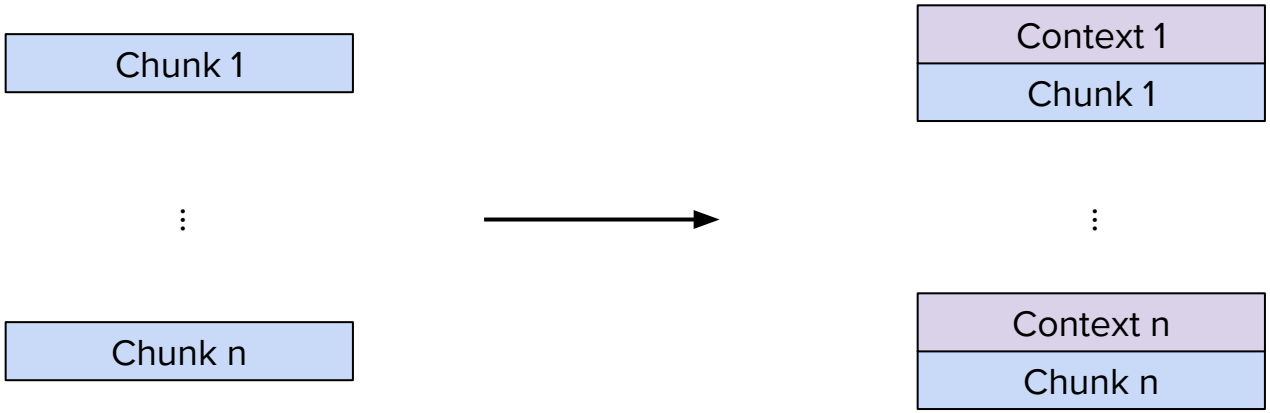
Hypothetical **D**ocument **E**mbeddings (HyDE)



Precise Zero-Shot Dense Retrieval without Relevance Labels, Gao et al., 2022.

Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings
- **Contextualize** document chunks



Introducing Contextual Retrieval, Anthropic team, 2024.

Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings
- **Contextualize** document chunks

```
<document>
{WHOLE_DOCUMENT}
</document>
Here is the chunk we want to situate within the whole document:
{CHUNK_CONTENT}


Please give a short succinct context to situate this chunk within the overall
document for the purposes of improving search retrieval of the chunk. Answer
only with the succinct context and nothing else.
```


Introducing Contextual Retrieval, *Anthropic team, 2024.*

Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings
- **Contextualize** document chunks

Prompt caching

```
<document>
{WHOLE_DOCUMENT}
</document>
Here is the chunk we want to situate within the whole document:
{CHUNK_CONTENT}

Please give a short succinct context to situate this chunk within the overall
document for the purposes of improving search retrieval of the chunk. Answer
only with the succinct context and nothing else.
```


Introducing Contextual Retrieval, Anthropic team, 2024.

Extensions that can help with initial retrieval

- Mitigate **discrepancy** in nature of embeddings
- **Contextualize** document chunks

Pricing

Flagship models

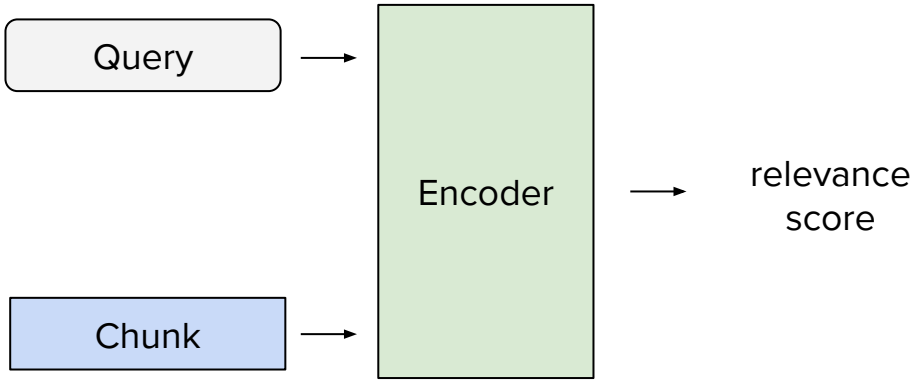
Model	Input	Output	Cached input
gpt-6-astra	$10.00	$50.00	$1.00
gpt-6.1-sol	$2.00	$10.00	$0.10
gpt-6.1-terra	$1.00	$5.00	$0.05
gpt-6-luna	$0.10	$0.50	$0.01
gpt-6-sol	$2.00	$10.00	$0.20

OpenAI model pricing page, screenshot taken on November 1st, 2026.

Step 2: ranking

Objective. Provide final relevance score using more sophisticated (re-)ranker

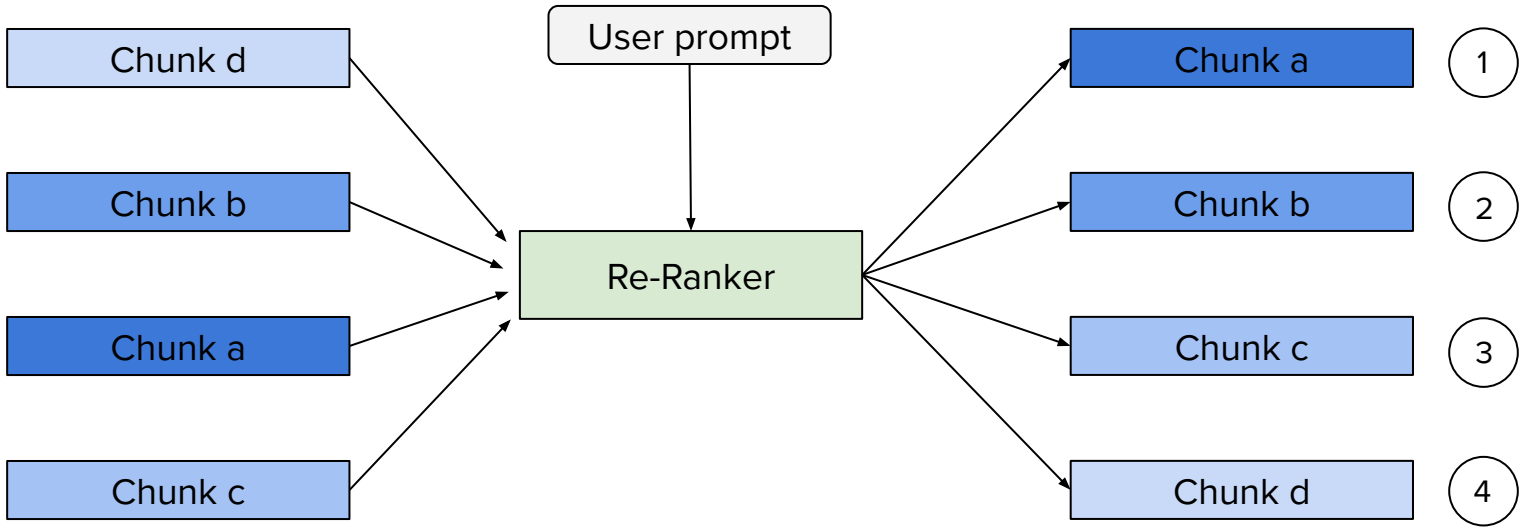
"cross-encoder"



Suggested reading: Cross-Encoders, SBERT.net

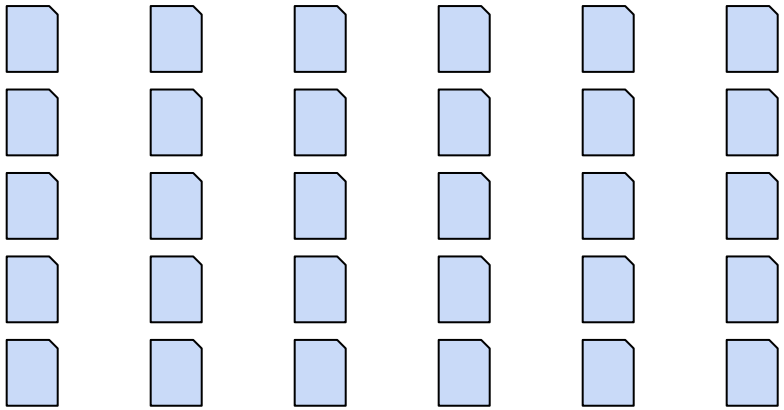
Step 2: ranking

Objective. Provide final relevance score using more sophisticated (re-)ranker



Quantify performance of retrieval

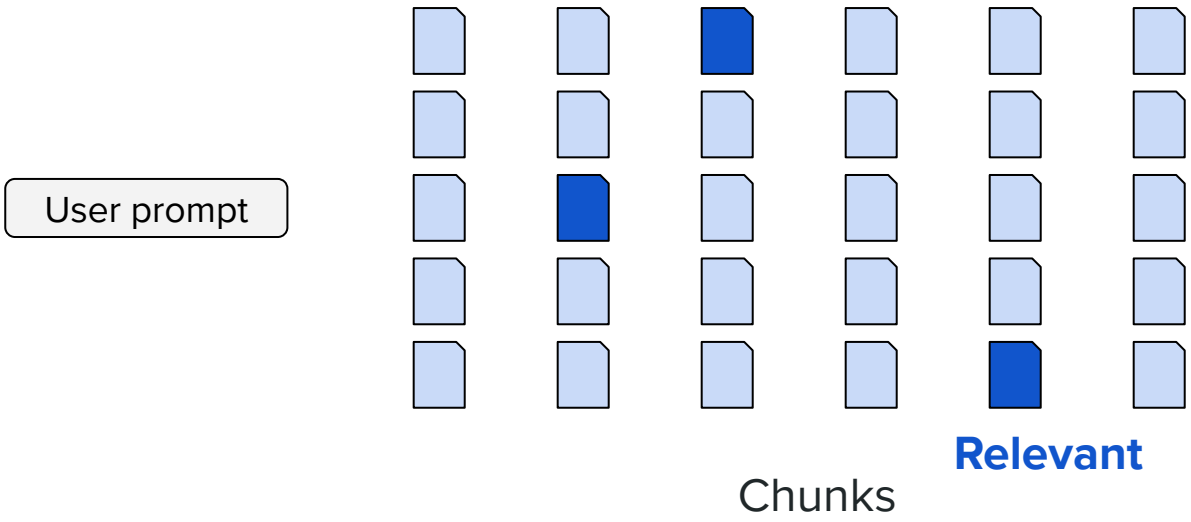
Setup. Evaluate if **retrieved chunks** are **relevant**



Chunks

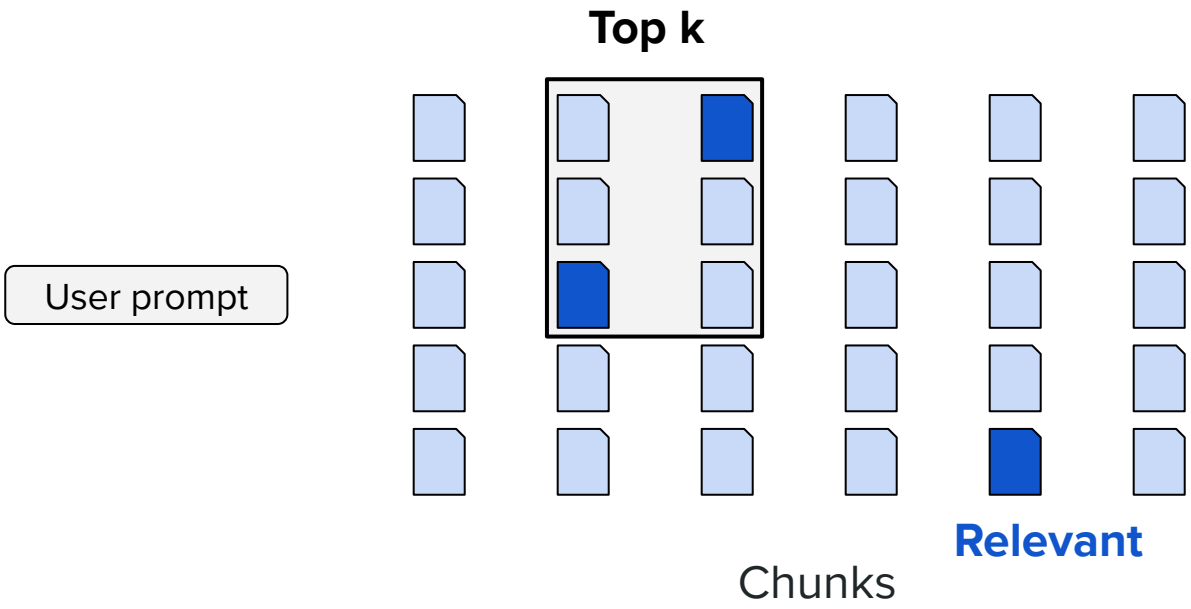
Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**



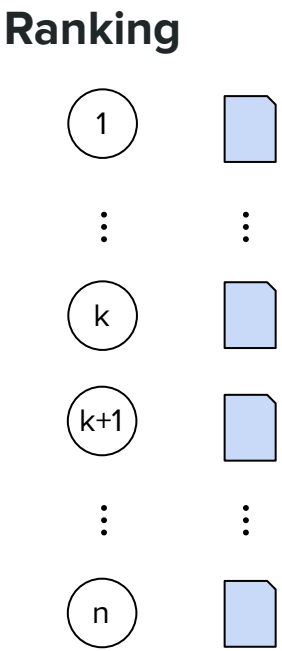
Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**



Quantify performance of retrieval

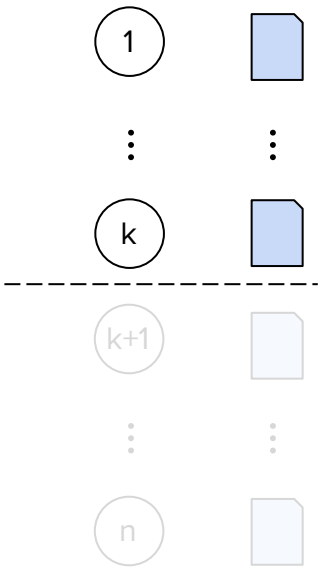
Setup. Evaluate if **retrieved chunks** are **relevant**



Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

Ranking



- **N**ormalized **D**iscounted **C**umulative **G**ain at **k** (**NDCG@k**)

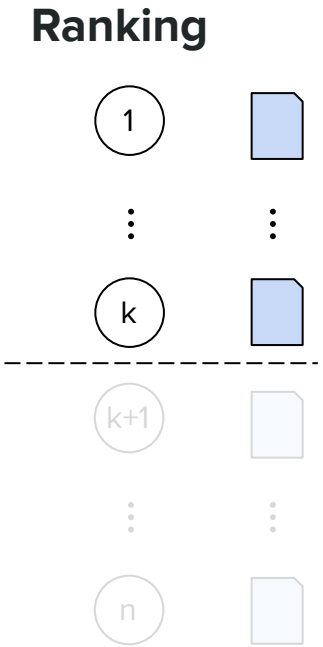
$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{\mathrm{rel}_i}{\log_2(i+1)} \qquad \text{with} \quad \mathrm{rel}_i \in \{0, 1\}$$

$$\mathrm{NDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}$$

$\mathrm{DCG@}k$ if ranking was perfect

Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**



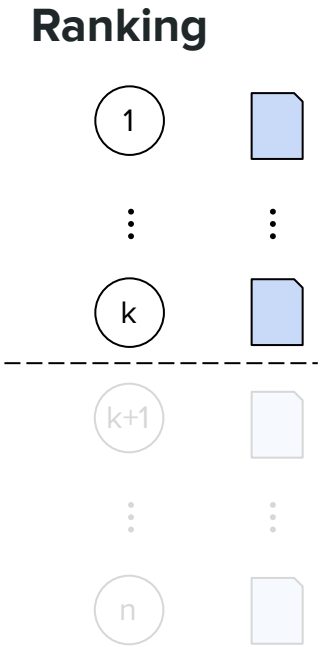
- Normalized Discounted Cumulative Gain at **k** (**NDCG@k**)
- **R**eciprocal **R**ank at **k** (**RR@k**)

$$\mathrm{RR} = \frac{1}{\mathrm{rank}}$$

rank of the first relevant chunk

Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**

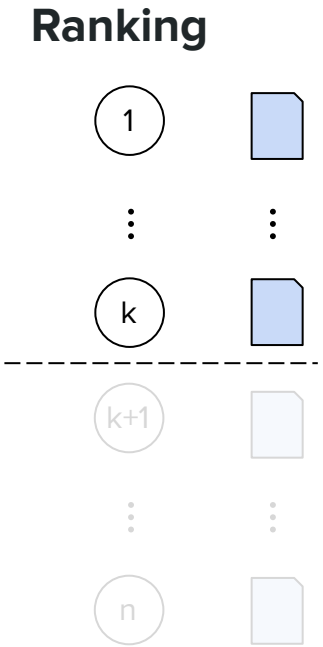


- Normalized Discounted Cumulative Gain at k (NDCG@k)
- Reciprocal Rank at k (RR@k)
- **Recall at k**

$$\text{Recall@}k = \frac{|\text{relevant in top } k|}{|\text{relevant}|}$$

Quantify performance of retrieval

Setup. Evaluate if **retrieved chunks** are **relevant**



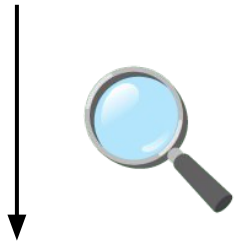
- Normalized Discounted Cumulative Gain at k (NDCG@k)
- Reciprocal Rank at k (RR@k)
- Recall at k
- **Precision at k**

$$\text{Precision@}k = \frac{|\text{relevant in top } k|}{k}$$

Special case: code

grep = **g**lobal **r**egular **e**xpression **p**rint

```
grep [options] "search_term" file_name
```



Gives all code locations where the term occurs



Transformers & Large Language Models

Motivation

Retrieval

Tool calling

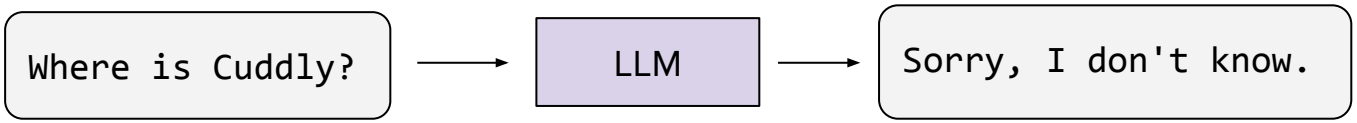
Harness engineering

Coding agents

Discussion

Pure information retrieval case

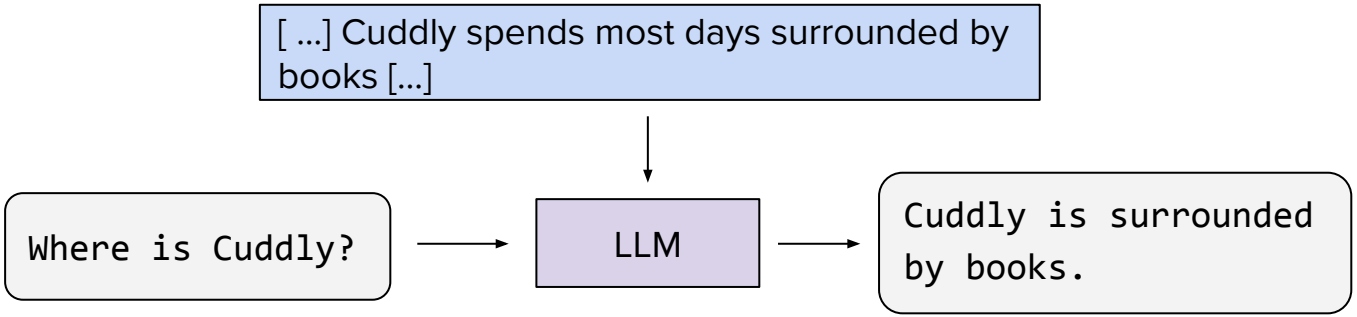
Vanilla LLM ❌



Pure information retrieval case

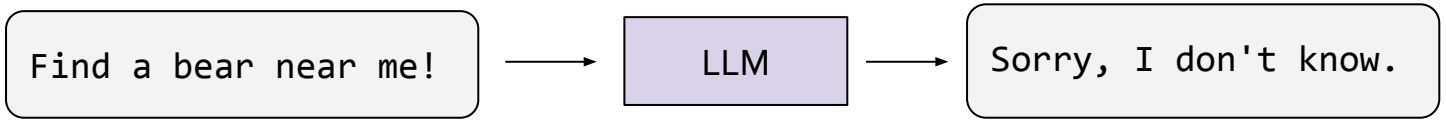
Vanilla LLM ❌

LLM + RAG ✅

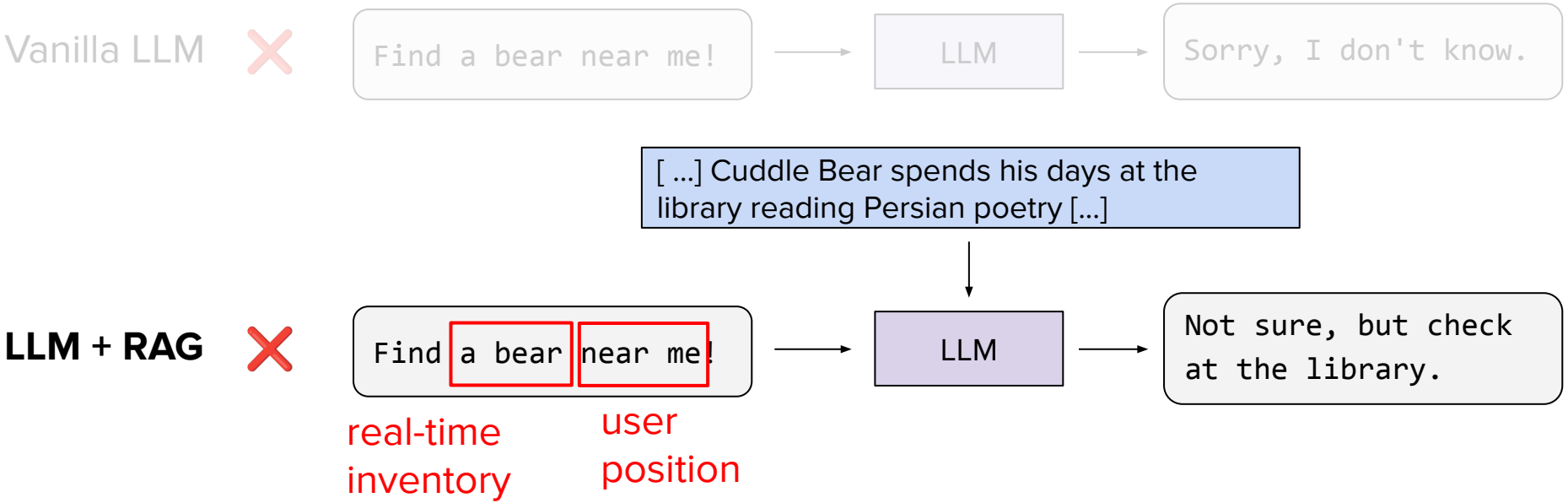


What if we need to perform actions?

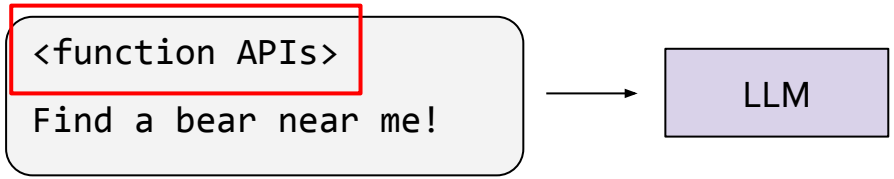
Vanilla LLM ❌



What if we need to perform actions?

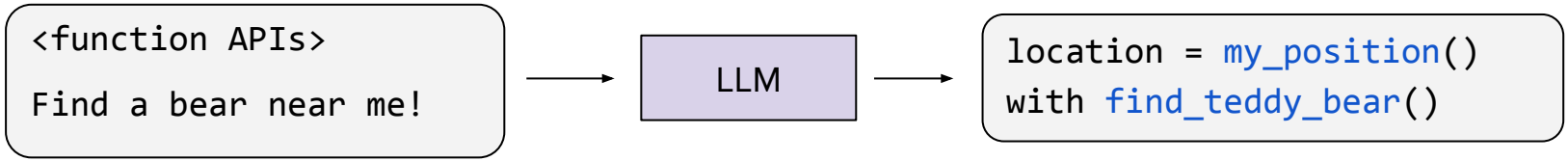


How to remedy? Tools!



How to remedy? Tools!

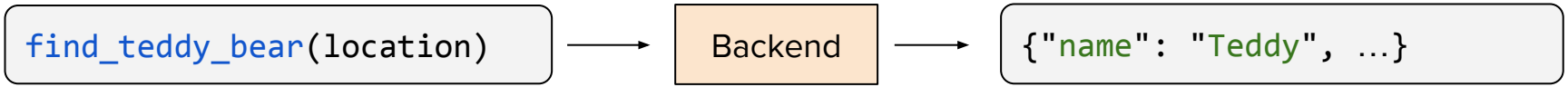
1. Let **LLM** find **argument** for **relevant function** call



How to remedy? Tools!

1. Let **LLM** find **argument** for **relevant function** call

2. Make **function call**

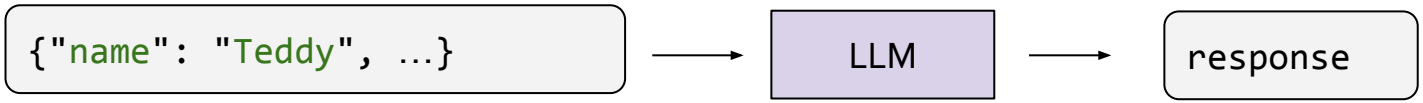


How to remedy? Tools!

1. Let **LLM** find **argument** for **relevant function** call

2. Make **function call**

3. Let **LLM deduce** conclusion **based on results**



Real-life example

find_teddy_bear.py

```python
from dataclasses import dataclass
from geopy.distance import geodesic
import requests

@dataclass
class TeddyBearInfo:
    name: str
    distance_meters: float
    mood: str
    message: str

def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
    """
    Finds the nearest teddy bear to the given GPS coordinates.

    Parameters:
        location: A (latitude, longitude) pair representing the user's current
            location.

    Returns:
        TeddyBearInfo: Information about the nearest teddy bear found.
    """
    # Call API to get the closest teddy bear
    user_lat, user_lon = location
    api_url = "https://api.to.teddy.bears.com/v1/closest"
    try:
        response = requests.get(
            api_url,
            params={"latitude": user_lat, "longitude": user_lon},
            timeout=5
        )
        response.raise_for_status()
        closest_teddy_bear = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch teddy bear data from API: {e}")
```

```python
    # Extract coordinates from API response
    bear_lat, bear_lon = closest_teddy_bear["coords"]

    # Compute distance to the bear using geopy (returns distance in meters)
    distance = geodesic((user_lat, user_lon), (bear_lat, bear_lon)).meters

    return TeddyBearInfo(
        name=closest_teddy_bear["name"],
        distance_meters=round(distance, 2),
        mood=closest_teddy_bear["mood"],
        message=f"{closest_teddy_bear['name']} is {closest_teddy_bear['mood']} "
        f"and only {round(distance, 2)} meters away!"
    )
```

has a descriptive, well-documented API

Real-life example

find_teddy_bear.py

```python
from dataclasses import dataclass
from geopy.distance import geodesic
import requests

@dataclass
class TeddyBearInfo:
    name: str
    distance_meters: float
    mood: str
    message: str

def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
    """
    Finds the nearest teddy bear to the given GPS coordinates.

    Parameters:
        location: A (latitude, longitude) pair representing the user's current
            location.

    Returns:
        TeddyBearInfo: Information about the nearest teddy bear found.
    """
    # Call API to get the closest teddy bear
    user_lat, user_lon = location
    api_url = "https://api.to.teddy.bears.com/v1/closest"
    try:
        response = requests.get(
            api_url,
            params={"latitude": user_lat, "longitude": user_lon},
            timeout=5
        )
        response.raise_for_status()
        closest_teddy_bear = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch teddy bear data from API: {e}")
```

```python
    # Extract coordinates from API response
    bear_lat, bear_lon = closest_teddy_bear["coords"]

    # Compute distance to the bear using geopy (returns distance in meters)
    distance = geodesic((user_lat, user_lon), (bear_lat, bear_lon)).meters

    return TeddyBearInfo(
        name=closest_teddy_bear["name"],
        distance_meters=round(distance, 2),
        mood=closest_teddy_bear["mood"],
        message=f"{closest_teddy_bear['name']} is {closest_teddy_bear['mood']} "
        f"and only {round(distance, 2)} meters away!"
    )
```

(optional) has some backend call

Real-life example

find_teddy_bear.py

```python
from dataclasses import dataclass
from geopy.distance import geodesic
import requests

@dataclass
class TeddyBearInfo:
    name: str
    distance_meters: float
    mood: str
    message: str

def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
    """
    Finds the nearest teddy bear to the given GPS coordinates.

    Parameters:
        location: A (latitude, longitude) pair representing the user's current
            location.

    Returns:
        TeddyBearInfo: Information about the nearest teddy bear found.
    """
    # Call API to get the closest teddy bear
    user_lat, user_lon = location
    api_url = "https://api.to.teddy.bears.com/v1/closest"
    try:
        response = requests.get(
            api_url,
            params={"latitude": user_lat, "longitude": user_lon},
            timeout=5
        )
        response.raise_for_status()
        closest_teddy_bear = response.json()
    except requests.RequestException as e:
        raise RuntimeError(f"Failed to fetch teddy bear data from API: {e}")
```

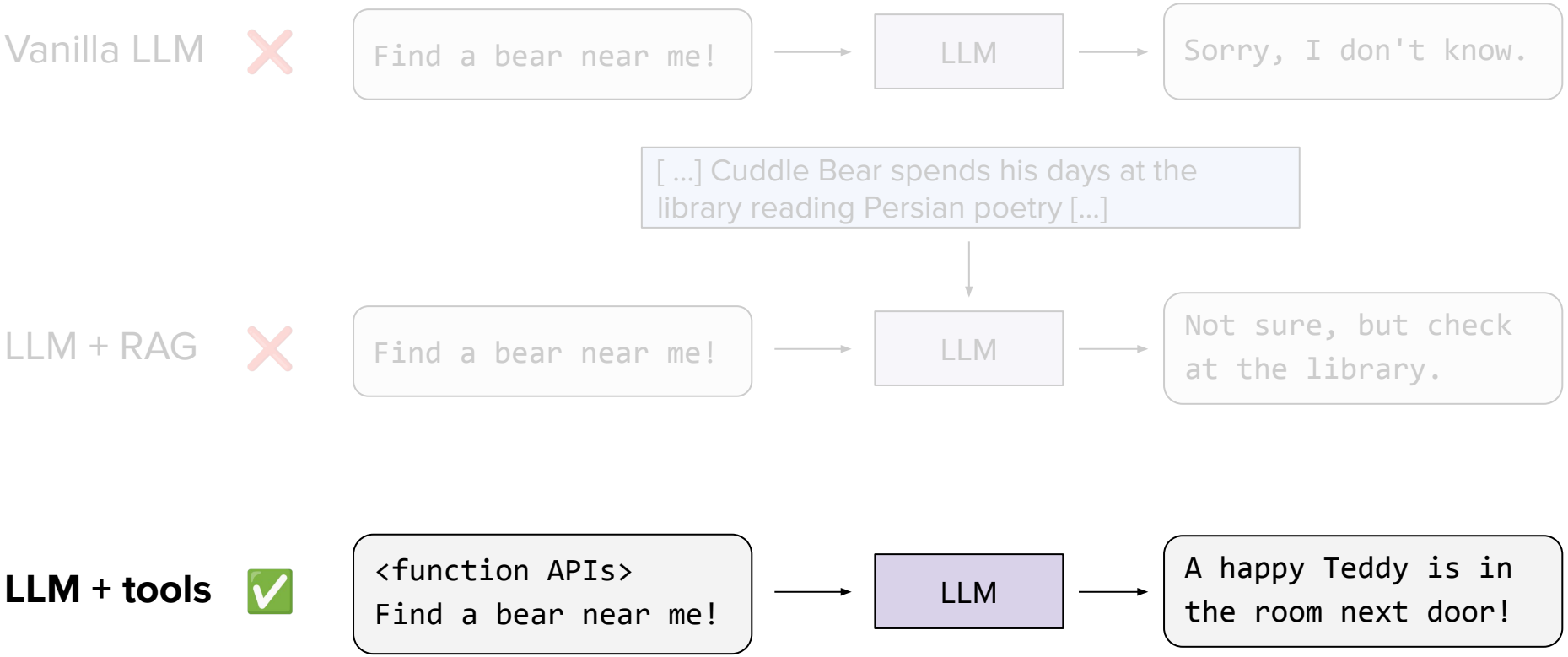
```python
    # Extract coordinates from API response
    bear_lat, bear_lon = closest_teddy_bear["coords"]

    # Compute distance to the bear using geopy (returns distance in meters)
    distance = geodesic((user_lat, user_lon), (bear_lat, bear_lon)).meters

    return TeddyBearInfo(
        name=closest_teddy_bear["name"],
        distance_meters=round(distance, 2),
        mood=closest_teddy_bear["mood"],
        message=f"{closest_teddy_bear['name']} is {closest_teddy_bear['mood']} "
        f"and only {round(distance, 2)} meters away!"
    )
```

returns some info

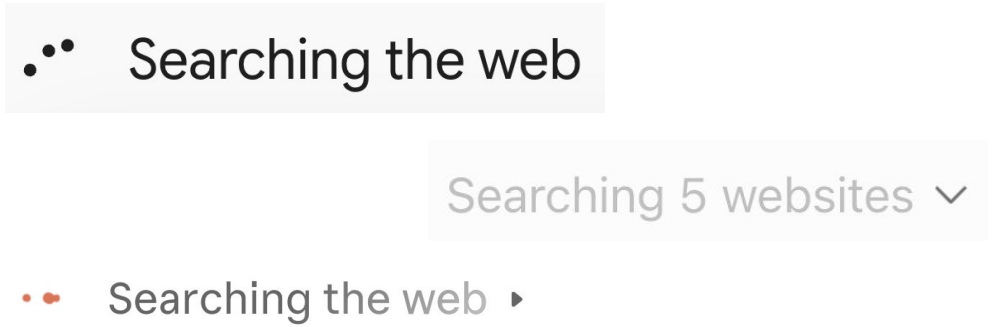
What if we need to perform actions?



Examples of common use cases

Real-time information

- Web/database search



Screenshots taken (top to bottom) from the Gemini, ChatGPT and Claude interfaces on November 5th, 2026.

Examples of common use cases

Real-time information

- Web/database search
- Weather, stocks, and any other tracker
- Codebase

Examples of common use cases

Real-time information

- Web/database search
- Weather, stocks, and any other tracker
- Codebase

Computation

- Calculator
- Code execution (often in Python)

Examples of common use cases

Real-time information

- Web/database search
- Weather, stocks, and any other tracker
- Codebase

Computation

- Calculator
- Code execution (often in Python)

Action

- Send emails/messages and other computer actions
- Anything else within the domain of an assistant

Examples of common use cases

Real-time information

- Web/database search
- Weather, stocks, and any other tracker
- Codebase

Computation

- Calculator
- Code execution (often in Python)

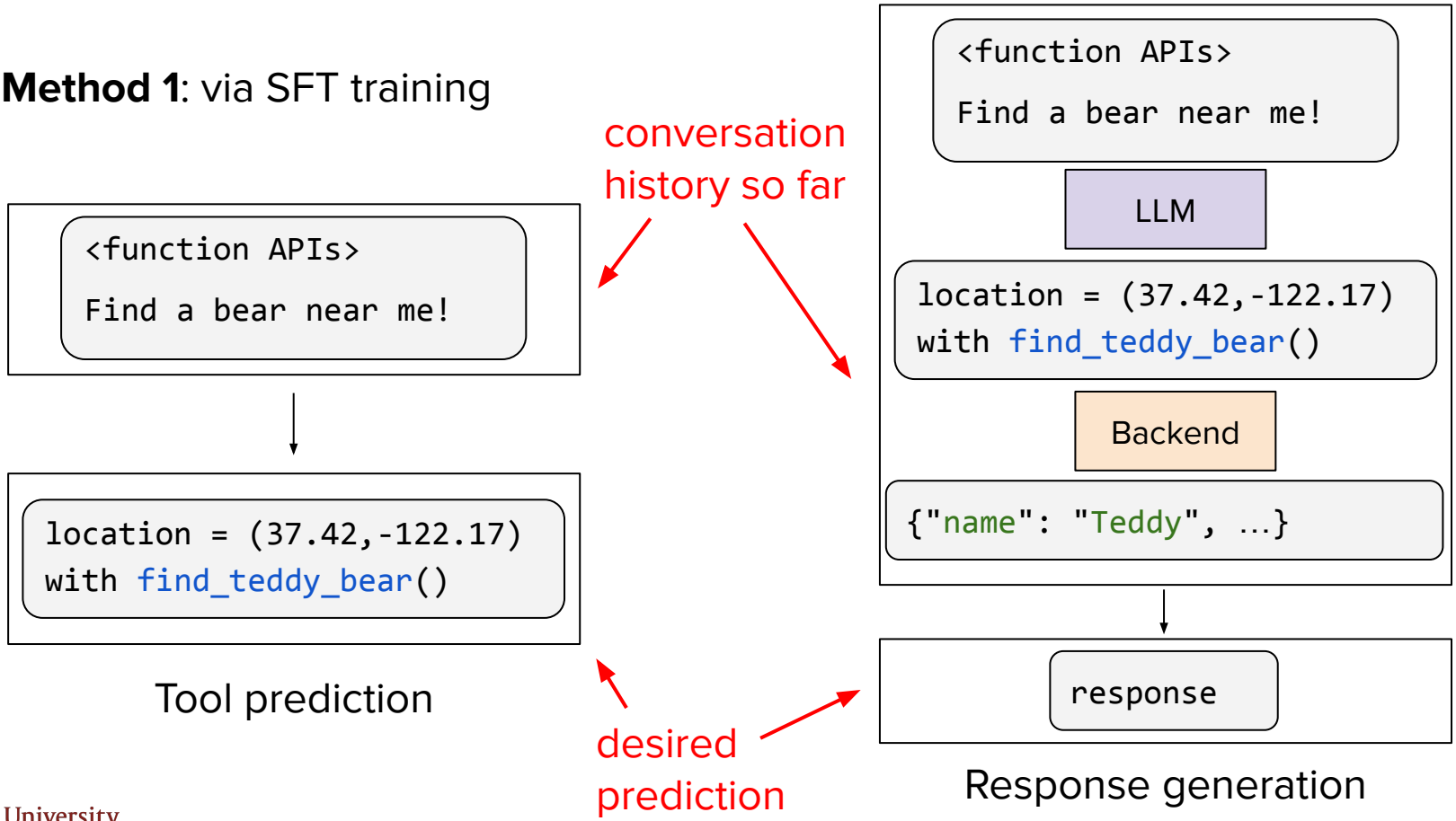
Action

- Send emails/messages and other computer actions
- Anything else within the domain of an assistant

...and many more!

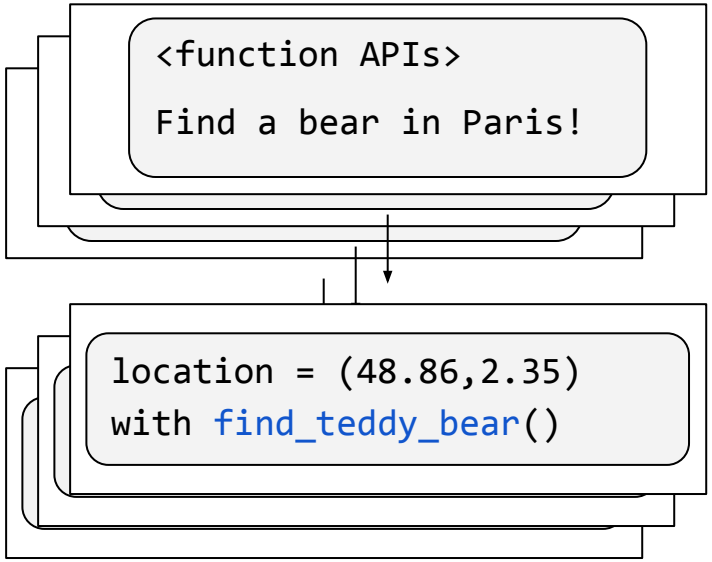
Teach a model to use a tool

Method 1: via SFT training

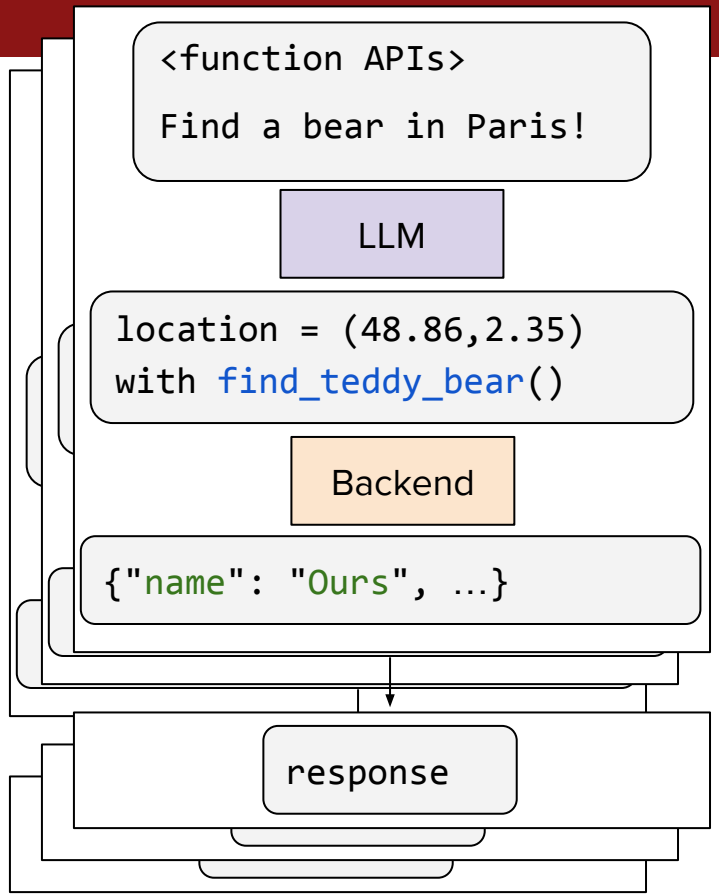


Teach a model to use a tool

Method 1: via SFT training



Tool prediction



Response generation

Teach a model to use a tool

Method 1: via SFT training

Method 2: via prompting

how to write such a description?

```
<function APIs> + <detailed explanation on how to use it>

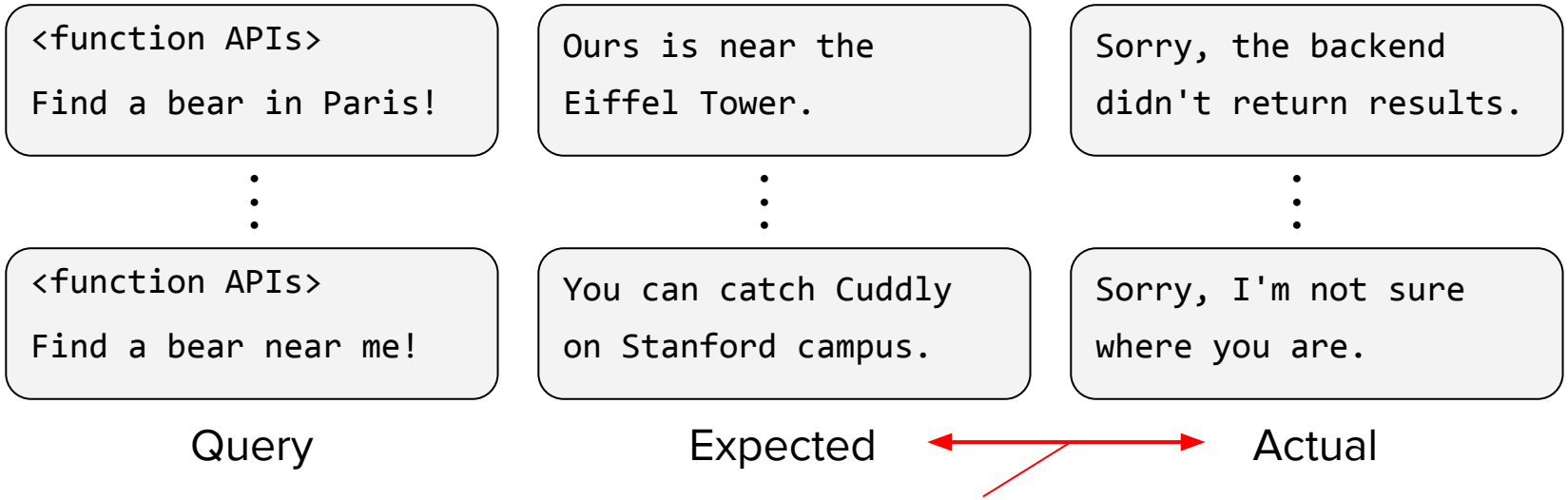
Find a bear near me!
```

One way: Use input/output pairs as evaluation + use a powerful reasoning model to write it for you!

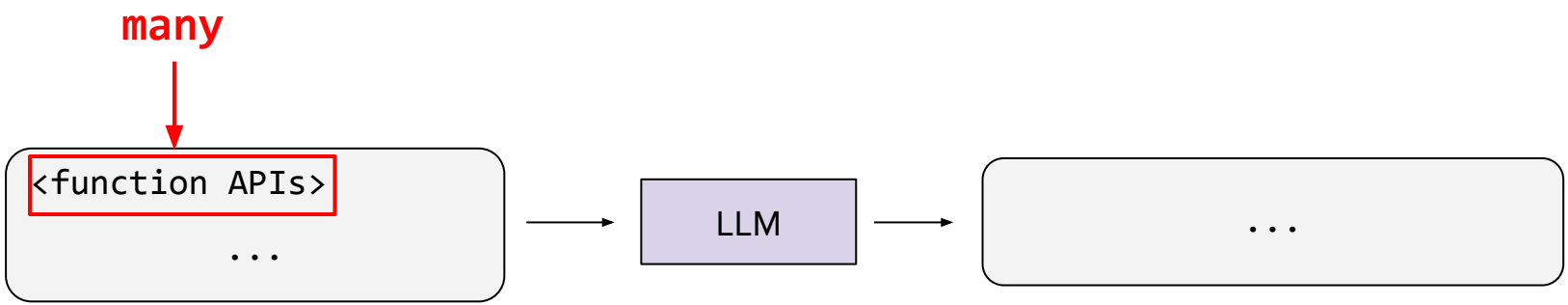
Teach a model to use a tool

Method 1: via SFT training

Method 2: via prompting



In practice: many tools



In practice: many tools

set_temperature

my_position

get_drive

order_food

list_files

search

resolve_id

get_weather

play_music

get_time

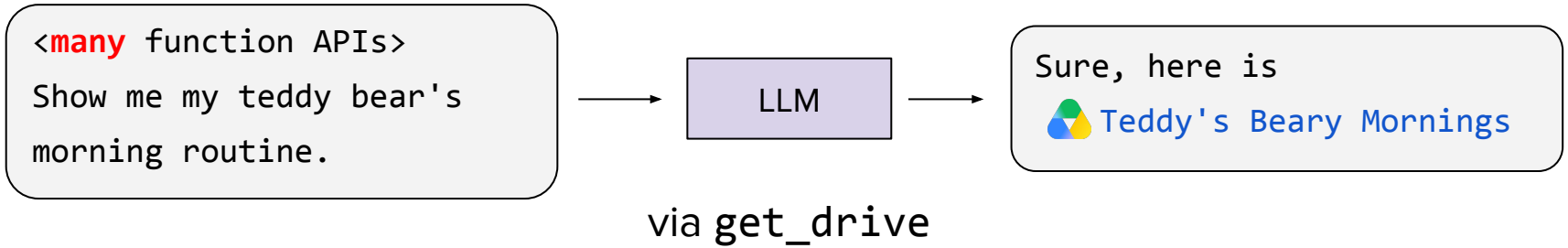
calculate

find_teddy_bear

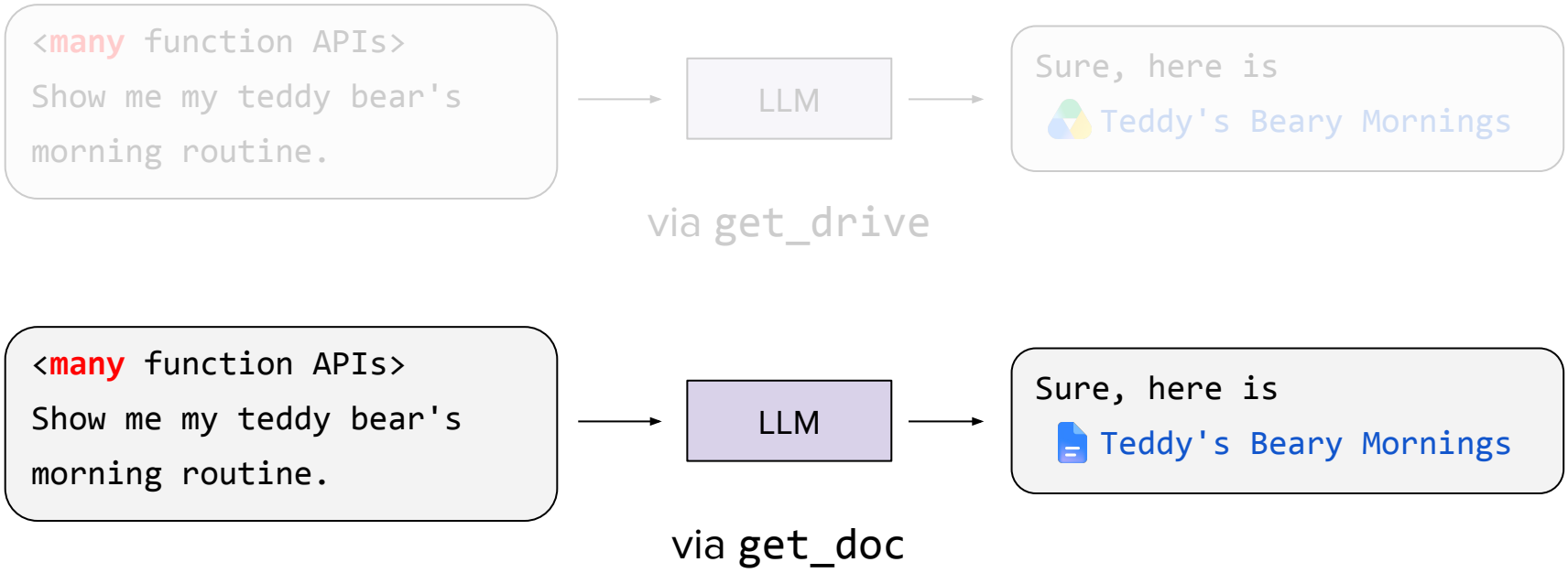
get_doc

fetch_data

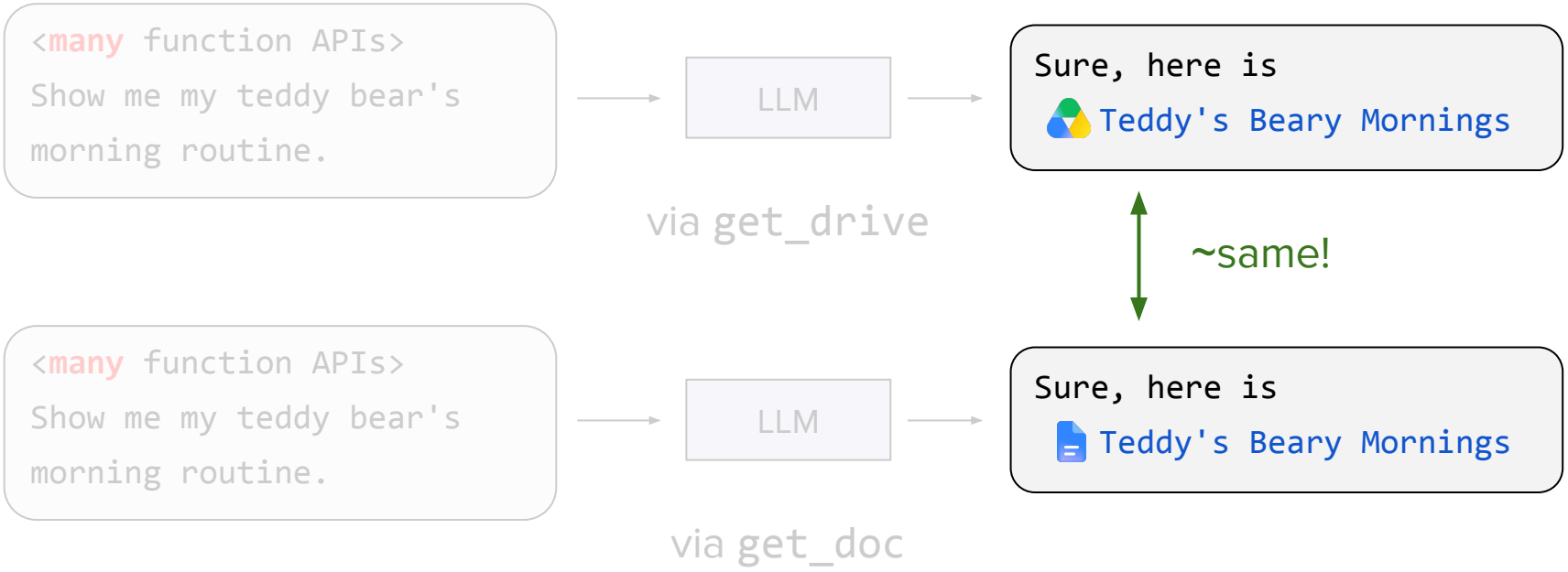
Risks for conflicts



Risks for conflicts



Risks for conflicts



Risks for conflicts

set_temperature

my_position

get_drive

order_food

list_files

search

resolve_id

get_weather

play_music

get_time

calculate

find_teddy_bear

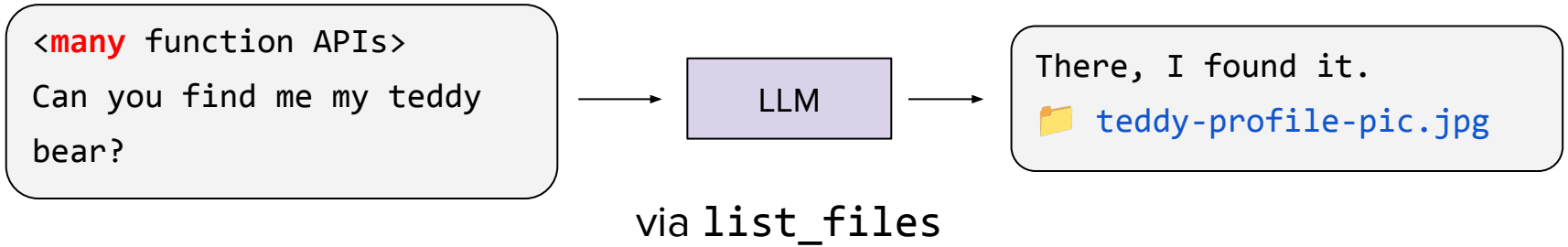
get_doc

fetch_data

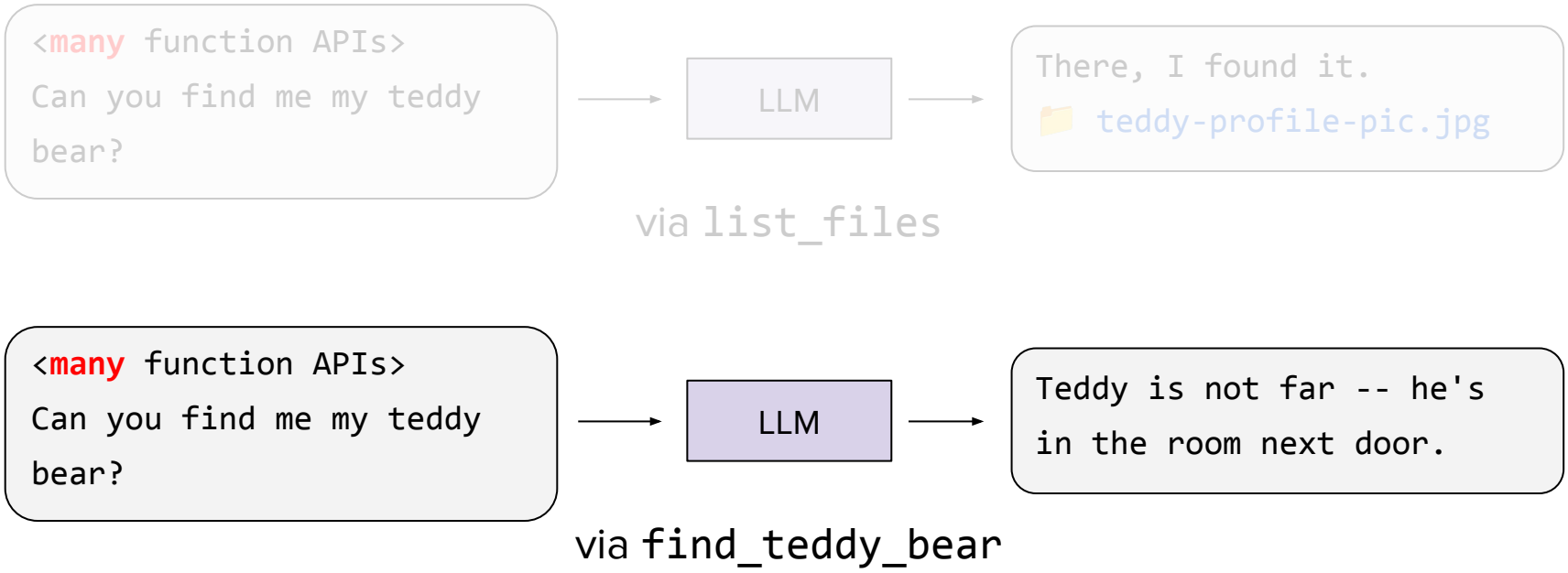
Risks for conflicts



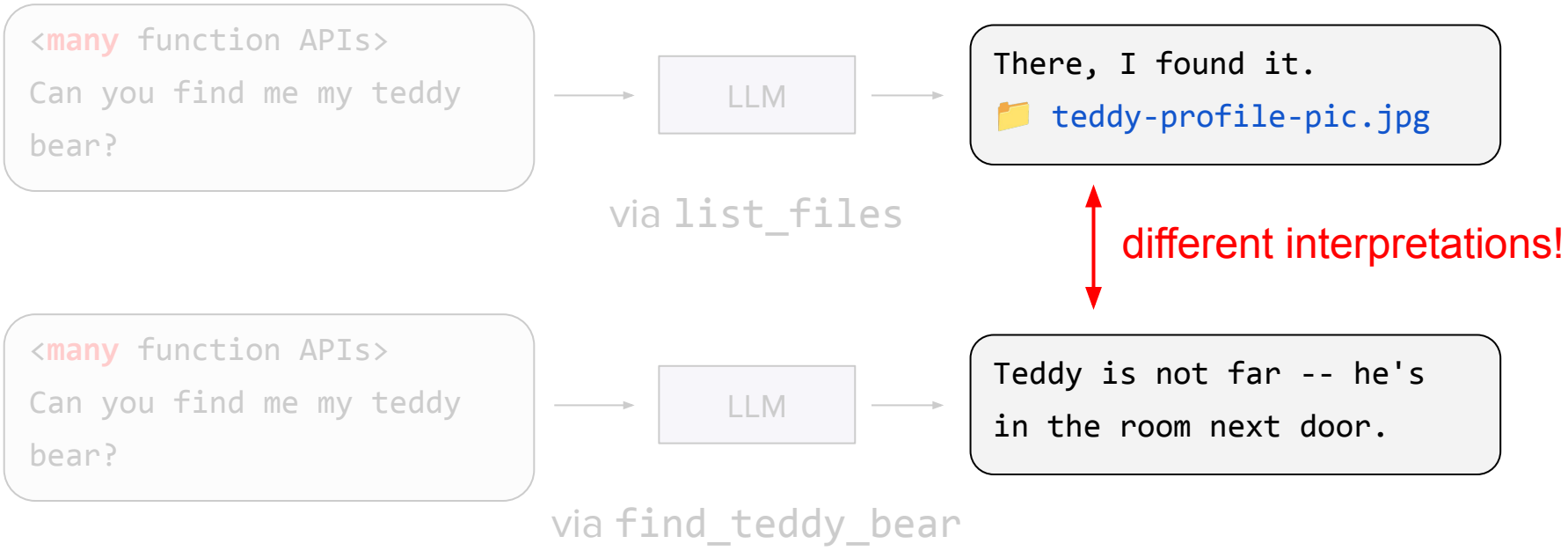
Risks for conflicts



Risks for conflicts



Risks for conflicts



Risks for conflicts

set_temperature

my_position

get_drive

order_food

list_files

search

resolve_id

get_weather

play_music

get_time

calculate

find_teddy_bear

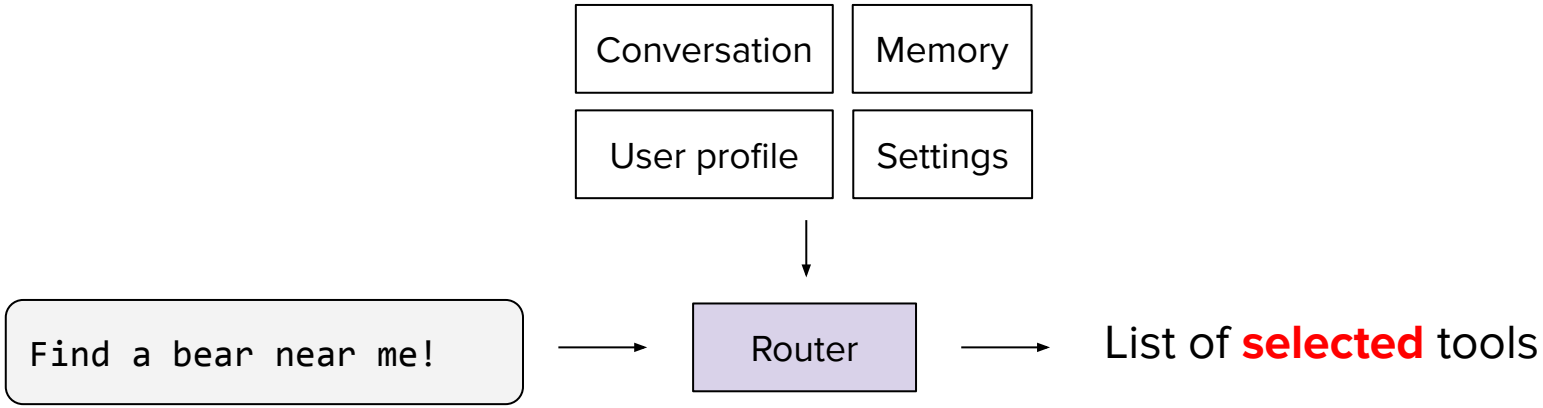
get_doc

fetch_data

Risks for conflicts

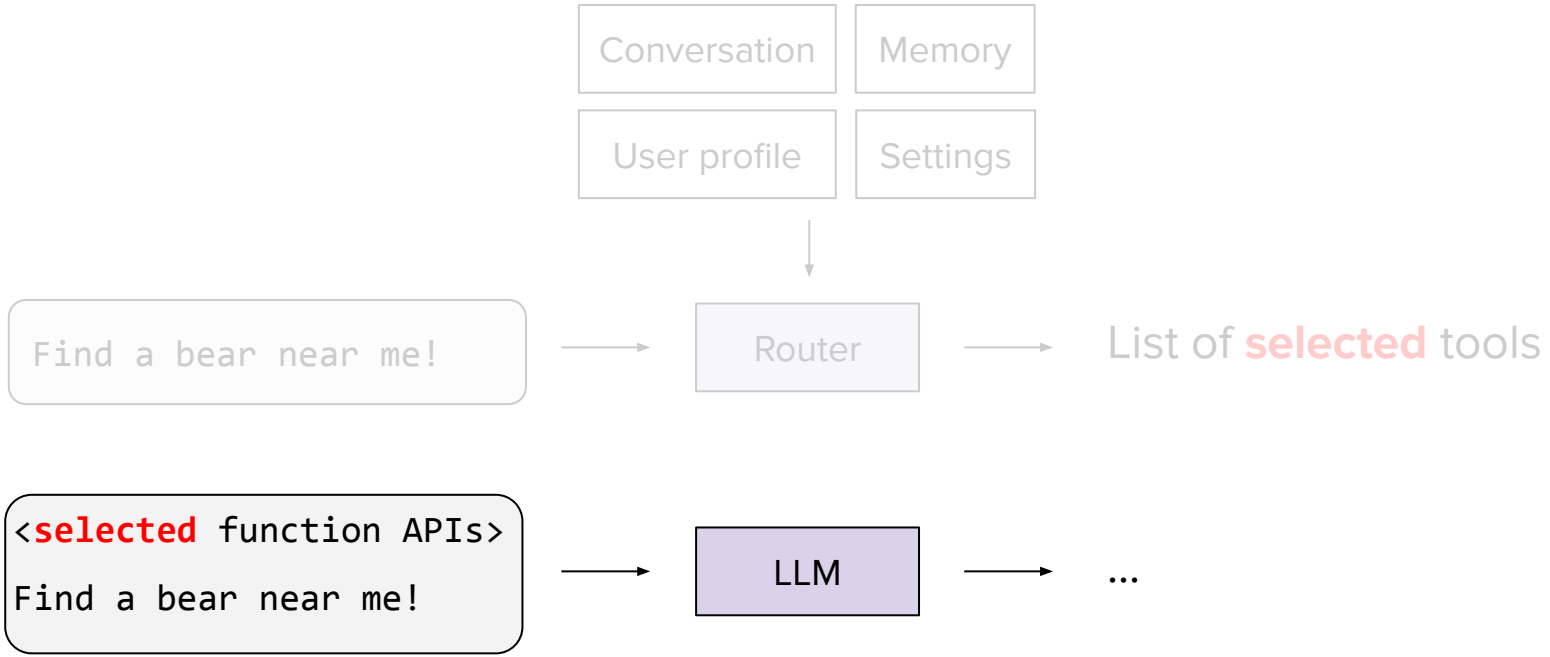


Tool selection



Automatic Tool Selection to Reduce Large Language Model Latency, Robert et al., 2024.

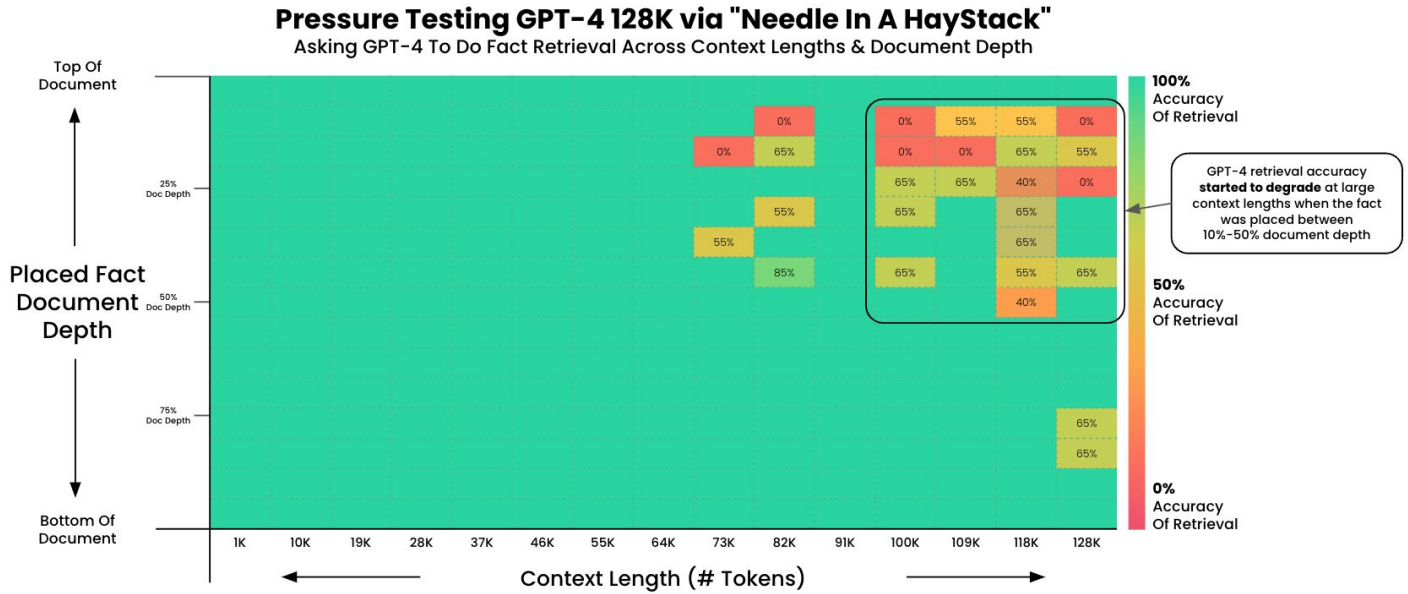
Tool selection



Automatic Tool Selection to Reduce Large Language Model Latency, Robert et al., 2024.

Dangers of context bloat

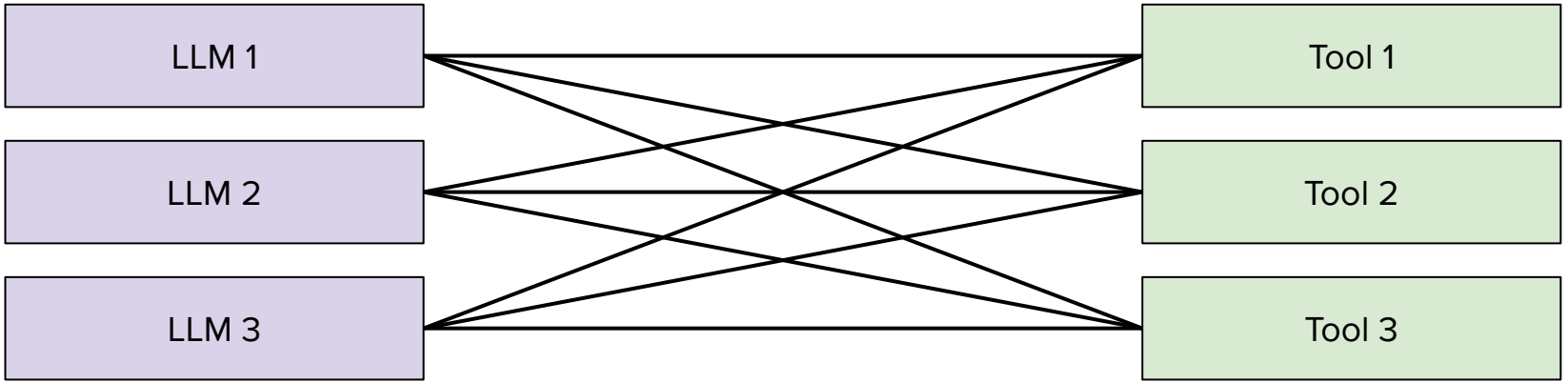
Observation. Even if no conflicts, too many tools can hurt performance!



Screenshot from https://github.com/gkamradt/needle-in-a-haystack, 2023.

Motivation for standardized protocol

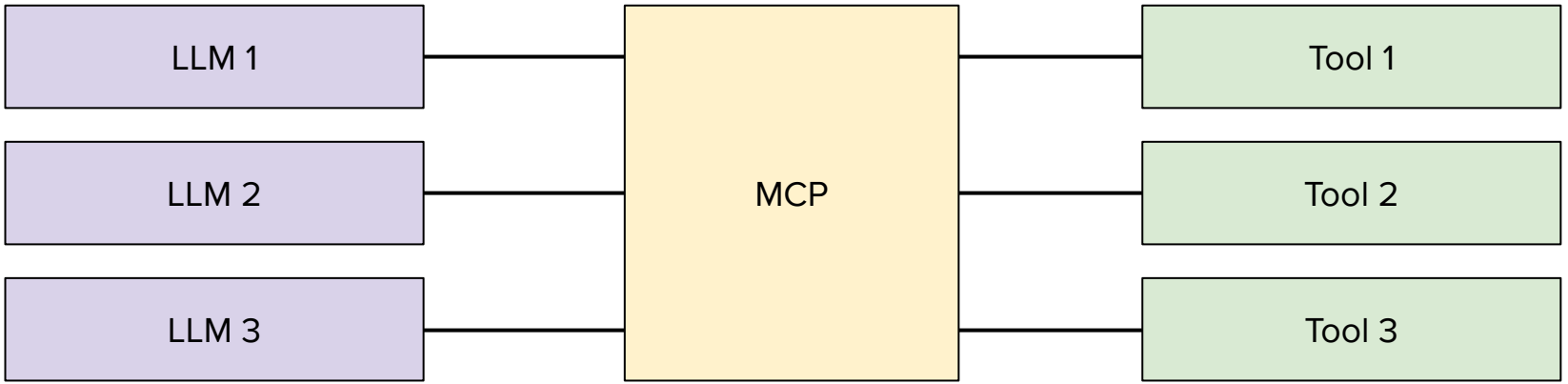
Goal. Avoid duplication of tool implementations



Standardization with MCP

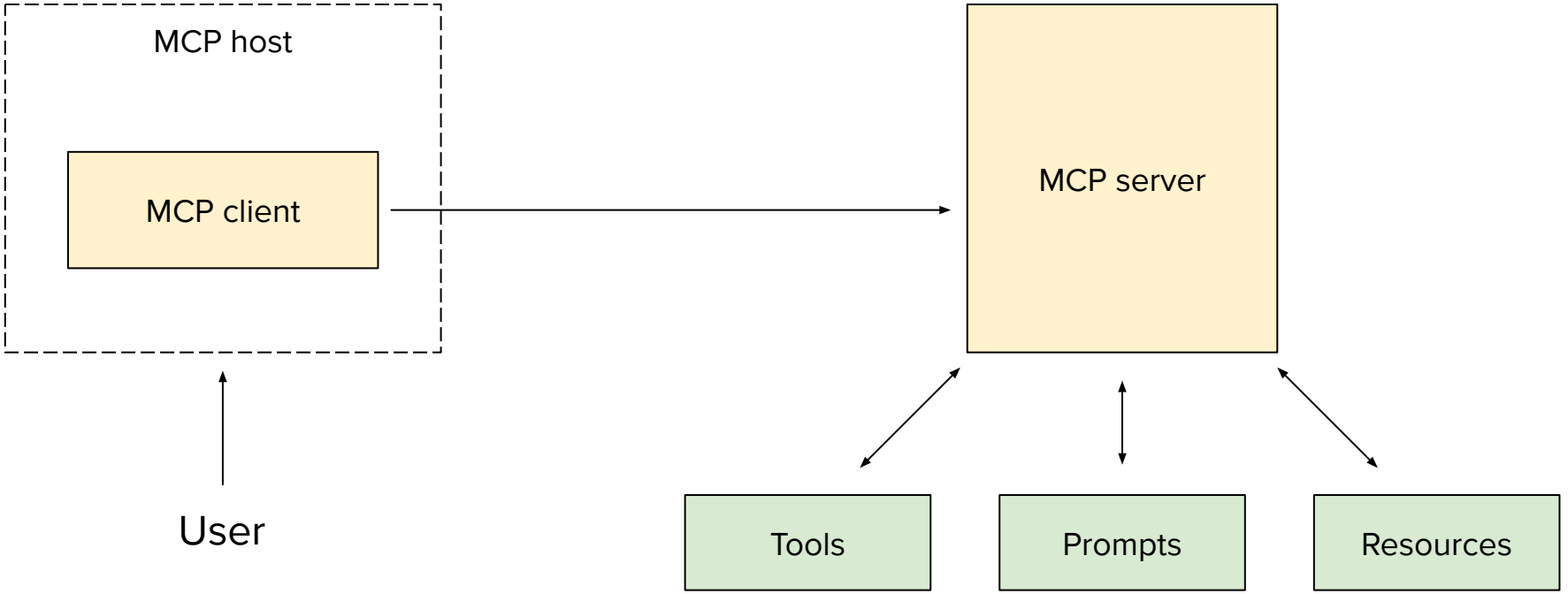
MCP = **M**odel **C**ontext **P**rotocol

Idea. Connect tools/data to LLMs in a standard way



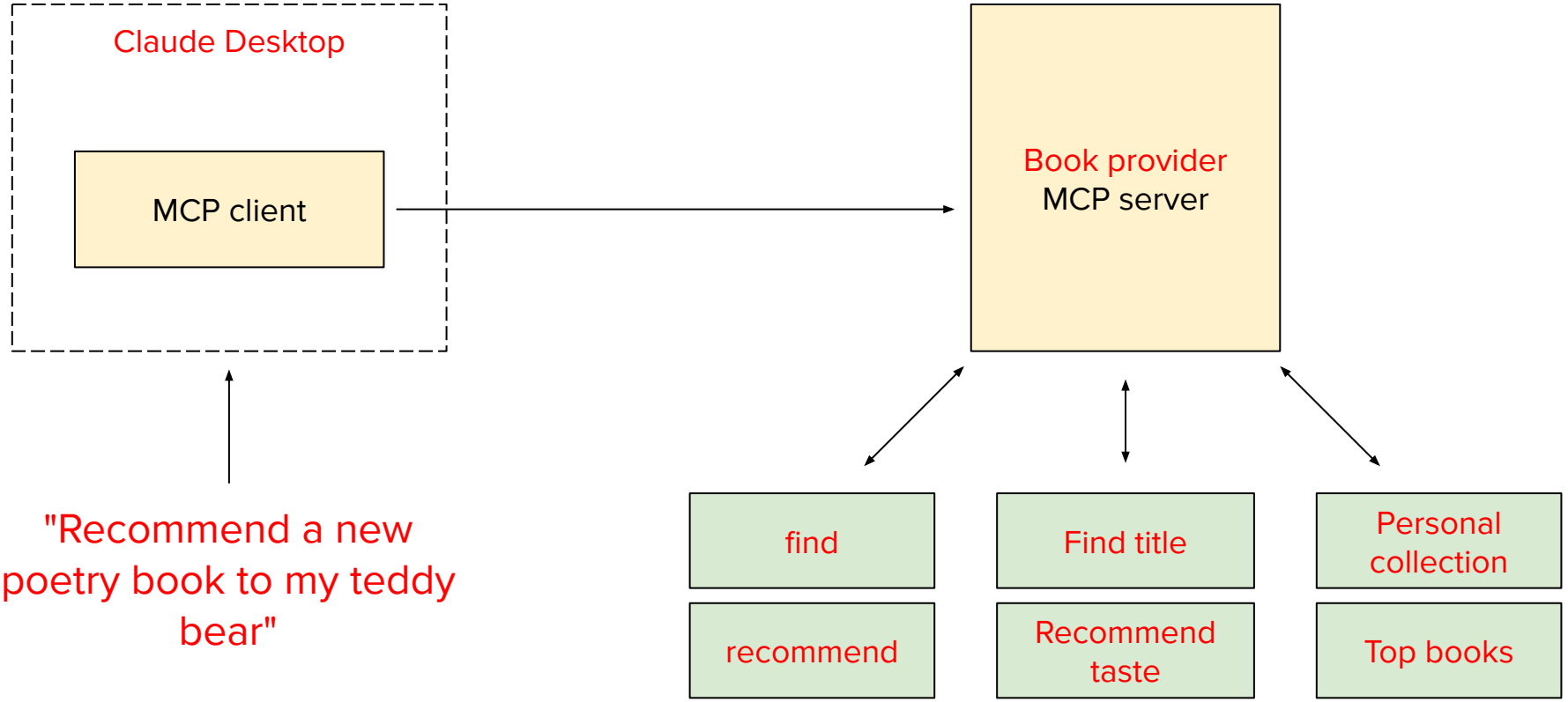
Introducing the Model Context Protocol, Anthropic, 2024.

Standardization with MCP



About MCP > Architecture overview, Anthropic, 2024.

Standardization with MCP



About MCP > Architecture overview, Anthropic, 2024.

What about costs?

Observation. Tools = additional tokens. How to avoid an expensive bill?

Solution. Leverage caching!

Standard | Batch | Flex | Priority | Contributor

Feature	Free Tier	Paid Tier, per 1M tokens in USD
Input price	Not available	$2.00 through November 30, 2026. $4.00 starting December 1, 2026.
Output price (including thinking tokens)	Not available	$10.00 through November 30, 2026. $20.00 starting December 1, 2026.
Context caching price	Not available	$0.10 through November 30, 2026. $0.20 starting December 1, 2026. Implicit caching only (explicit caching is not supported).

Model pricing

The following table shows pricing for all Claude models:

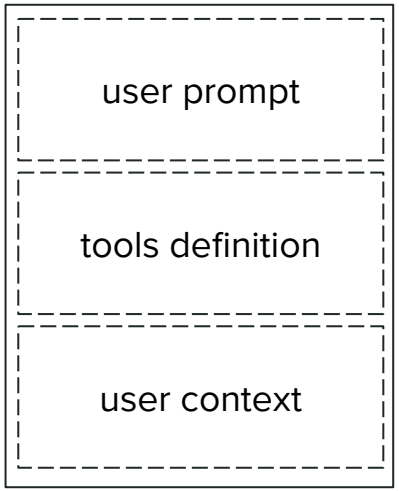
Model	Base tokens		Prompt caching		
Name	Input	Output	5m writes	1h writes	Hits and refreshes
Claude Fable 5.1 For demanding reasoning and long-horizon agentic work	$10 / MTok	$50 / MTok	$12.50 / MTok	$20 / MTok	$0.25 / MTok
Claude Opus 5.5 For long-running agentic coding and knowledge work	$4 / MTok	$20 / MTok	$5 / MTok	$8 / MTok	$0.20 / MTok

10-40x cheaper!

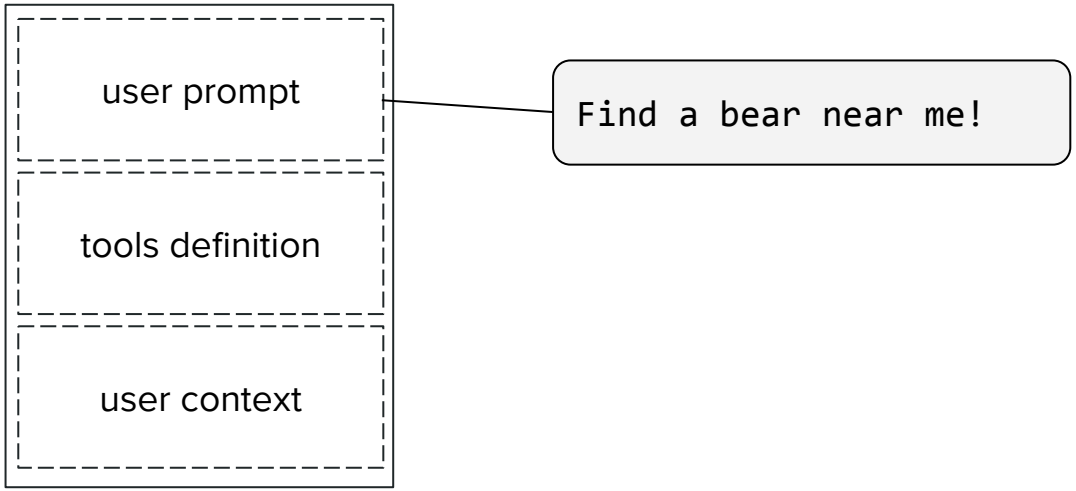
Model	Input	Output	Cached input
gpt-6-astra	$10.00	$50.00	$1.00
gpt-6.1-sol	$2.00	$10.00	$0.10
gpt-6.1-terra	$1.00	$5.00	$0.05

Screenshots of Gemini, Claude and OpenAI developer pricing pages taken on November 4th, 2026.

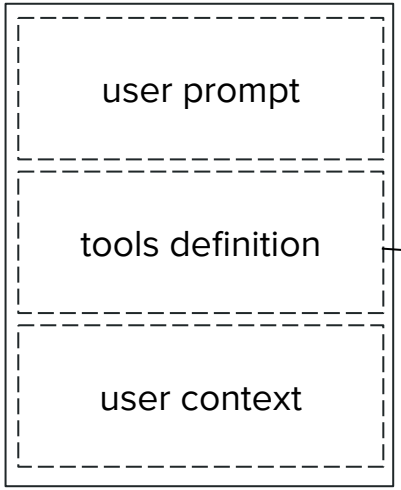
What about costs?



What about costs?



What about costs?




```python
from dataclasses import dataclass
from geopy.distance import geodesic
import requests

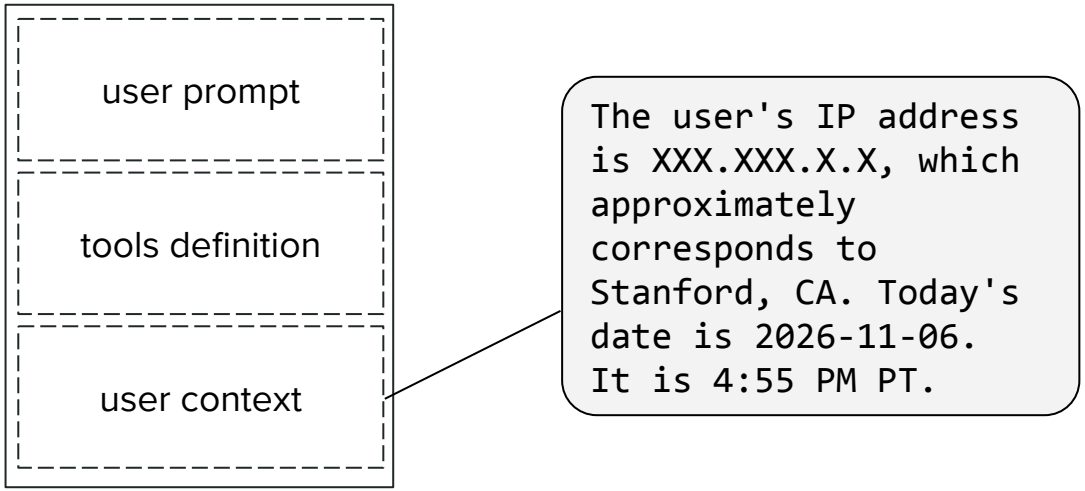
@dataclass
class TeddyBearInfo:
    name: str
    distance_meters: float
    mood: str
    message: str

def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
    """
    Finds the nearest teddy bear to the given GPS coordinates.

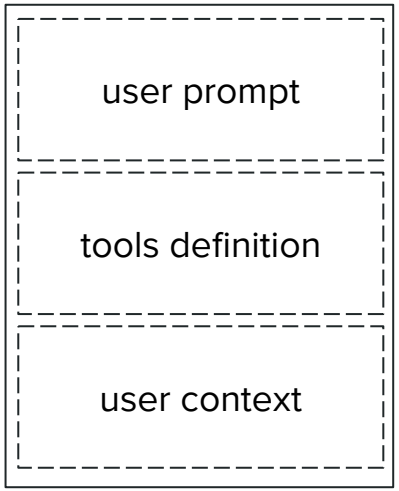
    Parameters:
        location: A (latitude, longitude) pair representing the user's current
            location.

    Returns:
        TeddyBearInfo: Information about the nearest teddy bear found.
    """
    ...
```

What about costs?

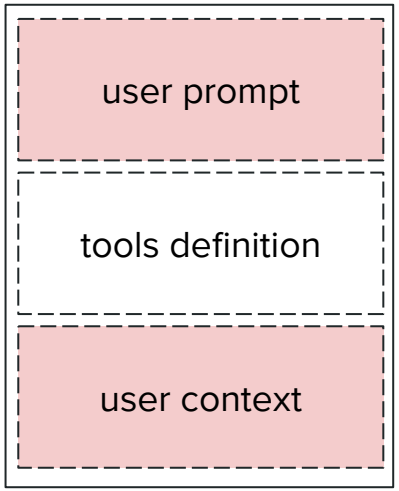


What about costs?



Is this ordering good?

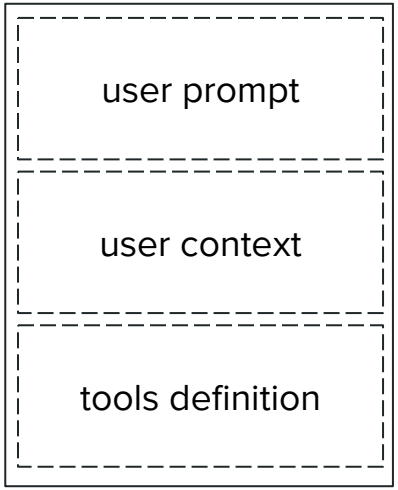
What about costs?



Bad

What about costs?

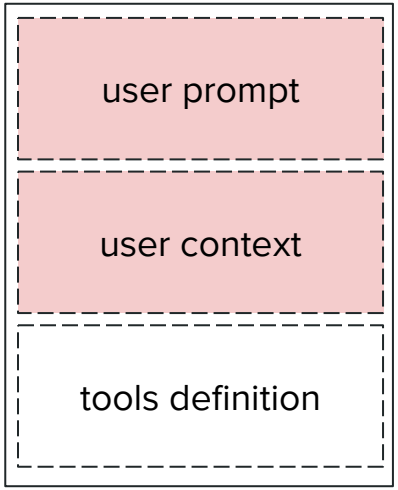
Bad



Is this ordering good?

What about costs?

Bad

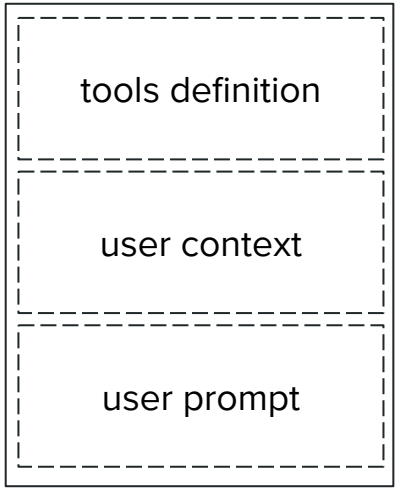


Bad

What about costs?

Bad

Bad



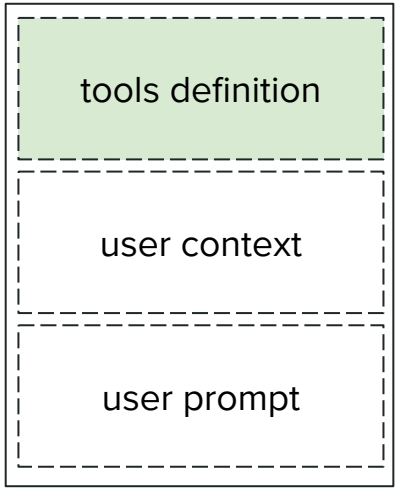
Is this ordering good?

What about costs?

Tip. Frontload all fixed parts to the beginning of the prompt!

Bad

Bad



Good



Transformers & Large Language Models

Motivation

Retrieval

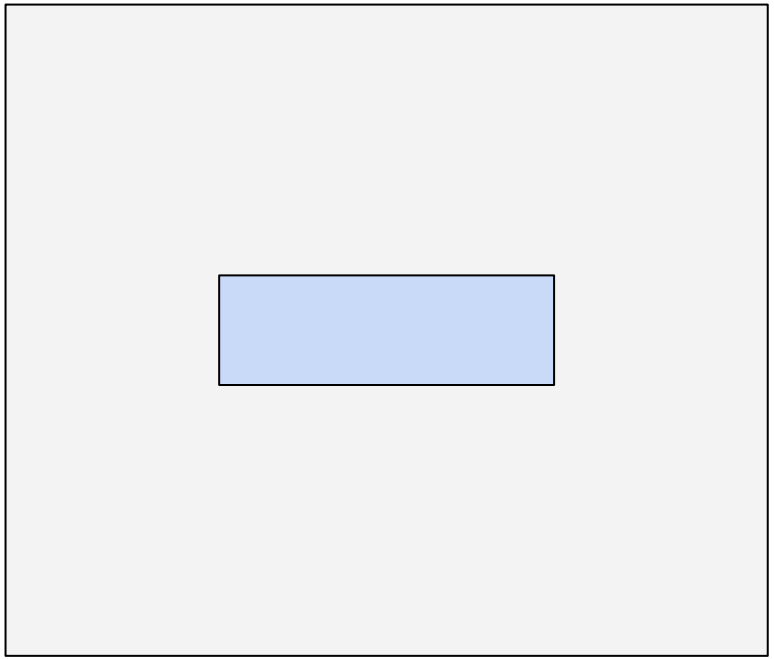
Tool calling

Harness engineering

Coding agents

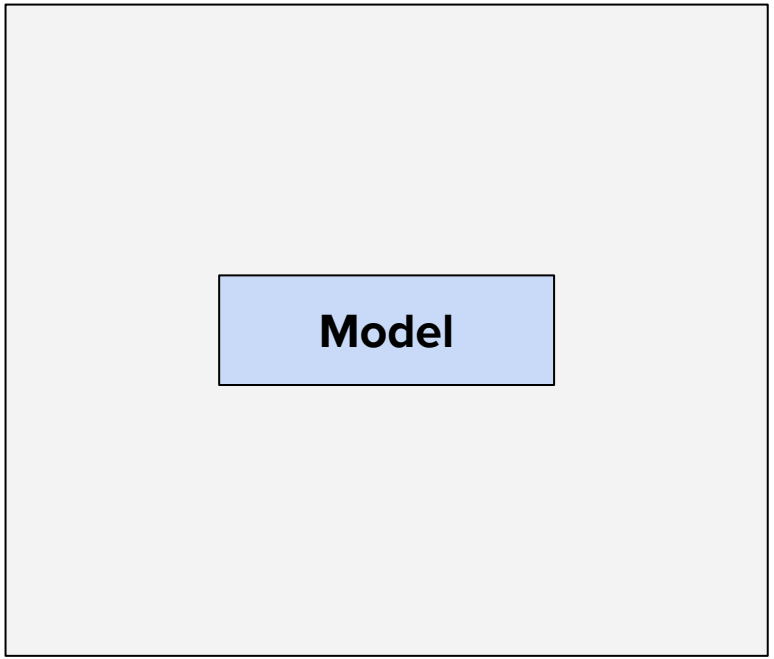
Discussion

Agent definition



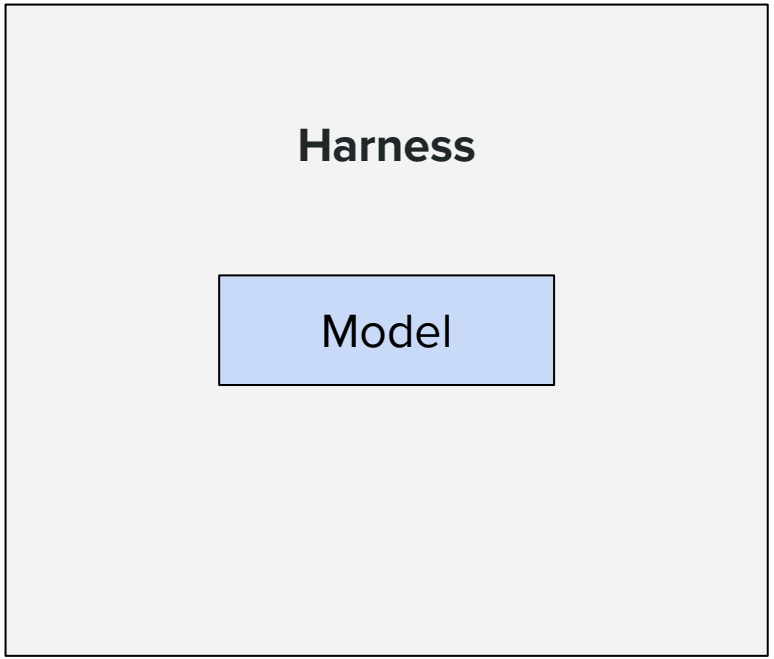
What is an AI Agent Harness?, Databricks, 2026.

Agent definition



What is an AI Agent Harness?, Databricks, 2026.

Agent definition



- Tools
- Succession of actions
- Compaction strategy
- Memory
- All surrounding prompting
- Guardrails

What is an AI Agent Harness?, Databricks, 2026.

Analogy



Model

+



Harness

=

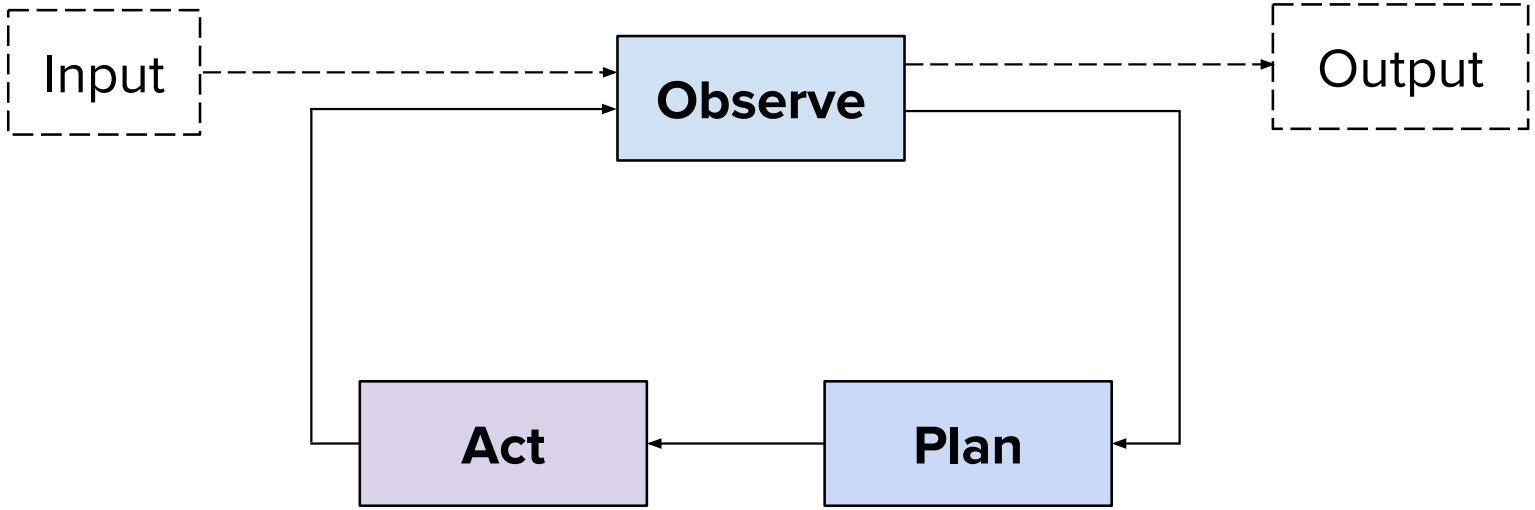


Agent

Illustration generated by ChatGPT, November 2026.

Paradigm

ReAct = **Re**ason + **Act**



ReAct: Synergizing Reasoning and Acting in Language Models, Yao et al., 2023.

ReAct in action

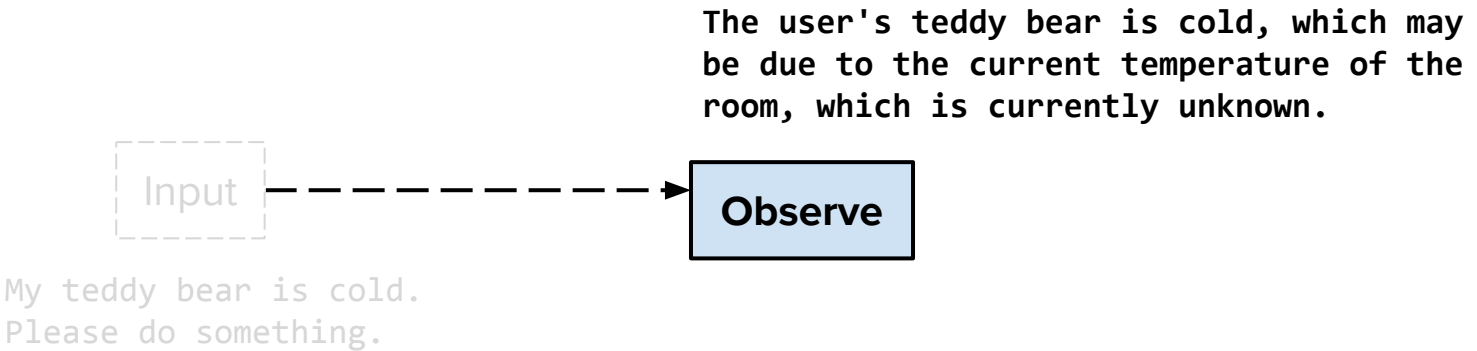



```
My teddy bear is cold.
Please do something.
```

Examples:

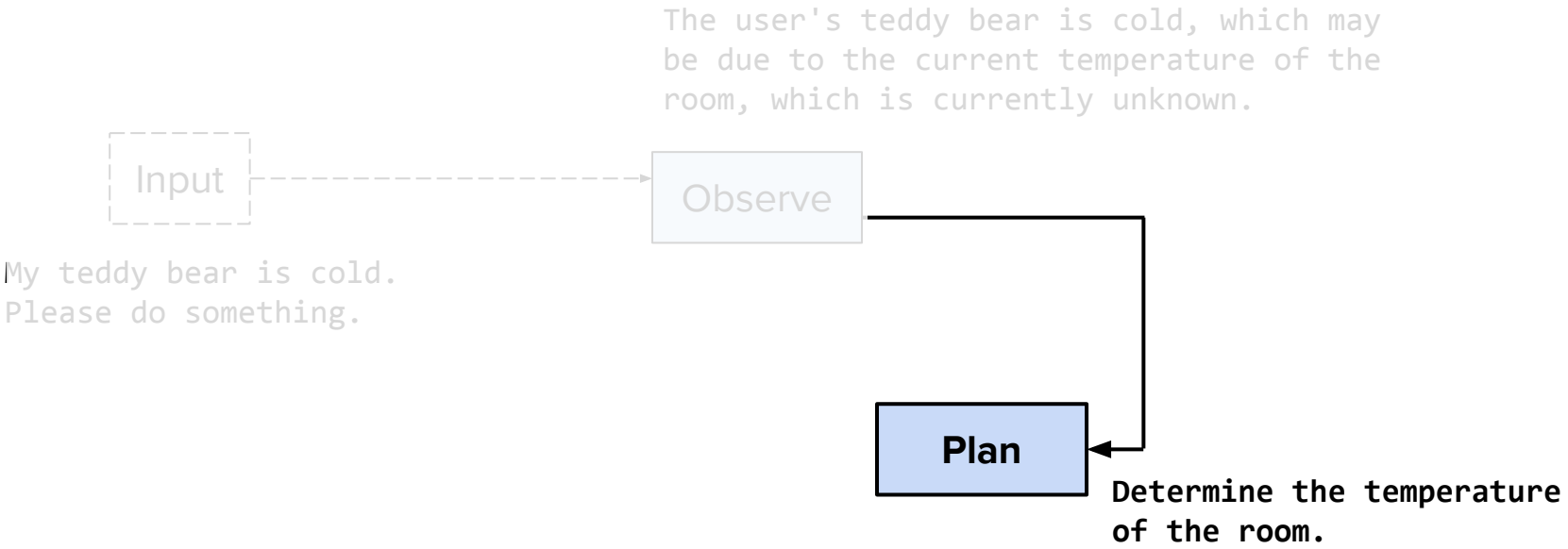
- Manually entered (e.g. user question)
- External event (e.g. metric going beyond a threshold)

ReAct in action



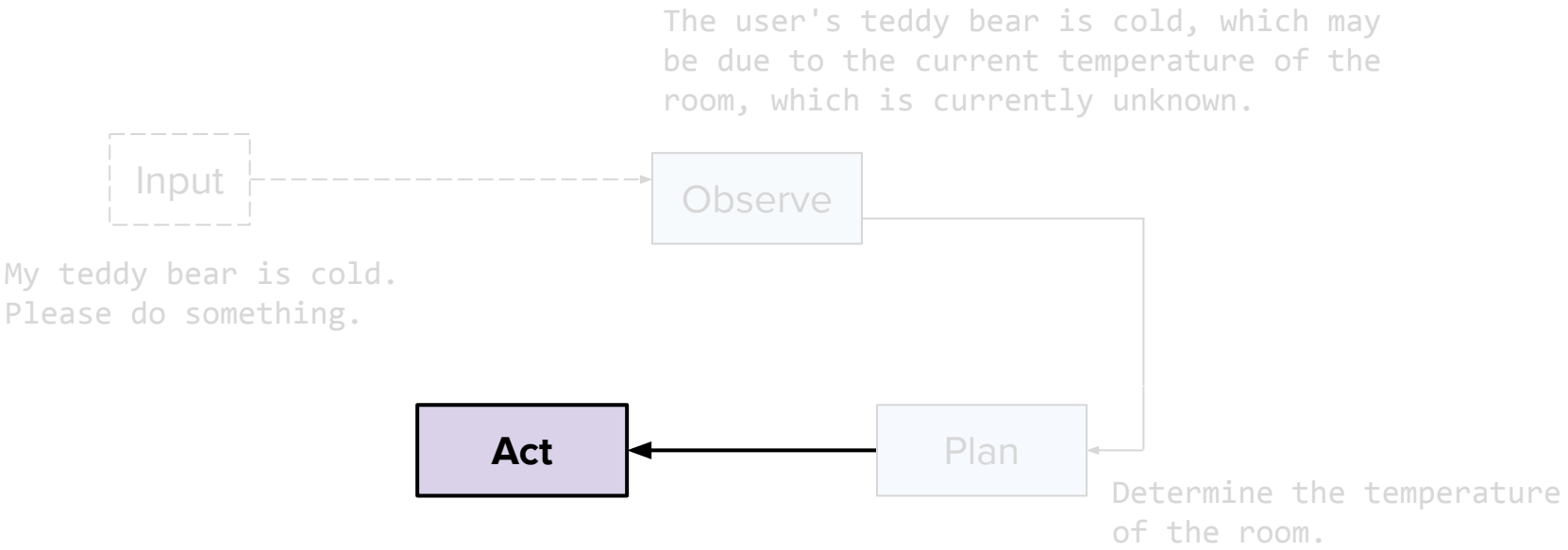
- **Synthesize** previous actions + explicitly **state** what is **currently known** including own knowledge
- **Reasoning**-heavy step to figure out **what is needed**

ReAct in action



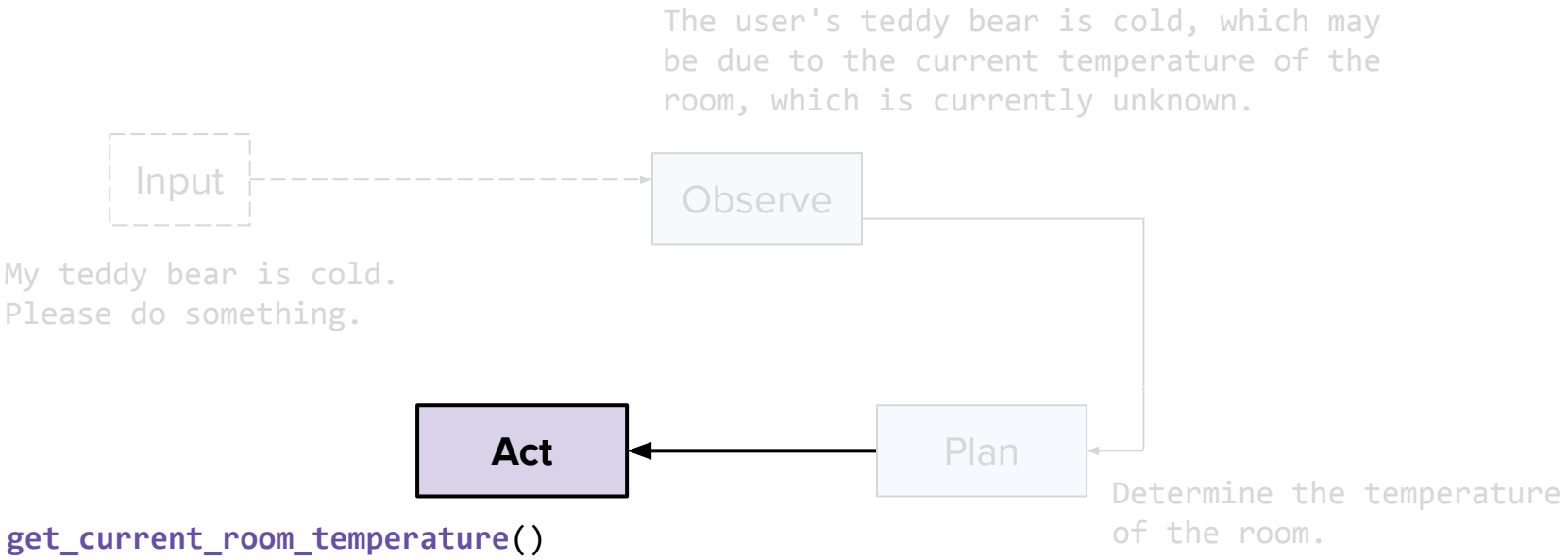
Detail **what tasks** need to be accomplished and **what tools** to call

ReAct in action



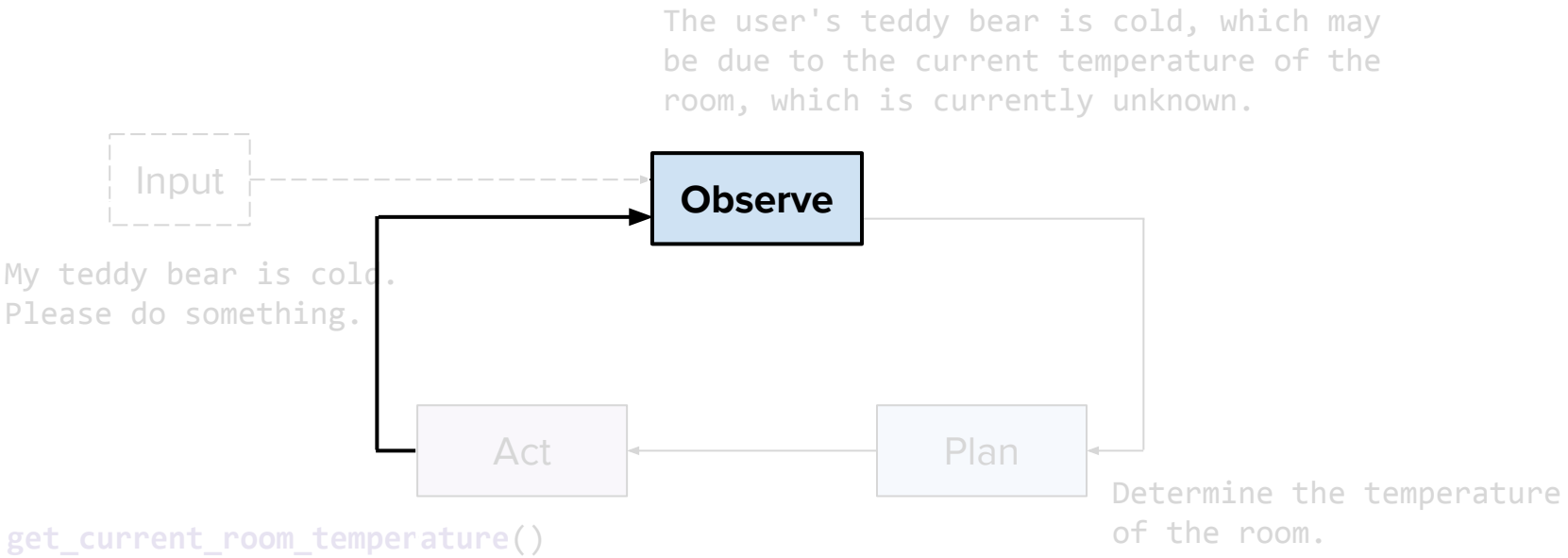
- Perform an action via an **API**
- **Look** for info in a **database** of documents

ReAct in action

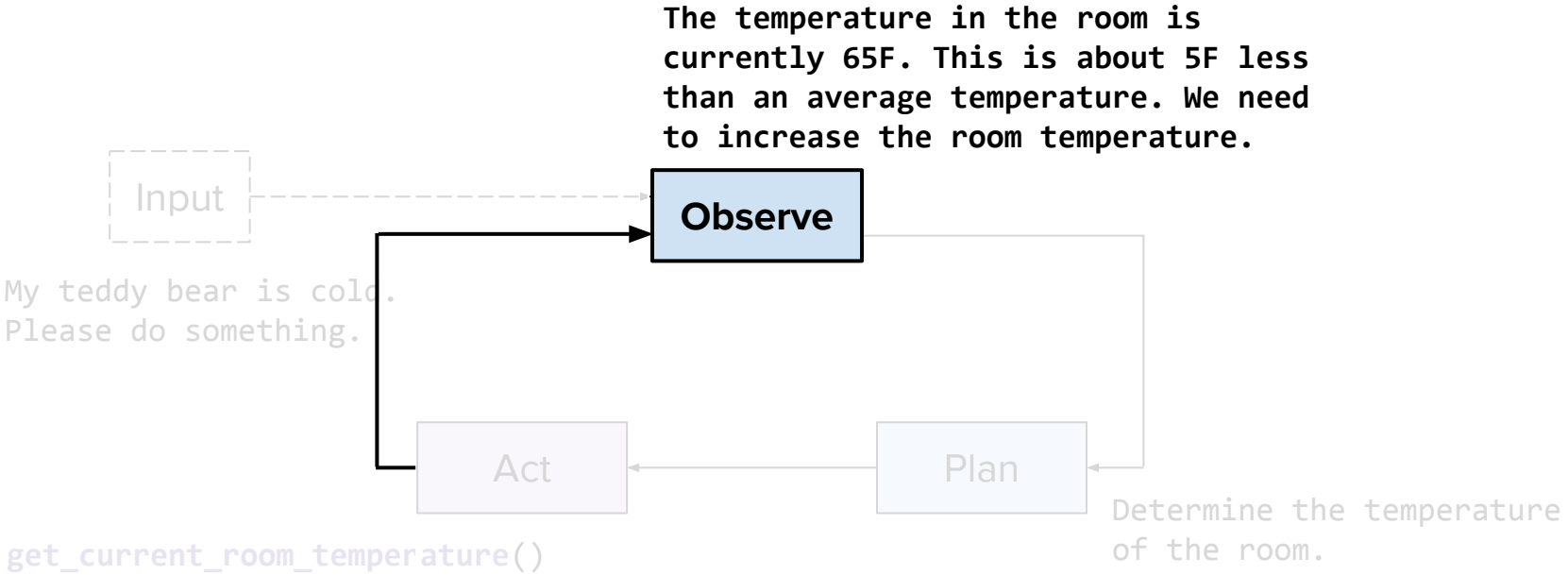


- Perform an action via an **API**
- **Look** for info in a **database** of documents

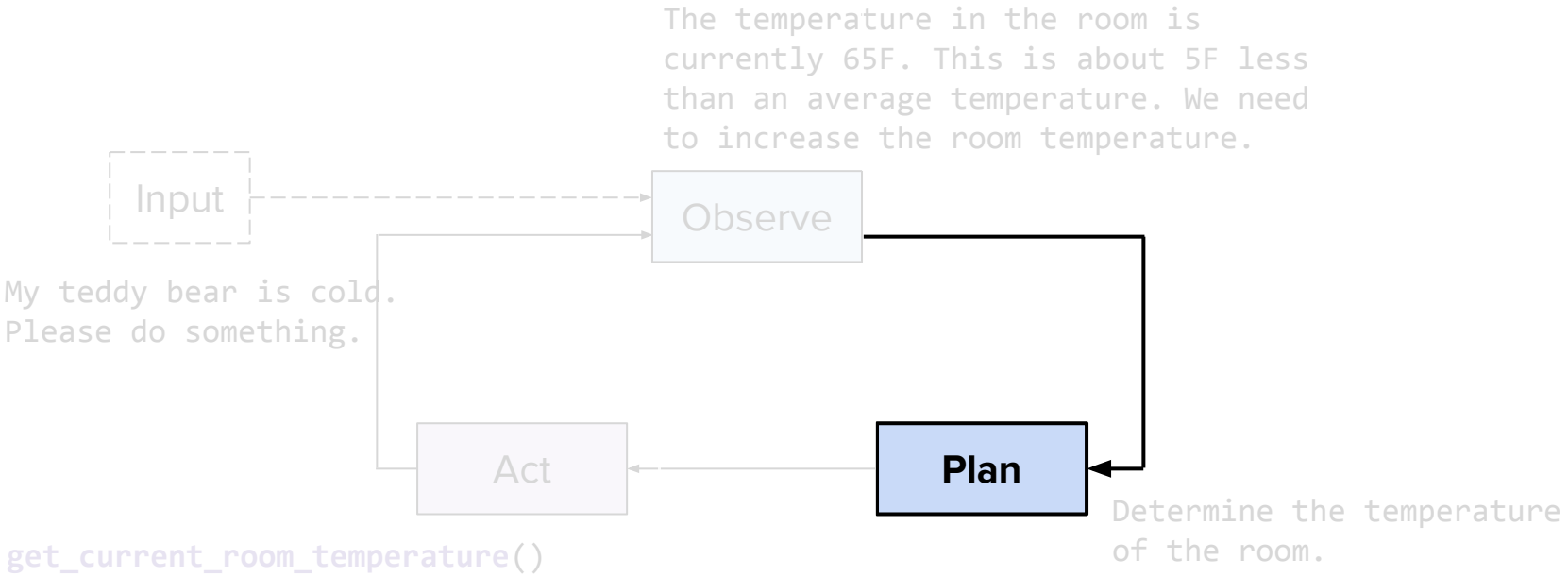
ReAct in action



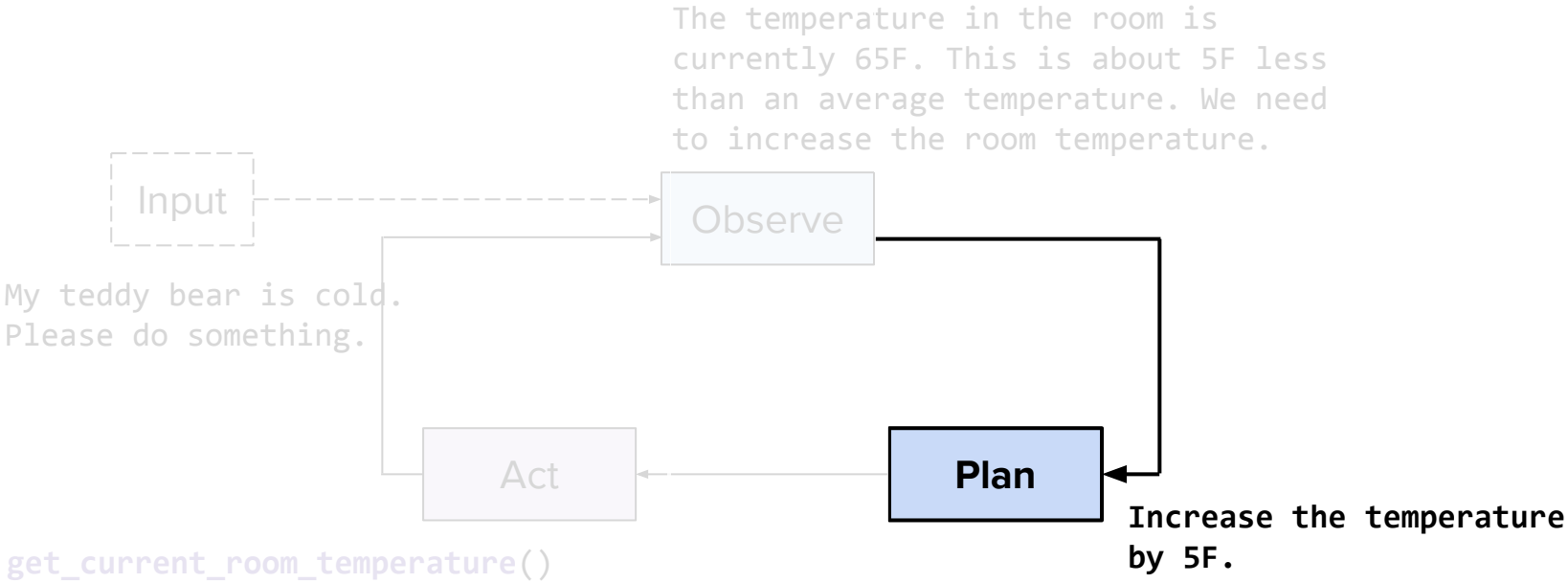
ReAct in action



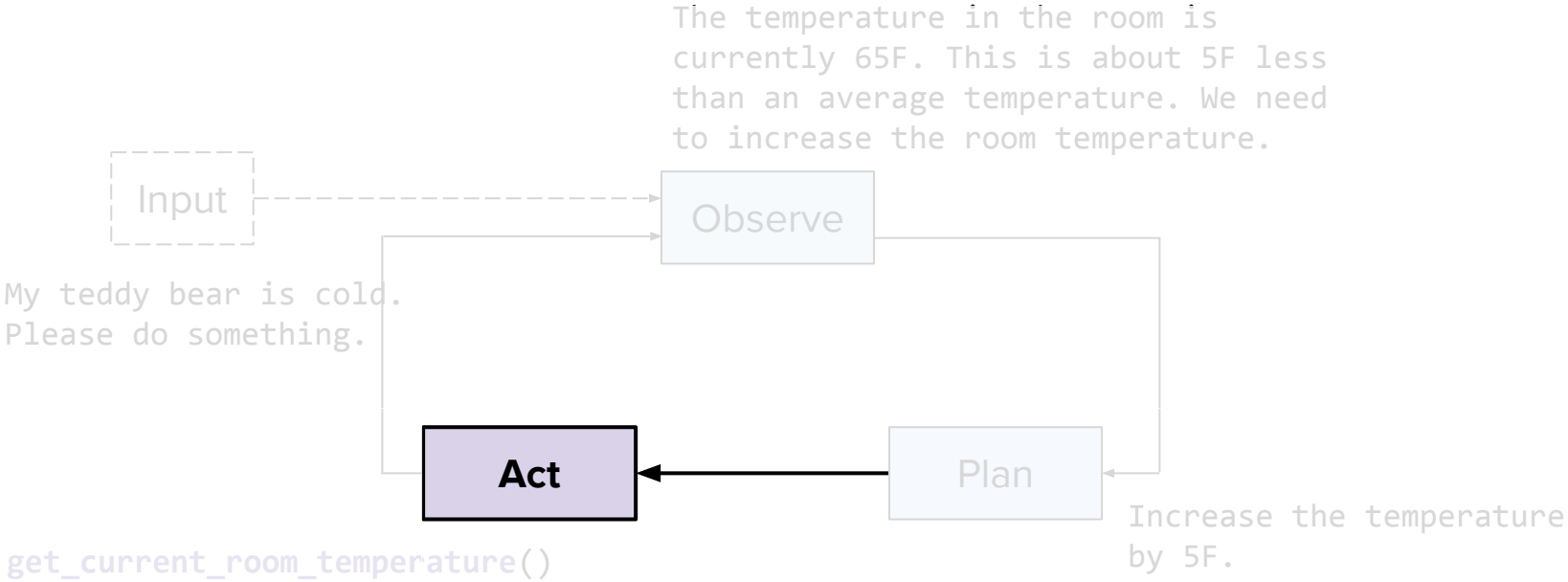
ReAct in action



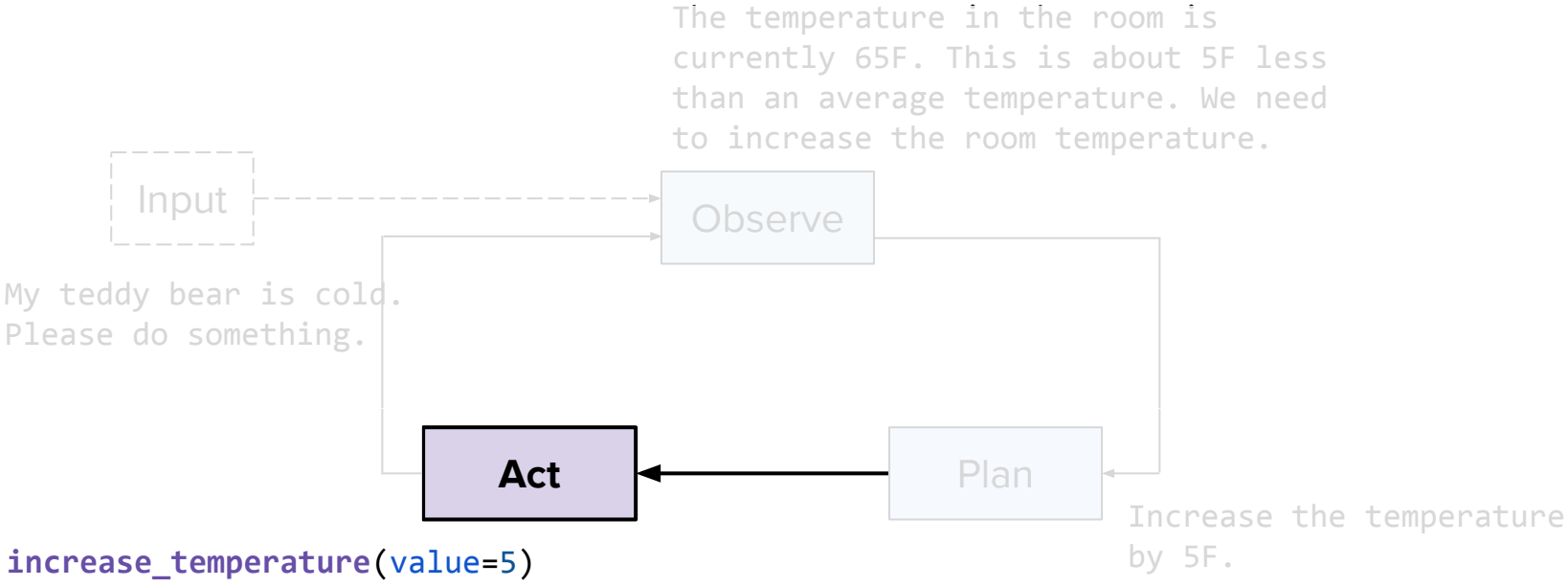
ReAct in action



ReAct in action

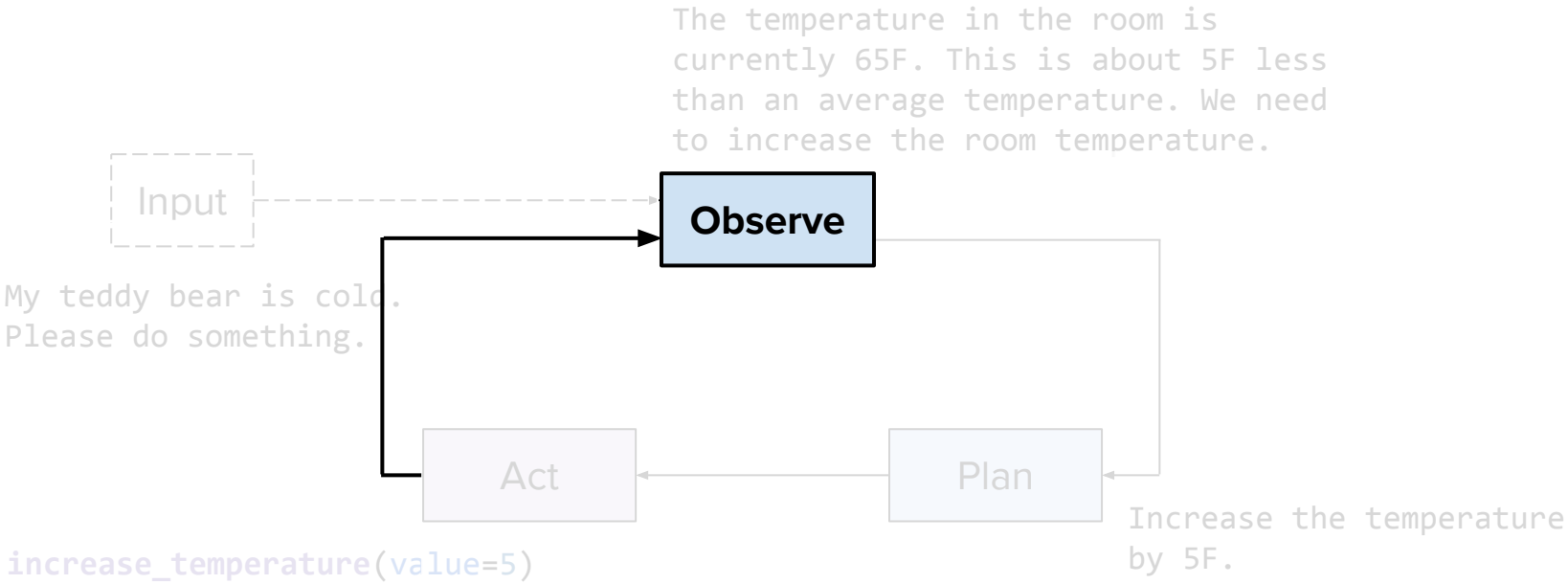


ReAct in action

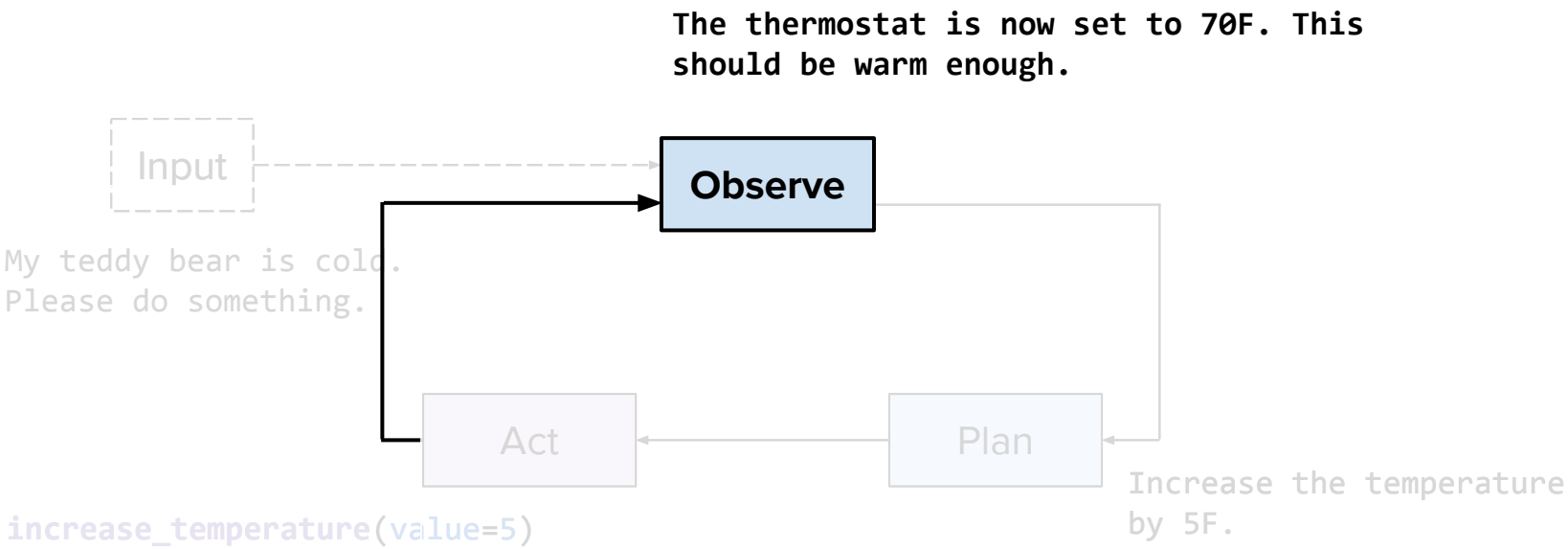



```
increase_temperature(value=5)
```

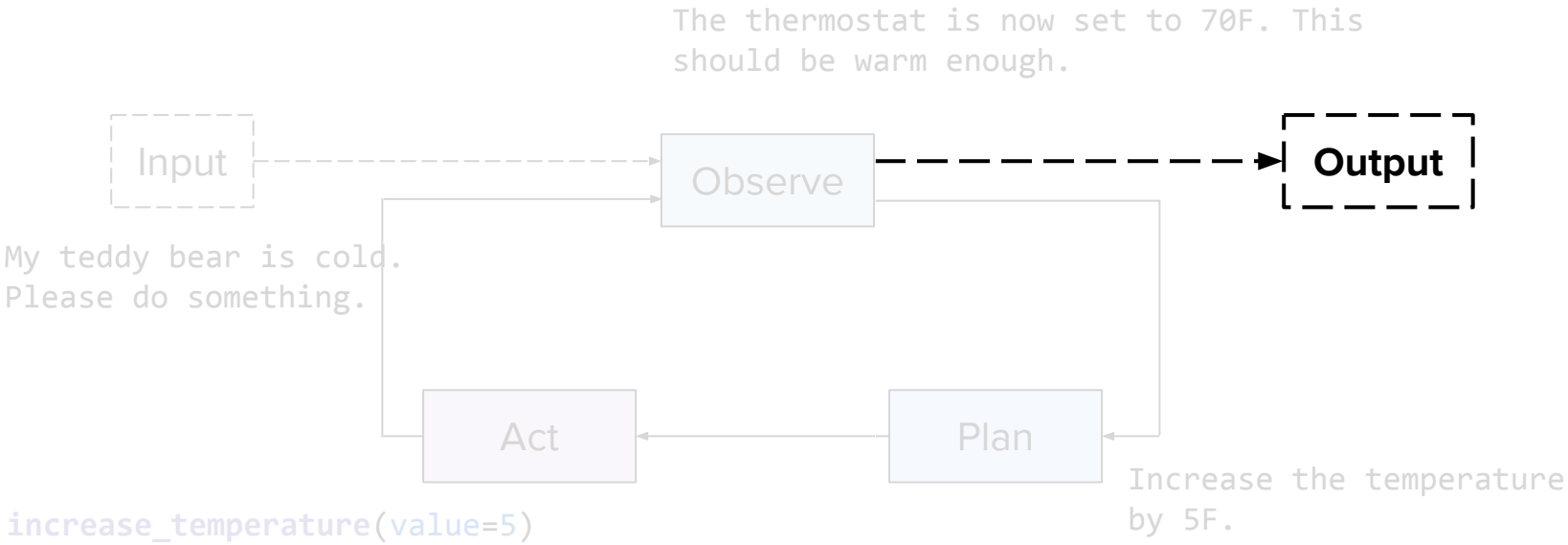
ReAct in action



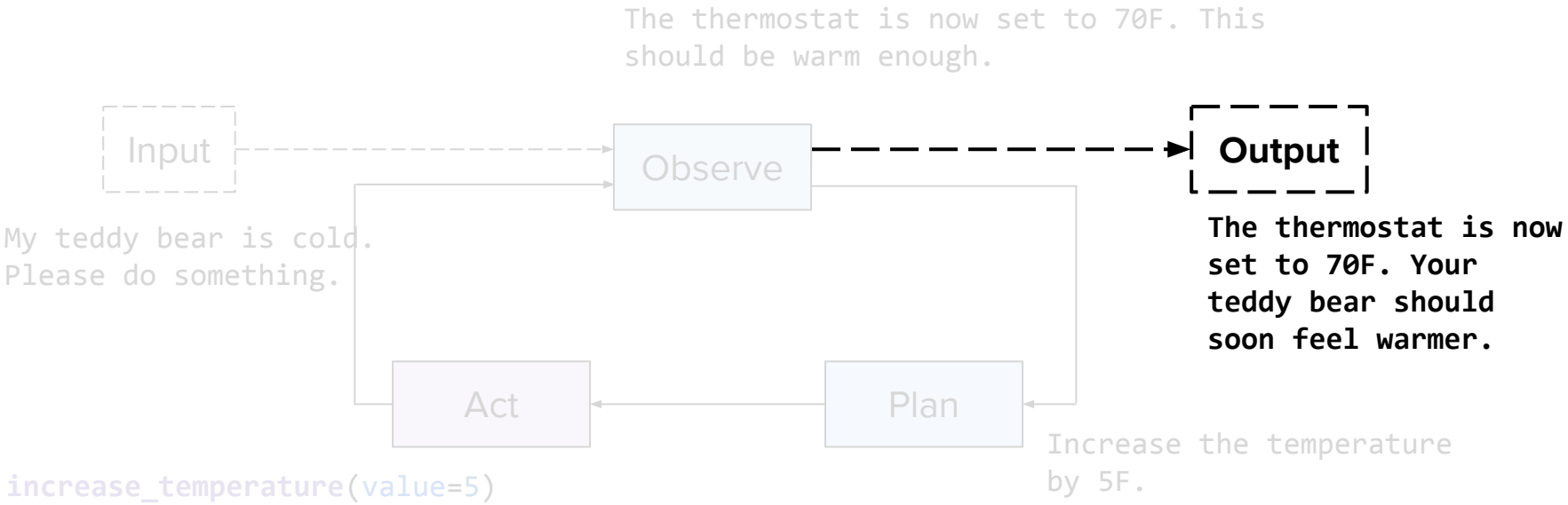
ReAct in action



ReAct in action



ReAct in action



"Agentic" view



There can be agents for many things

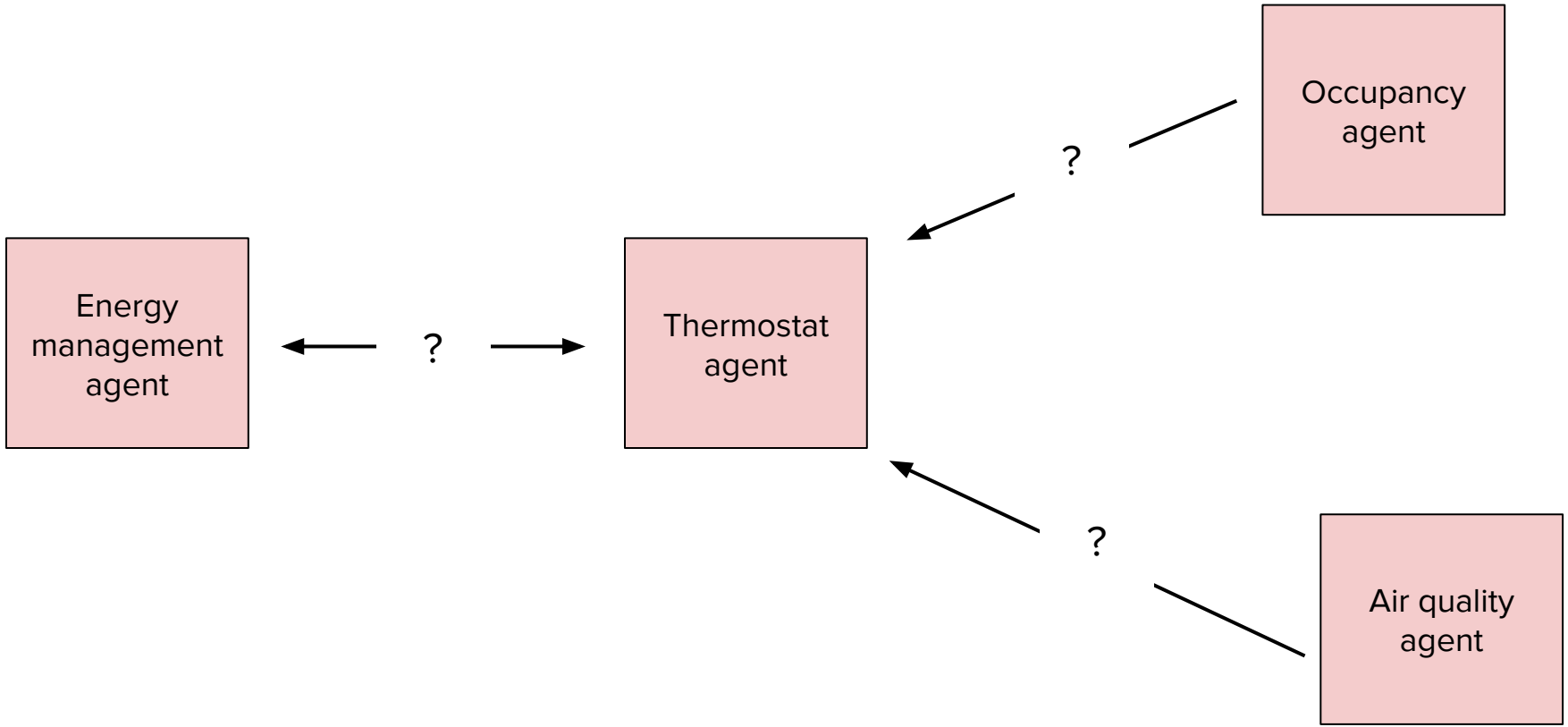
Occupancy agent

Energy management agent

Thermostat agent

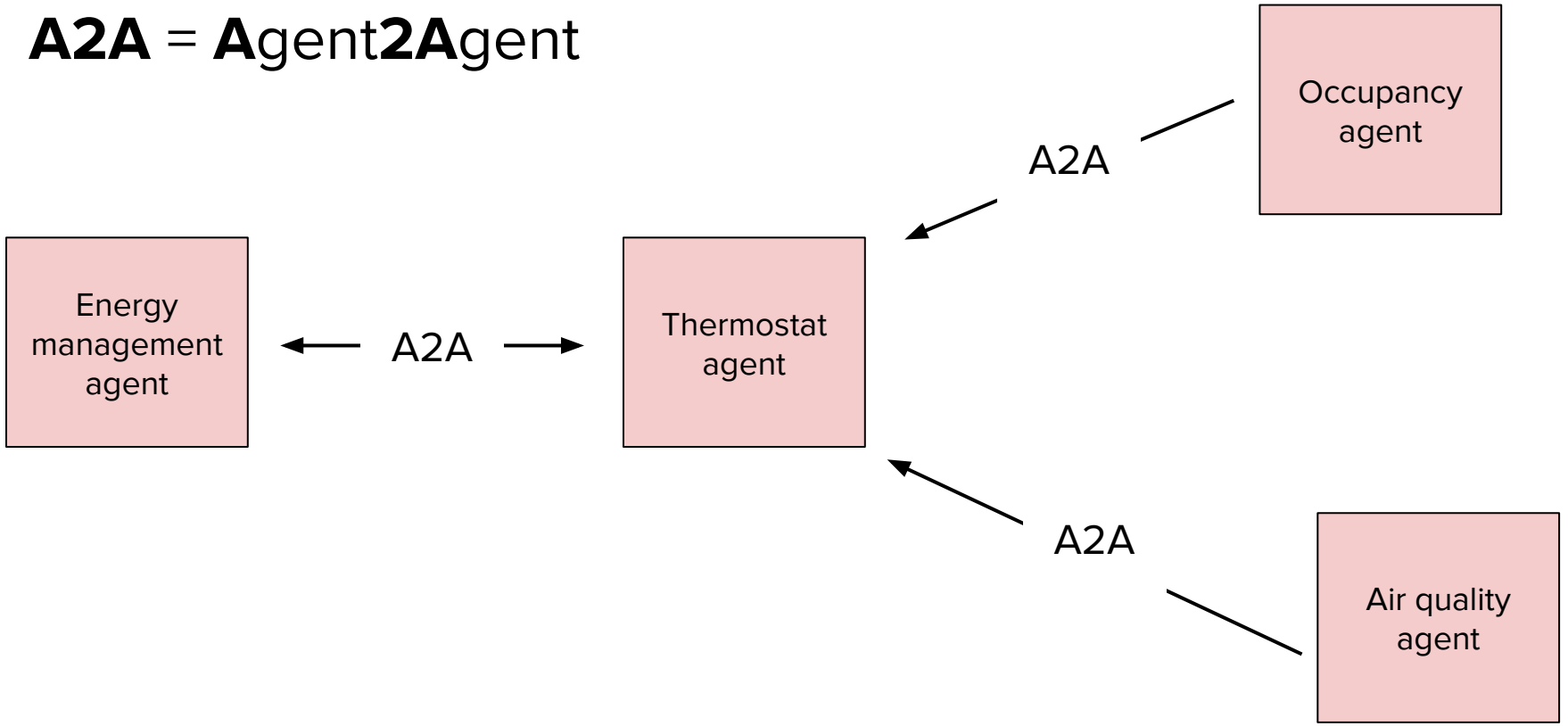
Air quality agent

How can agents communicate with each other?



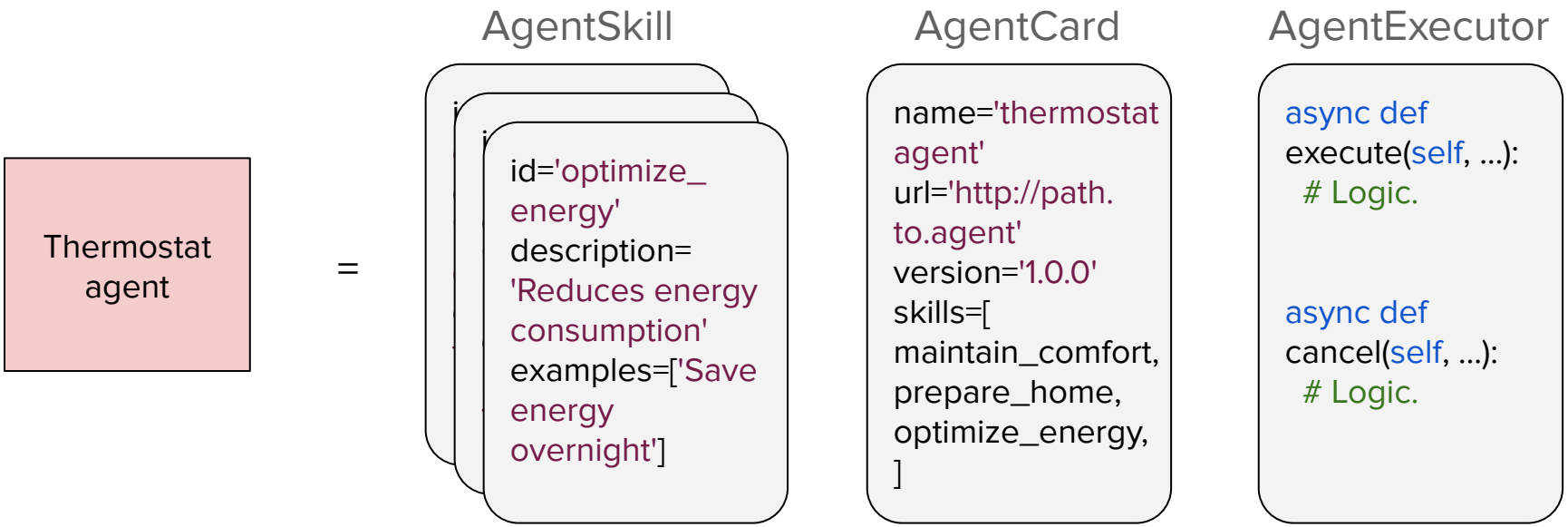
Standardization: A2A

A2A = **A**gent**2A**gent



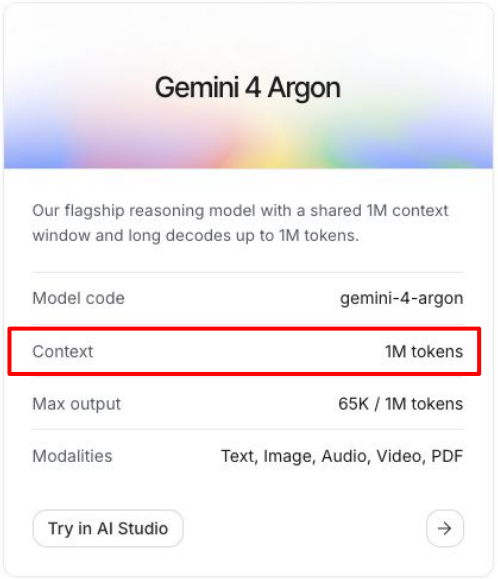
Announcing the Agent2Agent Protocol (A2A), Google, 2025.

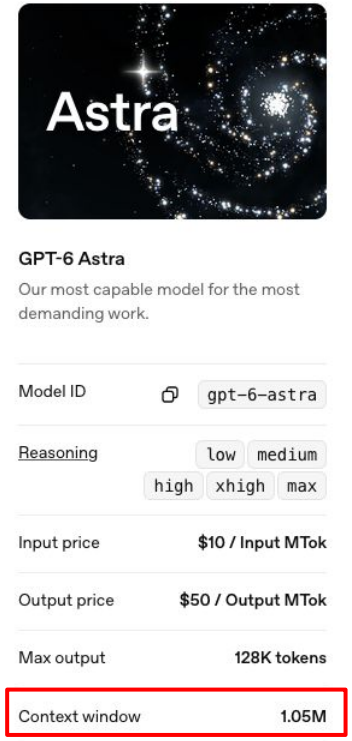
Standardization: A2A

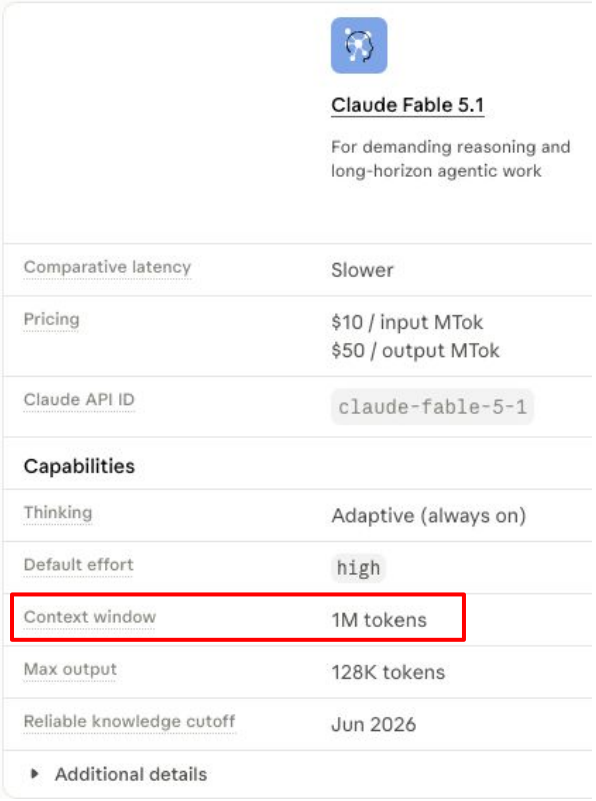


Agent2Agent (A2A) Protocol Official Specification, Google, 2025.

Managing context







What happens when the window fills up?

Screenshots of Gemini, ChatGPT and Claude model specs pages taken on November 5th, 2026.

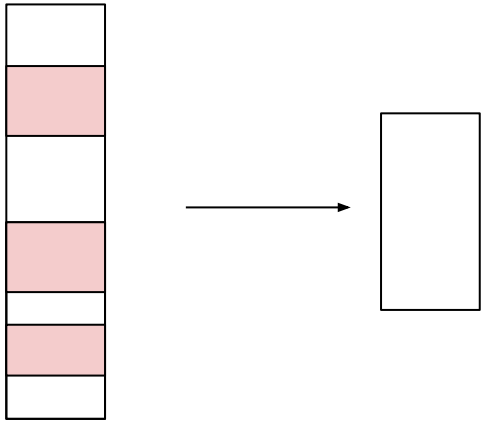
Managing context

Goal. Make some room in the context window!

Context compaction via deletion

Goal. Make some room in the context window!

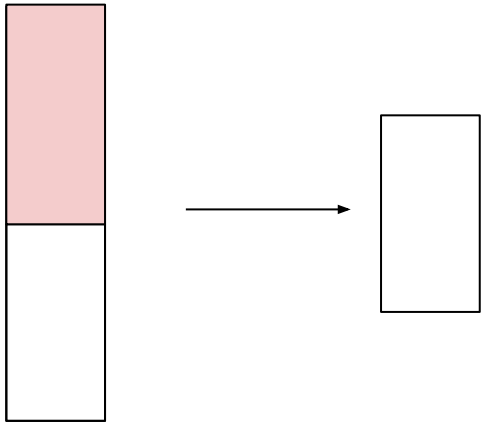
- Deletion



Context compaction via selection

Goal. Make some room in the context window!

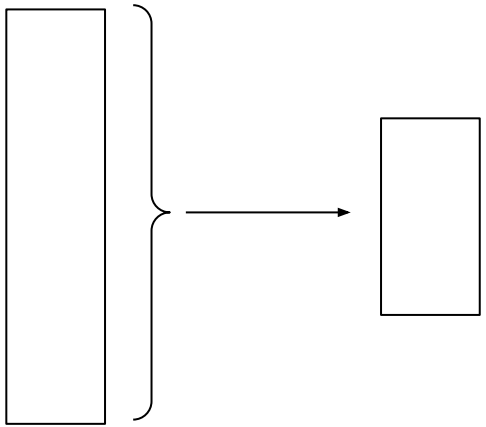
- Deletion
- Sliding window



Context compaction via generation

Goal. Make some room in the context window!

- Deletion
- Sliding window
- Summarization



- ❑ Current progress
- ❑ Constraints
- ❑ Remaining work
- ❑ References

Compaction Reference > Summary Format, Pi documentation, page accessed on November 4th, 2026.

Memory

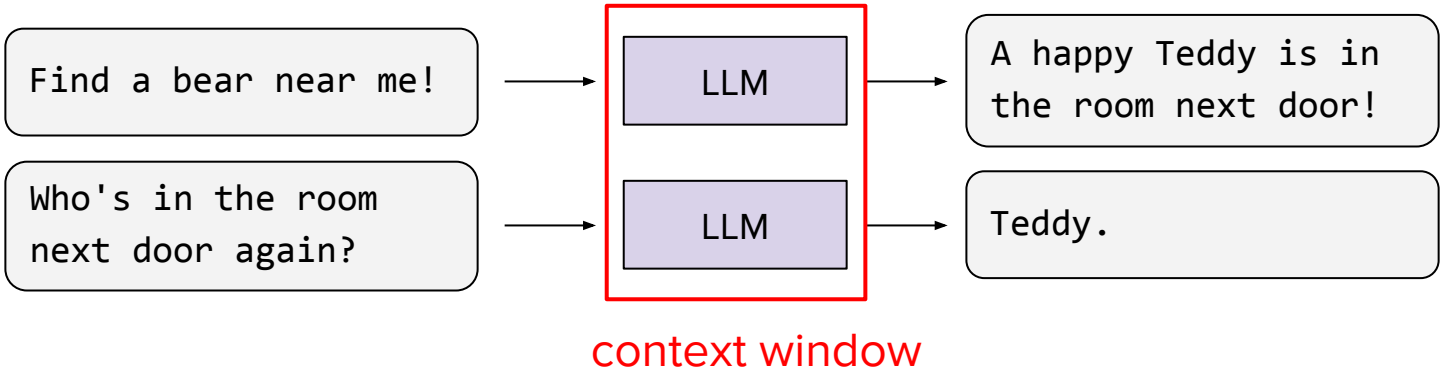
Goal. Make some room in the context window!

- Deletion
- Sliding window
- Summarization

How to make information persist?

Memory

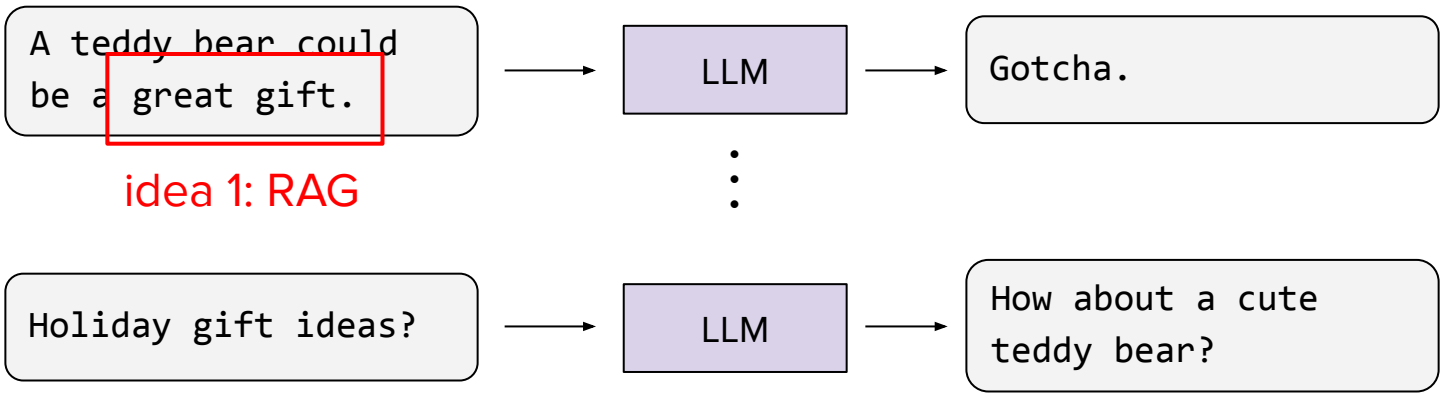
Short term.



Memory

Short term.

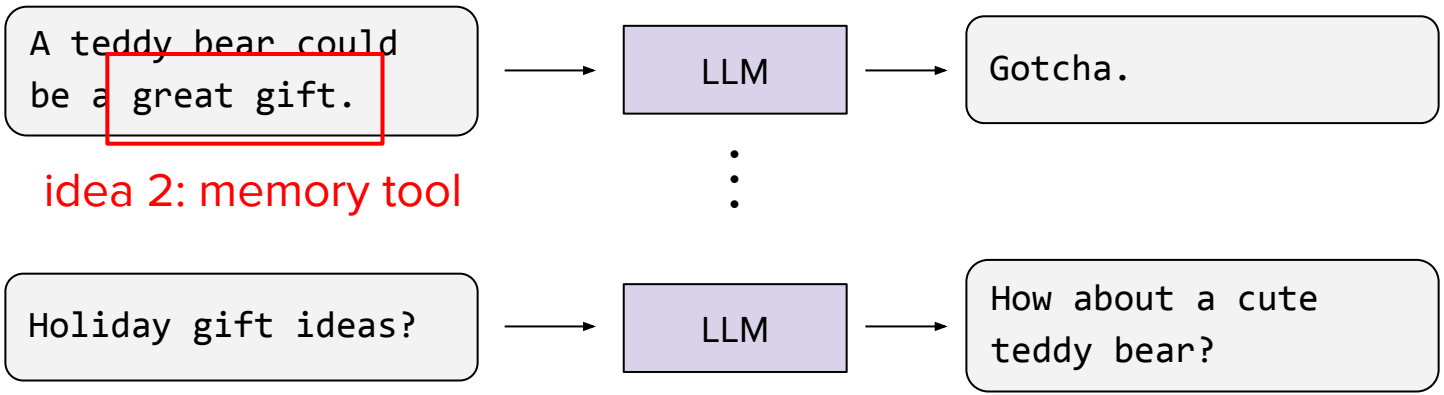
Long term.



Memory

Short term.

Long term.





Transformers & Large Language Models

Motivation

Retrieval

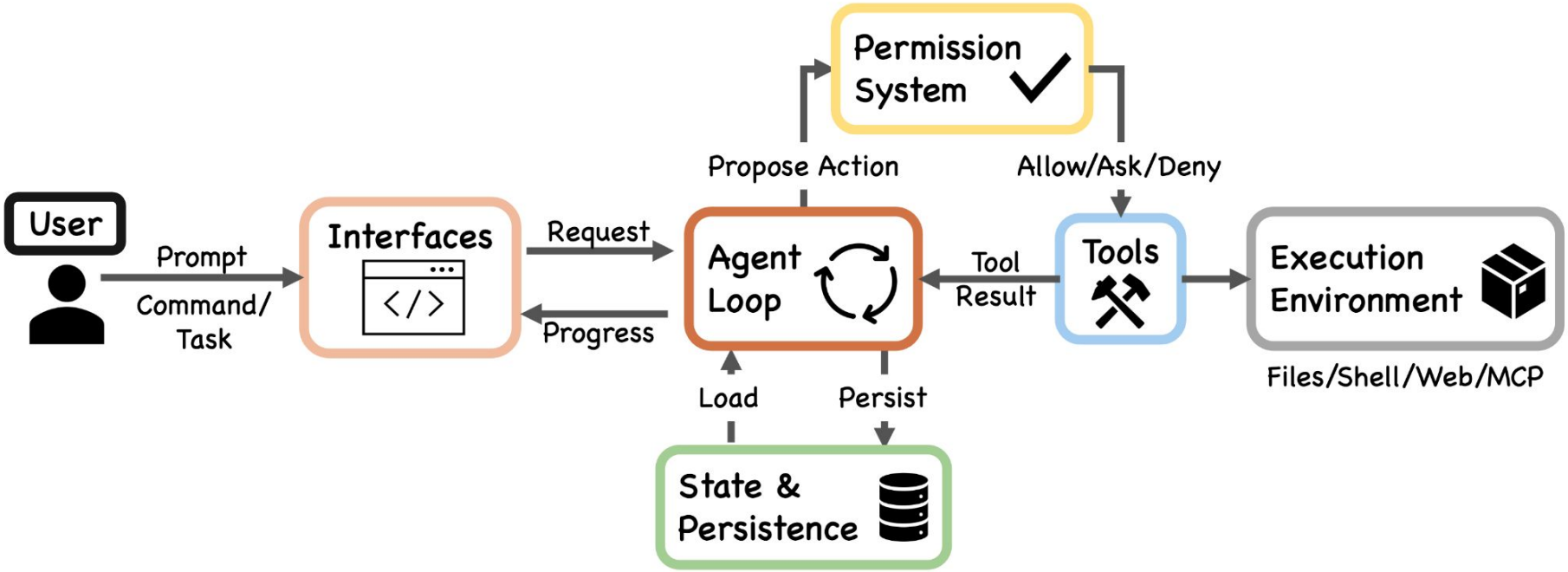
Tool calling

Harness engineering

Coding agents

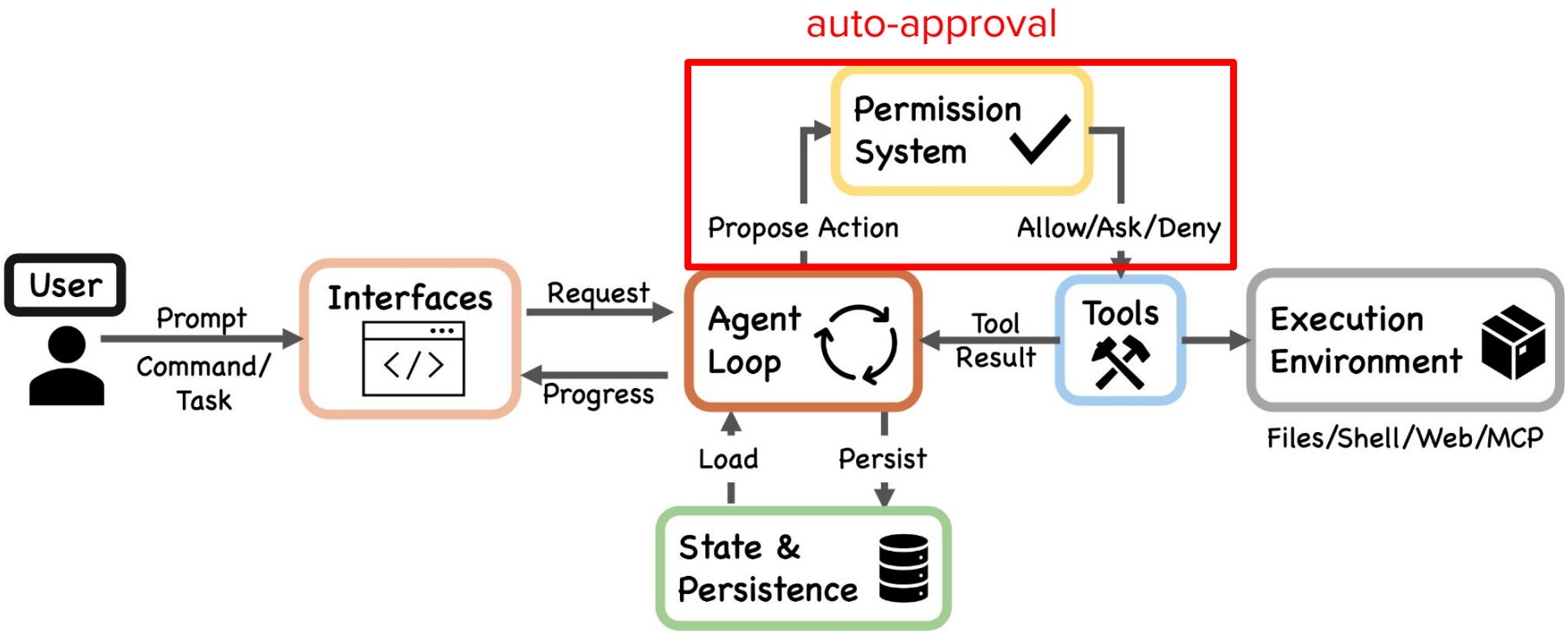
Discussion

Overview of a coding agent



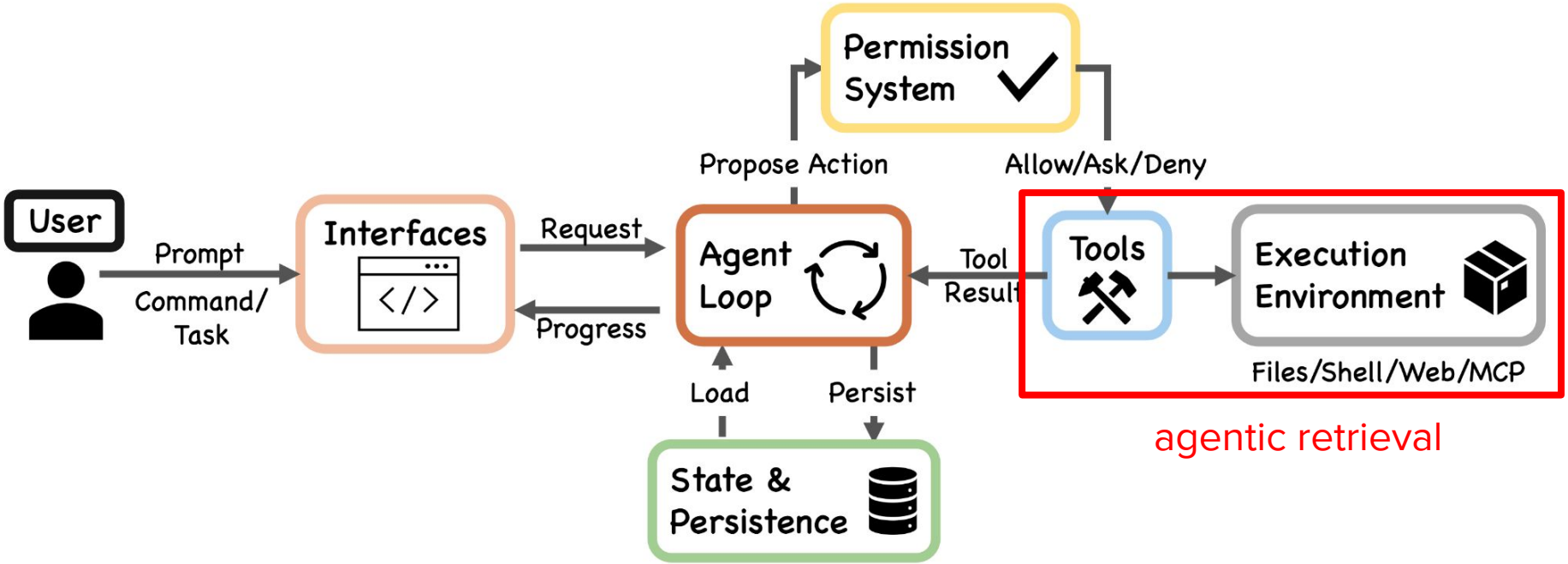
Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems, Liu et al., 2026.

Overview of a coding agent



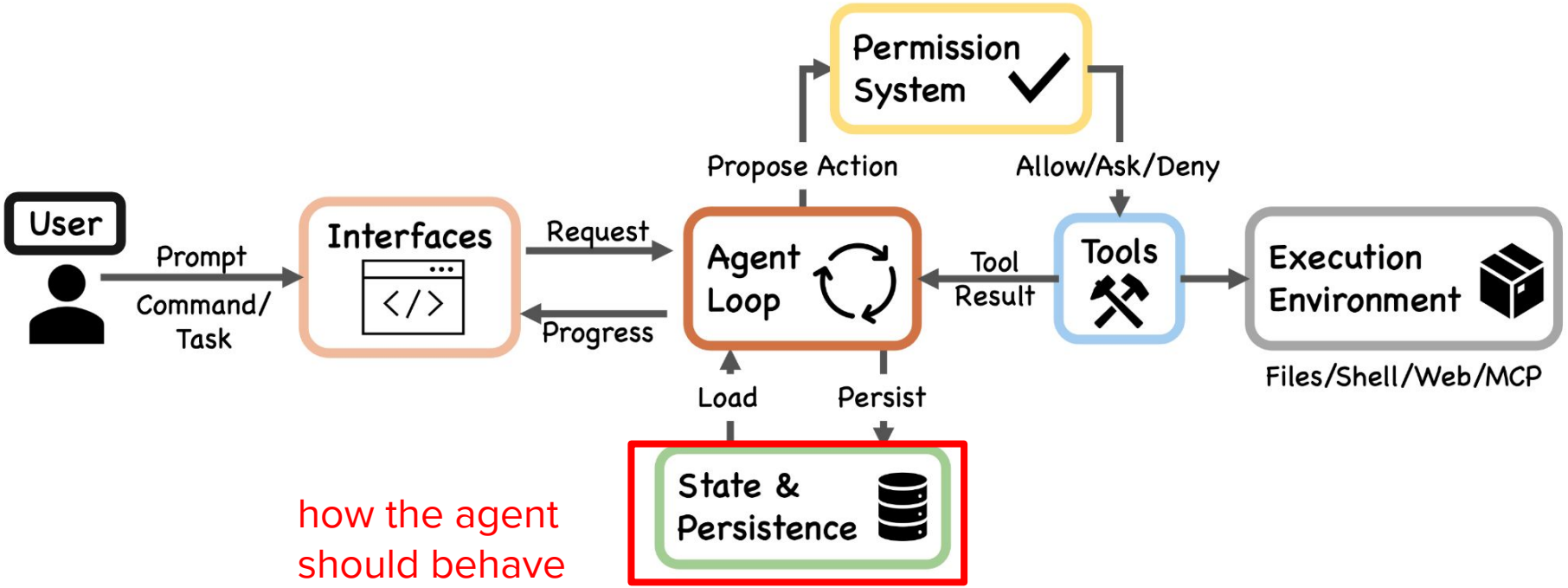
Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems, Liu et al., 2026.

Overview of a coding agent



Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems, Liu et al., 2026.

Overview of a coding agent



Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems, Liu et al., 2026.

AGENTS.md

Purpose. Defines how the agent should behave.

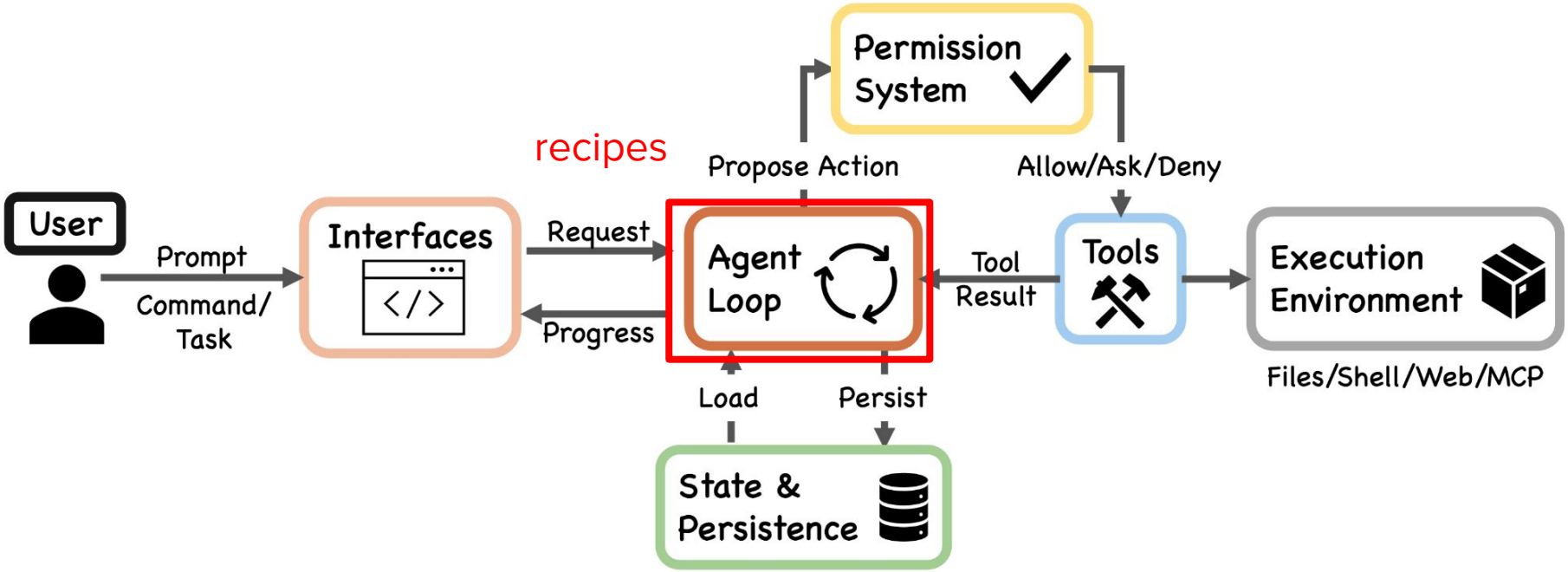
```
# Sample AGENTS.md file

## Dev environment tips
- Use `pnpm dlx turbo run where <project_name>` to jump to a package instead of scanning with `ls`.
- Run `pnpm install --filter <project_name>` to add the package to your workspace so Vite, ESLint, and TypeScript can see it.
- Use `pnpm create vite@latest <project_name> -- --template react-ts` to spin up a new React + Vite package with TypeScript checks ready.
- Check the name field inside each package's package.json to confirm the right name—skip the top-level one.

## Testing instructions
- Find the CI plan in the .github/workflows folder.
- Run `pnpm turbo run test --filter <project_name>` to run every check defined for that package.
- From the package root you can just call `pnpm test`. The commit should pass all tests before you merge.
```


Screenshot from AGENTS.md, August 14th, 2026.

Overview of a coding agent



Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems, Liu et al., 2026.

Skills

Purpose. Improves reusability of prompt/procedure

```
my-skill/
├── SKILL.md          # Required: metadata + instructions
├── scripts/          # Optional: executable code
├── references/       # Optional: documentation
├── assets/           # Optional: templates, resources
└── ...               # Any additional files or directories
```


Screenshot from Agent Skills, August 14th, 2026.

Today's main coding agents



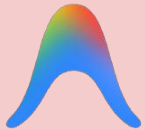
Claude Code



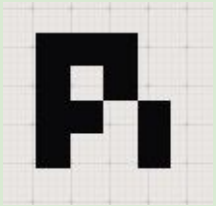
Codex



Cursor



Antigravity



Pi



Hermes Agent

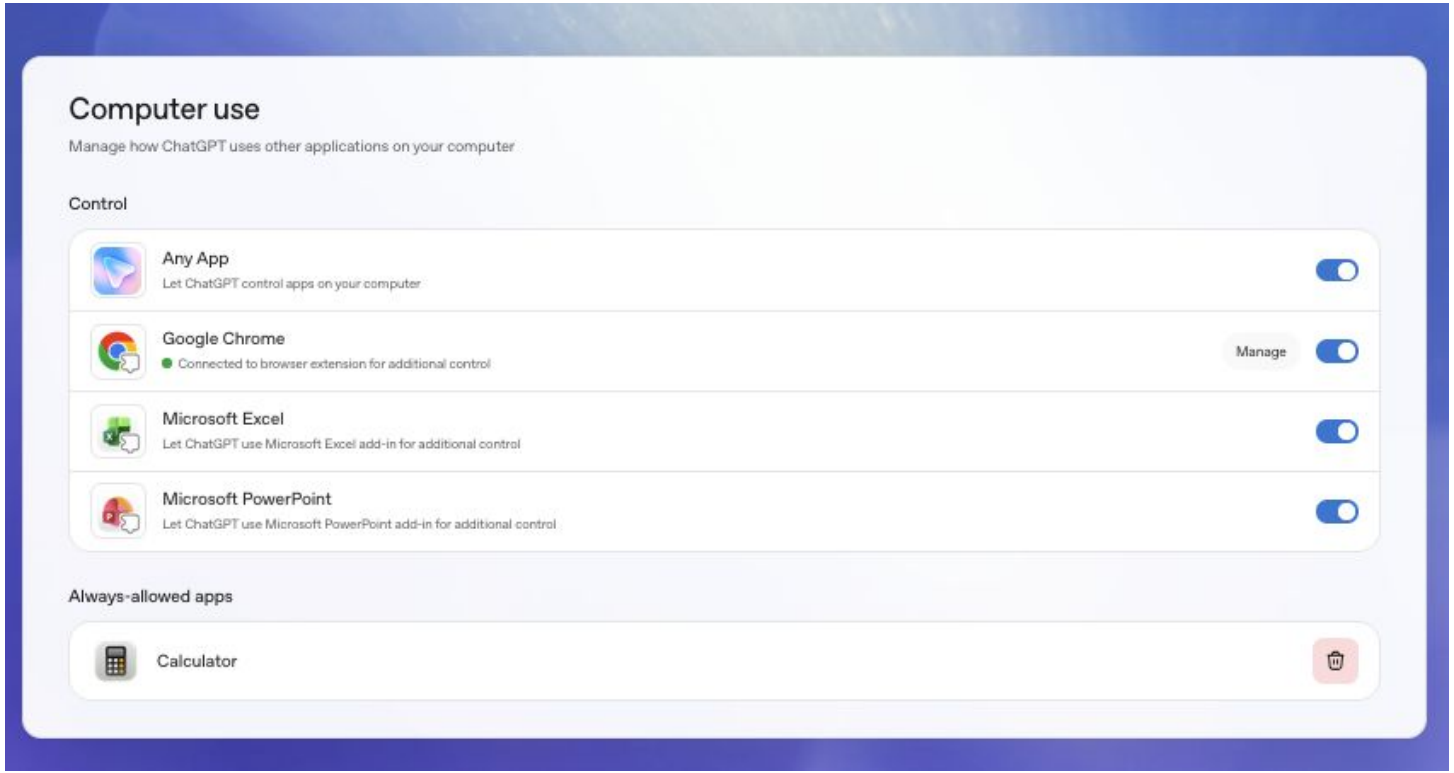


DSH

Open

Closed

Other developing use case: computer use



Computer Use, *OpenAI docs, 2026.*



Transformers & Large Language Models

Motivation

Retrieval

Tool calling

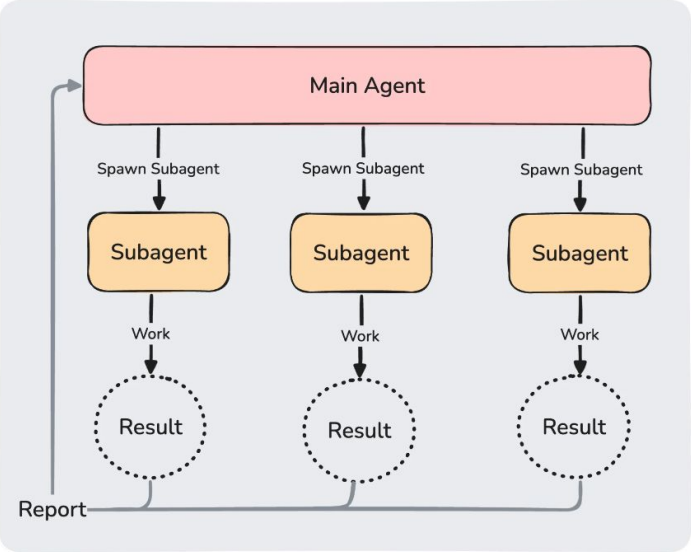
Harness engineering

Coding agents

Discussion

Orchestration patterns

Subagents



Subagents have specific task, no communication

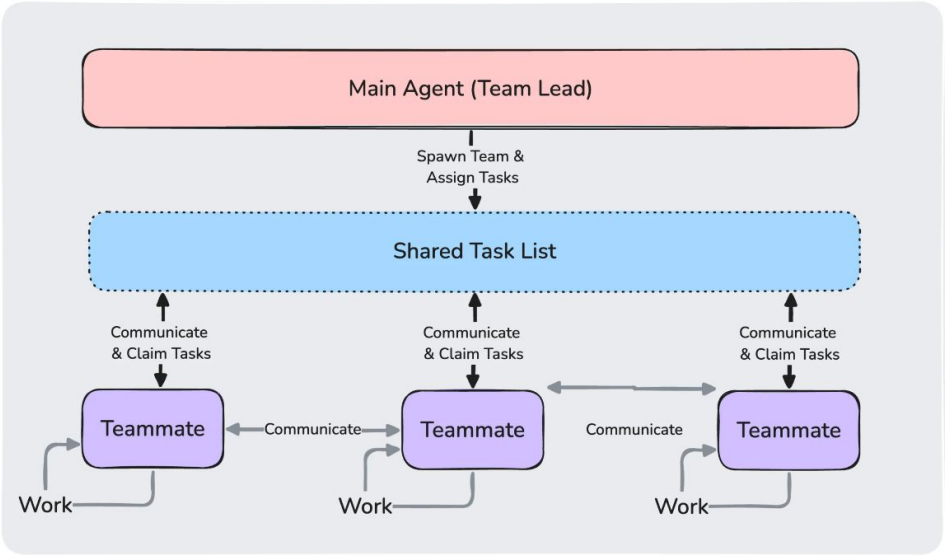
Orchestrate teams of Claude Code sessions, Claude Code docs.

Orchestration patterns

Subagents

Subagents have specific task, no communication

Agent Team



Subagents are "teammates"

Orchestrate teams of Claude Code sessions, *Claude Code docs.*

Wins...

September 8, 2026 Research Publication

On the Navier–Stokes Millennium Prize Problem

Read the paper Link to Lean formalized proof >

OpenAI announcement

...and vulnerabilities!

July 21, 2026 Security

OpenAI and Hugging Face partner to address security incident during model evaluation

OpenAI announcement

Suggested video: The OpenAI-Hugging Face incident, Black Hat conference, August 2026.
Suggested reading: METR report, August 2026.

Thank you for your attention!